Reversible modular arithmetic in Shor's algorithm

What it must do, how it is built, and how it runs — with verified Qiskit code

What this is, and how to read it

Shor's circuit is three layers: Hadamards, a ladder of controlled modular multiplications, and an inverse QFT. The first and third are drawn gate by gate everywhere. The middle one is almost always a box labelled "$\times a^{2^j} \bmod N$" — and it is over 99% of the circuit. This note opens that box twice: once as mathematics, once as running code.

Parts I–III	why the arithmetic is what it is: what phase estimation demands, why garbage is fatal, the tower from Toffoli up, and how a pile of permutations turns into a measurable phase.
Part IV	a complete, minimal, *verified* implementation — about 150 lines of Qiskit, with the tests that make it believable.
Part V	making it fast: ripple-carry arithmetic, windowing, unary-iteration lookups, measurement-based uncomputation, nested windows.
Part VI	reading somebody else's implementation, and what its bugs teach.
Parts VII–VIII	the other Shor: elliptic-curve discrete logarithms at cryptographic size, factoring at size, and from logical to physical qubits.
Part IX	the physics beneath it all: postulates to gates, how each qubit technology works, error correction and fault-tolerant gates, and the state of the art in 2026.

Every listing in Parts IV–V is copied from a file that was executed, and every number quoted as measured was measured. The implementation factors 15, 21 and 33 end to end; 21 and 33 are the honest tests, because their orders do not divide 2^t and so force the continued-fraction step to do real work.

Notation: N is the modulus, $n = \lceil \log_2 N \rceil$, a the base with $\gcd(a, N) = 1$, r its order, $t = 2n$ the counting register. Qubit k of a register holds bit k of its value, LSB first (Qiskit's convention).

Contents

Part I — Orientation **5**

1 Where the arithmetic sits **5**

1.1 What phase estimation asks for, and what it costs . . . 5
1.2 The four demands on U_a . . . 6
1.3 One circuit, two descriptions . . . 7
1.4 Why garbage is fatal, not untidy . . . 7
1.5 The zoom . . . 8

2 How to read these circuits 8

Part II — The tower 9

3 Level 0: the gate set 9

4 Level 1: addition 10
4.1 The recurrences . . . 10
4.2 The two blocks . . . 10
4.3 The matrices . . . 11
4.4 The CDKM ripple-carry adder . . . 11
4.5 Subtraction and comparison . . . 12

5 Level 2: modular addition by a classical constant 13

6 Level 3: modular multiplication 14

7 Level 4: the controlled multiplier, in place 15

8 Level 5: the ladder 16

Part III — Closing the loop 17

9 From permutations to phases 17

10 The bill 18

Part IV — Building it: a verified implementation 18

11 Level 0: the QFT 19

12 Level 1: addition by a classical constant 20

13 Level 2: modular addition — the hard one 21

14 Levels 3–4: multiplication, in place 22

15 Level 5: the ladder and the full circuit 23

16 The classical half 24

17 Verifying it — level by level 26

Part V — Making it fast 29

18 The other Level 1: ripple-carry arithmetic 30
18.1 Two blocks that do everything . . . 30
18.2 Adding a classical constant . . . 31
18.3 What it costs, measured . . . 32

19 Windowing: trading lookups for additions 32
19.1 The idea in three steps . . . 33
19.2 What has to be built . . . 33
19.3 What it costs, measured . . . 35

19.4 End to end . . . 35
19.5 Making the lookup cheap: unary iteration . . . 36
19.6 Uncomputing a lookup for free . . . 37
19.7 Nesting: windowing the exponent too . . . 38
19.8 What of the paper is actually here . . . 39

20 Emitting temporary ANDs **39**

21 One counting qubit: $4n + 2 \to 2n + 3$ **40**

22 Coset representation: deleting the modular adder **41**
22.1 The deviation law, measured . . . 42
22.2 What it saves . . . 43
22.3 Multiplying in the coset representation . . . 43

23 Where to go from here **45**

Part VI — Reading someone else's implementation **45**

24 The package at a glance **45**

25 Level 1 in code, twice **46**
25.1 The ripple-carry backend . . . 46
25.2 The Fourier backend . . . 47

26 Levels 2–4: written once, above the interface **49**
26.1 The modular adder . . . 49
26.2 The multiplier, out of place and then in place . . . 50

27 Level 5 and the two order-finding circuits **51**
27.1 One control qubit: the $2n + 3$ circuit . . . 52
27.2 The bill, as the package pays it . . . 52

28 Cross-check: factoring 15 from scratch **53**

29 Running it **54**

Part VII — The other Shor: elliptic-curve discrete logarithms **56**

30 What changes, and what does not **56**

31 Why it works **57**
31.1 The state, in four lines . . . 57
31.2 Two lattices . . . 58
31.3 The amplitude . . . 58
31.4 Reading off k, and what it costs . . . 59

32 The group law, and why it resists reversibility **59**

33 Level 2: modular arithmetic over $\mathbb{F}_p$ **60**

34 Level 3: inversion by Kaliski's algorithm **61**

35 Level 4: the eleven-step point addition **64**

36 Making it fast **65**
36.1 Level 1: which adder, and how to control it . . . 66
36.2 Level 0: temporary ANDs . . . 67
36.3 Level 2: approximate and pseudo-Mersenne reduction . . . 67
36.4 Level 3: Montgomery multiplication over a carry-save tree . . . 68
36.5 Level 3: dropping the counter and postponing the reduction . . . 69
36.6 Level 3, rebuilt: the Euclidean dialog . . . 70
36.7 Level 4, rebuilt: projective coordinates . . . 74
36.8 Level 5: windowing, zig-zag, and the semiclassical transform . . . 75
36.9 Implemented ECDSA.Fail backends and a measured toy frontier . . . 77

37 Everything at once **81**
37.1 Space at cryptographic size . . . 82

38 The 2026 refinements **83**
38.1 Uncomputing by measurement, and checking that it worked . . . 83
38.2 The GCD, five ways . . . 84
38.3 Squaring . . . 84
38.4 One point addition, at $n = 256$ . . . 85

39 At cryptographic size **85**
39.1 Building a 256-bit circuit on a laptop . . . 86
39.2 Matching IonQ's Toffoli count . . . 86
39.3 IonQ's count and IonQ's qubits, in one circuit . . . 88
39.4 SELECT-SWAP lookups . . . 89
39.5 Fewer qubits . . . 90
39.6 Depth . . . 92

40 The rest of the catalogues **92**
40.1 Carry-lookahead addition . . . 92
40.2 Projective coordinates with a quantum addend, and twisted Edwards curves . . . 93
40.3 Batch inversion . . . 93
40.4 The register-shared Euclidean algorithm . . . 94
40.5 Luo's point addition on three field registers . . . 94
40.6 Gidney's classical-quantum adder . . . 95
40.7 ECDSA.Fail's fire census . . . 97

41 Verifying it **97**
41.1 The distribution, and the control that makes it mean something . . . 99
41.2 Complete extended-Edwards arithmetic at toy size . . . 100
41.3 Kim Sec. 4.4: coherent three-constant lookup . . . 101
41.4 Dynamic dirty-ancilla modular addition in a toy affine oracle . . . 102

42 What is not here **103**

Part VIII — Factoring at size, and why elliptic curves fall first **105**

43 Gidney 2025, built **105**
43.1 What the circuit computes . . . 105
43.2 How it is checked . . . 105
43.3 At size . . . 106

44 From logical to physical qubits **106**
44.1 Choosing the code distance . . . 106
44.2 Counting physical qubits . . . 107

44.3 Counting time . . . 107
44.4 Two worked examples . . . 108
44.5 In code . . . 108

45 ECDSA against RSA **109**

Part IX — The physics beneath the circuits **110**

46 Quantum mechanics, as the circuits need it **111**
46.1 States, observables, and the Born rule . . . 111
46.2 Commutators, uncertainty, and why pulses must be slow . . . 111
46.3 Unitary evolution: from Hamiltonian to gate . . . 112
46.4 The driven two-level system . . . 112
46.5 Composite systems, entanglement, and two theorems . . . 113
46.6 Open systems: what an error is . . . 114
46.7 Measurement, fidelity, and the error budget . . . 115

47 How the qubits are built **115**
47.1 Superconducting circuits . . . 116
47.2 Trapped ions . . . 118
47.3 Neutral atoms . . . 119
47.4 Spins in silicon . . . 120
47.5 Photons . . . 120
47.6 Three others . . . 121
47.7 The platforms side by side, and what they feed into the estimate . . . 122
47.8 The challenges, platform by platform . . . 123

48 Errors, codes, and the price of a logical qubit **123**
48.1 Stabilisers: measuring parity, not data . . . 124
48.2 The stabiliser formalism, with the mathematics . . . 124
48.3 Threshold and the exponential . . . 125
48.4 Decoding, in real time . . . 126
48.5 Logical gates, and the one that costs . . . 127
48.6 Beyond the surface code . . . 128
48.7 Logical qubits, as demonstrated . . . 129

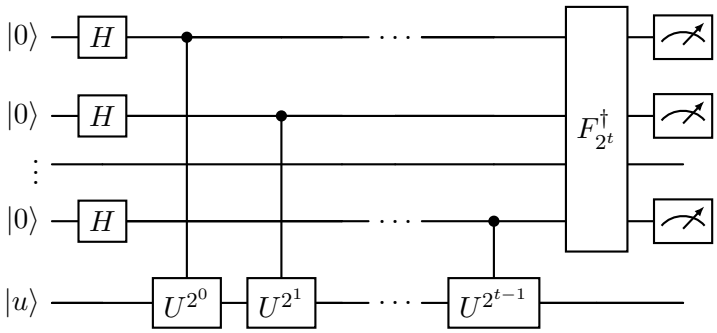
49 Where things stand **130**
49.1 Milestones . . . 130
49.2 What has actually been factored . . . 130
49.3 The resource-estimate landscape . . . 131
49.4 Forecasts . . . 131

Part I — Orientation

1 Where the arithmetic sits

1.1 What phase estimation asks for, and what it costs

Phase estimation [2, 3] is a general procedure. Give it a unitary U, an eigenvector $|u\rangle$ with $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, and t blank qubits, and it writes t bits of φ into those qubits. It is entirely indifferent to what U is.



But look at what it charges you. The rungs are $U^1, U^2, U^4, \ldots, U^{2^{t-1}}$, so building them by repetition costs

$$1 + 2 + 4 + \cdots + 2^{t-1} = 2^t - 1$$

applications of U — exponentially many. **Phase estimation is efficient only for unitaries whose powers are cheap.** For a generic U — a black box, a Hamiltonian evolution e^{iHt} — there is no shortcut, and the procedure is useless. This is worth saying plainly because the usual presentation, which introduces QPE abstractly and then "applies" it to U_a, makes the application look routine. It is not routine; it is the whole trick.

Modular multiplication is special twice over:

Gift 1: the powers collapse. $U_a^{2^j} = U_{a^{2^j} \bmod N}$. A high power of U_a is not a long circuit — it is *the same circuit with a different classical constant*, and that constant is computed classically by repeated squaring before the machine is switched on. This turns $2^t - 1$ applications into t.

Gift 2: one rung is cheap. U_a itself has a circuit of size $O(n^2)$. This is Part II.

Both are needed, and neither implies the other. Gift 1 without Gift 2 would be a short ladder of unaffordable rungs. Gift 2 without Gift 1 would be an affordable rung repeated an unaffordable number of times. Shor's algorithm [1] exists in the narrow space where arithmetic hands you both.

1.2 The four demands on U_a

Phase estimation does not accept just any circuit that computes $ay \bmod N$. Four properties are load-bearing, and each one is discharged at a specific level of the tower.

Demand	Why	Discharged at
Unitary	QPE's whole framework is spectral; a non-unitary map has no eigenphase to read.	Level 4 (§7)
Cheap powers	otherwise the ladder is exponentially long (§1.1).	Level 5 (§8)
Controllable	an *uncontrolled* U contributes a global phase, which no measurement can see; the control is what makes the phase relative.	Level 4 (§7)
Garbage-free	retained scratch entangles with the counting register and destroys the interference the inverse QFT depends on.	Levels 1–2 (§4, §5)

Unitarity is a number-theoretic condition. The map $y \mapsto ay \bmod N$ is injective on $\{0, \ldots, N-1\}$ precisely when a is invertible mod N, i.e. when $\gcd(a, N) = 1$. If that fails, the map collapses distinct inputs together, is not a permutation, and is not unitary — so there is no operator, no spectrum, and nothing for QPE to estimate. (In the algorithm this is checked classically first, and if it fails you have already factored N by computing the gcd.) The same condition returns at Level 4 in a second role: $\bar{a} = a^{-1} \bmod N$ is what makes the multiplier realisable *in place*. One hypothesis, two jobs.

1.3 One circuit, two descriptions

Here is the conceptual hinge of the whole subject, and the reason a note about *classical* arithmetic belongs inside a *quantum* algorithm.

The ladder is a single unitary. Written in the computational basis it says

$$|x\rangle|y\rangle \;\longmapsto\; |x\rangle\,|a^x y \bmod N\rangle \qquad \text{— it tabulates the function } f(x) = a^x \bmod N, \tag{1}$$

which is how Shor described it in 1994 [1]: build a periodic function, Fourier-sample its period. Written in the eigenbasis of U_a the very same unitary says

$$|x\rangle|u_s\rangle \;\longmapsto\; e^{2\pi i x s/r}|x\rangle|u_s\rangle \qquad \text{— it kicks an eigenphase onto the control register,} \tag{2}$$

which is Kitaev's [2] and CEMM's [3] reading: phase estimation on U_a. The two are related by the change of basis $|1\rangle = r^{-1/2}\sum_s |u_s\rangle$, and they are descriptions of one circuit, not two algorithms.

Why this matters for the arithmetic

Reading (1), the ladder does nothing but arithmetic — and indeed everything in Part II is a permutation matrix with no complex number anywhere in it. Reading (2), the same gates manufacture the phases that the algorithm ultimately measures.

Nothing was added in between. The phases are not written by any gate; they appear because a classical permutation was applied to a *superposition* rather than to a number, and then interfered. That is the single thing the classical circuit — from which every gate in Part II is borrowed — could not do.

1.4 Why garbage is fatal, not untidy

This is the demand that most shapes the construction, so it deserves the argument rather than the slogan. Suppose some ancilla ends in a state $|g_x\rangle$ that depends on which branch of the superposition it is in. Then instead of the clean state the algorithm wants, you have

$$\frac{1}{\sqrt{2^t}}\sum_x |x\rangle\,|a^x \bmod N\rangle\,|g_x\rangle.$$

The inverse QFT works by making branches x and x' interfere. Their interference term carries a factor $\langle g_{x'}|g_x\rangle$. If the garbage records anything about x — and for a useful computation it always does — those states are orthogonal, the factor is 0, and there is no interference at all. The measurement returns uniform noise. The circuit is still perfectly unitary; it simply computes nothing.

The consequence for every block below

You cannot erase scratch by overwriting it (that is not reversible) and you cannot discard it (it stays entangled). The only way out is to *run a computation backwards* and unwrite it. Hence:

- Level 1's adder ends with a reversed chain of UMA blocks that erase the carries;
- Level 2's modular adder spends three of its seven blocks recomputing and clearing one flag qubit;
- Level 4's multiplier runs a second multiplication *in reverse* purely to restore a scratch register to $|0\rangle$.

Roughly half of every circuit in Part II exists to undo the other half. That is not waste — it is the price of the interference, and §4 shows the undoing can be made to write the answer on its way out.

1.5 The zoom

Every box at one level is a whole circuit at the level below. Reading downwards is the plan of the document; the right-hand column is where the n^3 comes from. The last column names the method of the `qiskit-shor` package [26] that realises the level — Part IV reads those methods in full.

Level	Object	Made of	Multiplier	In code (§24)
5	the ladder	t rungs, one per counting qubit	$\times 2n$	`exponentiate_modulo`
4	$c\text{-}U_{a^{2^j}}$	$2\times$ CMULT and n Fredkins		`c_multiply_modulo`
3	CMULT(k)	doubly-controlled modular adders	$\times n$	`c_add_quantum_modulo`
2	ADD $^{\bmod N}(k)$	constant additions and sign tests	$\times 5$	`c_add_classical_modulo`
1	ADD(k)	MAJ chain, CNOT, UMA chain	$\times 2n$	`add_classical`
0	Toffoli	Clifford+T	$7T$; $4T$ if uncomputed	`MAJGate`, `UMAGate`

Three of those steps each multiply by roughly n — rungs, bits of the multiplicand, bits per addition — and that, and nothing subtler, is why the algorithm is cubic.

2 How to read these circuits

The single most misleading thing about circuit diagrams: **the horizontal lines are not wires and nothing flows along them.** There is no signal travelling rightwards. Each line is *one qubit*, and moving right means moving *forward in time*. The diagram is closer to sheet music than to a plumbing schematic: one staff line per qubit, read left to right.

So a diagram with eight lines has eight qubits and always will. Lines never split, merge, or get created. Whatever is written at the far left is the input; whatever is at the far right is the output. *Read those two edges first* — the middle is only the recipe for getting from one to the other.

Symbol	Meaning
filled dot •	a *control*: look at this qubit, change nothing
$\oplus$	a *target*: flip this qubit, if every dot on its vertical line is 1
box across several lines	one operation on those qubits jointly (its innards are drawn elsewhere)
vertical line	not a wire — visual glue saying "these symbols are simultaneous"
line labelled $\|x\rangle_{n+1}$	a *bundle*: $n{+}1$ qubits drawn as one line to keep the picture small

Everything in one vertical column happens at the same instant. When a control sits on a bundle, the caption must tell you *which* qubit of the bundle is the control — in the modular adder of §5 it is always the top one, the sign qubit.

Part II — The tower

3 Level 0: the gate set

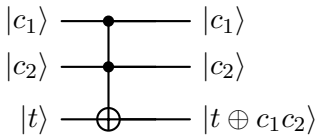
The conceptual point of this level. Every gate in Part II is a *permutation matrix*: one 1 per row and per column, everything else 0. Such a matrix is real and orthogonal, hence unitary, and it maps computational basis states to computational basis states. In other words the entire tower is a classical reversible circuit that happens to act linearly on superpositions. **No complex number appears anywhere below Level 5.** Keep §1.3 in mind: this is exactly why the phases of Shor's algorithm cannot be attributed to any gate.

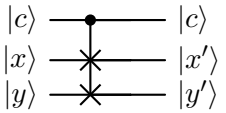
$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

$$\mathrm{TOF} = \begin{pmatrix} 1&0&0&0&0&0&0&0 \\ 0&1&0&0&0&0&0&0 \\ 0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&0&0 \\ 0&0&0&0&1&0&0&0 \\ 0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&0&1 \\ 0&0&0&0&0&0&1&0 \end{pmatrix}, \qquad \mathrm{FRED} = \begin{pmatrix} 1&0&0&0&0&0&0&0 \\ 0&1&0&0&0&0&0&0 \\ 0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&0&0 \\ 0&0&0&0&1&0&0&0 \\ 0&0&0&0&0&0&1&0 \\ 0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&0&1 \end{pmatrix}$$

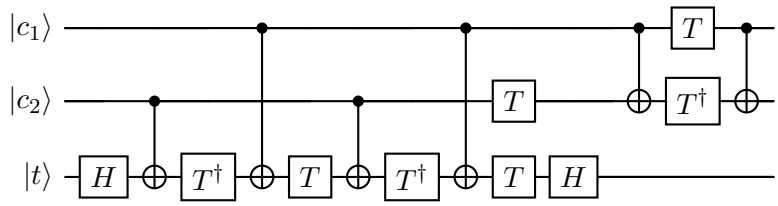
$$X|a\rangle = |a \oplus 1\rangle, \qquad \mathrm{CNOT}|c,t\rangle = |c,\, t \oplus c\rangle, \qquad \mathrm{TOF}|c_1,c_2,t\rangle = |c_1,c_2,\, t \oplus c_1c_2\rangle,$$

$$\mathrm{FRED}|c,x,y\rangle = |c,\, x \oplus c(x \oplus y),\, y \oplus c(x \oplus y)\rangle \quad (\text{swap } x,y \text{ iff } c=1).$$





The Toffoli is the *only* nonlinear gate. CNOT implements $\mathbb{F}_2$-linear maps and is therefore free in the fault-tolerant cost model; the Toffoli must be synthesised, and its standard Clifford+T decomposition costs 7 T gates and 6 CNOTs:



This is why gate counts in this subject are quoted in *Toffolis* rather than in gates: the Toffoli count is a proxy for the T count, and the T count is what magic-state distillation actually bills you for.

Remark (the halving that matters). A Toffoli that is *later uncomputed* — a *temporary logical-AND* [16] — costs only 4 T for the compute/uncompute *pair*, because the uncomputation can be done by

an X-basis measurement of the ancilla and a classically conditioned CZ, which is Clifford. By §1.4 nearly every Toffoli below is of exactly this kind, so this is not a corner case — it is the common case.

4 Level 1: addition

The problem this level solves. Adding two n-bit numbers is column-by-column addition: add the two digits and whatever carried in, write a digit, carry out. The carries are the difficulty, because column $i{+}1$ cannot start until column i has finished. Everything expensive about Shor's algorithm traces back to that chain.

4.1 The recurrences

With $c_0 = 0$,

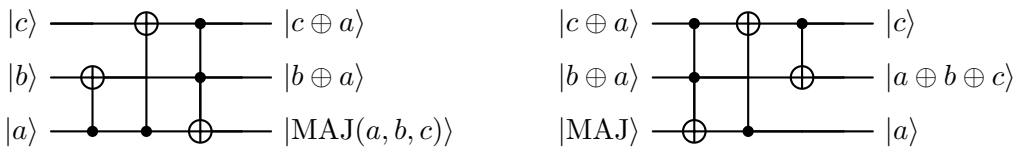
$$s_i = a_i \oplus b_i \oplus c_i, \qquad c_{i+1} = \mathrm{MAJ}(a_i, b_i, c_i) = a_i b_i \oplus a_i c_i \oplus b_i c_i, \qquad a + b = \sum_i s_i 2^i + c_n 2^n. \tag{3}$$

Read this as a cost statement. The sum bits are $\mathbb{F}_2$-*linear* given the carries, so they are CNOTs and therefore free. The carries are *quadratic*, so each one costs a Toffoli. And they form a chain, so the depth is $\Theta(n)$. Linear parts free, quadratic parts expensive, chain gives depth: that one line prices the algorithm.

4.2 The two blocks

On an ordered wire triple (c, b, a) define, following Cuccaro, Draper, Kutin and Moulton [9],

$$\begin{aligned} \mathrm{MAJ}: &\ \mathrm{CNOT}(a \to b),\ \mathrm{CNOT}(a \to c),\ \mathrm{TOF}(c, b \to a); \\ \mathrm{UMA}: &\ \mathrm{TOF}(c, b \to a),\ \mathrm{CNOT}(a \to c),\ \mathrm{CNOT}(c \to b). \end{aligned}$$



Lemma 4.1 (MAJ/UMA are partial inverses)**.**

$$\mathrm{MAJ}: (c, b, a) \mapsto \big(c{\oplus}a,\ b{\oplus}a,\ \mathrm{MAJ}(a, b, c)\big), \qquad \mathrm{UMA}: \big(c{\oplus}a,\ b{\oplus}a,\ \mathrm{MAJ}(a, b, c)\big) \mapsto \big(c,\ a{\oplus}b{\oplus}c,\ a\big).$$

Proof. For MAJ the only nontrivial step is the Toffoli, which sends $a \mapsto a \oplus (c \oplus a)(b \oplus a) = a \oplus \big(cb \oplus ca \oplus ab \oplus a\big) = ab \oplus ac \oplus bc$, using $a^2 = a$ and $a \oplus a = 0$ over $\mathbb{F}_2$. For UMA the same computation read backwards gives $\mathrm{MAJ}(a, b, c) \oplus (c \oplus a)(b \oplus a) = a$; then $c \oplus a \mapsto (c \oplus a) \oplus a = c$, and $b \oplus a \mapsto (b \oplus a) \oplus c$, which is the sum bit $s = a \oplus b \oplus c$. □

The idea, in one sentence

Look at what the second half of Lemma 4.1 says: UMA **erases the carry and writes the sum bit in the same three gates.**
By §1.4 the carries *must* be erased, so a reverse pass is compulsory. The design makes that compulsory pass do the useful work. The paper analogy is exact: when you add by hand you scribble little carry digits above the columns, and you are left with a page full of scribbles. Here you go back over the columns rubbing out each scribble, and the act of rubbing out deposits the answer.

4.3 The matrices

Ordering the basis $|c\,b\,a\rangle$ as $0,\dots,7$, the blocks are the permutations $\mathrm{MAJ} = (0)(1\,6\,7)(2)(3\,5)(4)$ and $\mathrm{UMA} = (0)(1\,7\,4\,6\,3\,5)(2)$:

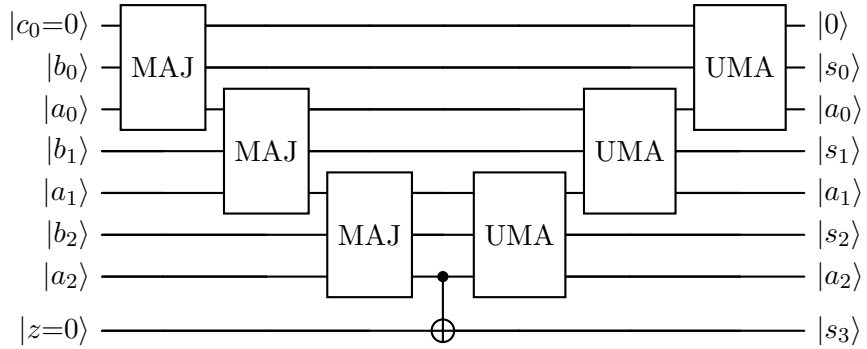
$$\mathrm{MAJ} = \begin{pmatrix} 1&0&0&0&0&0&0&0\\ 0&0&0&0&0&0&0&1\\ 0&0&1&0&0&0&0&0\\ 0&0&0&0&0&1&0&0\\ 0&0&0&0&1&0&0&0\\ 0&0&0&1&0&0&0&0\\ 0&1&0&0&0&0&0&0\\ 0&0&0&0&0&0&1&0 \end{pmatrix}, \qquad \mathrm{UMA} = \begin{pmatrix} 1&0&0&0&0&0&0&0\\ 0&0&0&0&0&1&0&0\\ 0&0&1&0&0&0&0&0\\ 0&0&0&0&0&0&1&0\\ 0&0&0&0&0&0&0&1\\ 0&0&0&1&0&0&0&0\\ 0&0&0&0&1&0&0&0\\ 0&1&0&0&0&0&0&0 \end{pmatrix}$$

(rows indexed by output, columns by input). Both are orthogonal, and $\mathrm{UMA}\cdot\mathrm{MAJ} : (c,b,a) \mapsto (c,\ a\oplus b\oplus c,\ a)$ — the permutation $0,3,2,1,6,5,4,7$, which is Lemma 4.1.

4.4 The CDKM ripple-carry adder

Why the wires look scrambled

The wire order is $c_0, b_0, a_0, b_1, a_1, \dots, z$ — alternating, not "all the a's then all the b's". This is the feature that makes the diagram look impenetrable, and it means *nothing mathematically*. It is a seating plan. Each block must touch three particular qubits; interleave them this way and those three are always *adjacent*, so the blocks are compact rather than a tangle of long-range connections. Re-order the wires and the circuit is identical, only uglier.



Theorem 4.2 (CDKM adder [9])**.** *The circuit above implements*

$$|c_0{=}0\rangle\,|b\rangle\,|a\rangle\,|z\rangle \;\longmapsto\; |0\rangle\,|(a+b) \bmod 2^n\rangle\,|a\rangle\,|z\oplus c_n\rangle$$

using $2n$ Toffoli gates, $4n{+}1$ CNOTs and $2n{+}2$ qubits, with the operand a preserved, addition in place on b, and ***zero garbage****. Fault-tolerantly it costs $4n$ T gates, each carry being a temporary AND [16].*

A run, in full: $a = 5$, $b = 3$

after	c_0	b_0	a_0	b_1	a_1	b_2	a_2	z
start	0	**1**	1	**1**	0	**0**	1	0
MAJ(c_0, b_0, a_0)	1	0	**1**	1	0	0	1	0
MAJ(a_0, b_1, a_1)	1	0	1	1	**1**	0	1	0
MAJ(a_1, b_2, a_2)	1	0	1	1	0	1	**1**	0
CNOT $a_2 \to z$	1	0	1	1	0	1	1	**1**
UMA(a_1, b_2, a_2)	1	0	1	1	1	**0**	1	1
UMA(a_0, b_1, a_1)	1	0	1	**0**	0	0	1	1
UMA(c_0, b_0, a_0)	**0**	**0**	1	0	0	0	1	1

Read the b column downwards: it starts as $011 = 3$ and ends as 000, with $z = 1$ — that is $1000_2 = 8$. The a wires end at $1, 0, 1 = 5$, exactly as they started, and c_0 is back to 0. Nothing is left dirty, which is what Theorem 4.2 promises and §1.4 demands.

The bold entries on the way down are the carries being deposited. Notice that a_1 shows 1 after its own block and then drops to 0 after the *next* one — because each MAJ also XORs into the wire below it. **The only wire holding a clean, unmodified carry at any moment is the topmost one, a_2, immediately after the last** MAJ. That is exactly where the single CNOT sits. The placement is not arbitrary: it is the one instant at which the final carry is sitting in the clear.

Verified

The circuit was built as a literal CNOT/TOF list and applied to all 2^{2n+2} basis states for $n = 2, 3, 4, 5$: the map is a permutation (reversible), the gate counts are exactly $2n$ Toffoli and $4n{+}1$ CNOT, and $\big(b, a, c_0, z\big) \mapsto \big((a{+}b) \bmod 2^n,\ a,\ 0,\ z \oplus c_n\big)$ holds for every input pair. The trace above is machine-generated.

Remark (where the ancillas went). CDKM was a genuine economy. The first reversible adders — Vedral–Barenco–Ekert's [4], and the CARRY/SUM networks Draper [7] reviews before departing from them — kept every carry in its own ancilla: $3n$ qubits to add two n-bit numbers. The MAJ trick stores each carry *in place of* an operand bit it has already consumed, and $3n$ drops to $2n + 2$. Draper's own answer to the same problem is more radical still — no carries at all — and is the Fourier route of §25.2.

4.5 Subtraction and comparison

Running Theorem 4.2 *backwards* gives SUB: $|b\rangle|a\rangle \mapsto |b - a \bmod 2^n\rangle|a\rangle$.

For comparison, the trick is to make the answer free rather than to build a comparator. Run the MAJ chain on a and $\bar{b}$ (bitwise complement) with carry-in 1: the register then holds $a + \bar{b} + 1 = a - b + 2^n$, whose carry-out satisfies

$$c_n = [\, a \geq b \,]. \tag{4}$$

Copy it with one CNOT and reverse the chain: n compute/uncompute Toffoli pairs, $4n$ T. Equivalently — and this is the form Level 2 uses — work one bit wider than you need, in two's complement, and the *top bit* of $x - y$ *is* the predicate $[\, x < y \,]$. **Comparison therefore costs nothing beyond the subtraction you were doing anyway**; you simply look at a qubit that is already there. That single sign bit is the workhorse of the whole level above.

Remark (compile-time constants). Every addition inside Shor's algorithm has one operand *classically known*: the constants $2^i a \bmod N$ and the modulus N. Specialising the circuit to a fixed operand

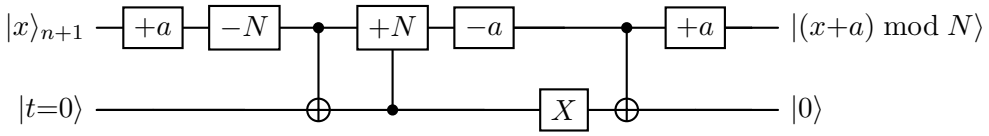
deletes or fixes roughly half the gates before emission — controls on known-0 bits vanish, controls on known-1 bits become unconditional. See the box in §6 for what this does and does not buy.

5 Level 2: modular addition by a classical constant

The problem this level solves, in one sentence: you want $(x + a) \bmod N$, which classically is "add them, and *if* the result is at least N, subtract N" — and a quantum computer cannot do *if*.

It cannot branch, because in general it is holding a superposition in which some branches overshot and some did not; there is no single answer to "did it overshoot?". So the conditional must be turned into arithmetic: compute the condition into a qubit, then use that qubit as a control. Blocks (1)–(4) below do exactly that, and if you could stop there the arithmetic would be finished. **Blocks (5)–(7) exist entirely to destroy the evidence**, for the reason in §1.4.

Theorem 5.1 (Beauregard's modular adder [8])**.** *For $0 \le a < N$ the seven blocks below implement $|x\rangle \mapsto |(x + a) \bmod N\rangle$ for all $0 \le x < N$, using five constant additions and one ancilla returned to $|0\rangle$. The value register is $n{+}1$ qubits wide so that intermediates up to $2N$ fit.*



Left to right, the columns are blocks (1)–(7) of the table below (block (6) is the X together with the CNOT that follows it). The two vertical lines into t are CNOTs *from the sign qubit* — the top bit of the $(n{+}1)$-qubit value register — and block (4) is the addition of N controlled on t.

	block	value register	t
(1)	ADD(a)	$x + a$ ($< 2N$, fits)	0
(2)	SUB(N)	$x + a - N$, sign bit $= [\, x + a < N \,]$	0
(3)	CNOT sign $\to t$	$x + a - N$	$[\, x + a < N \,]$
(4)	ADD(N) controlled on t	$R := (x + a) \bmod N$	unchanged
(5)	SUB(a)	$R - a$, sign bit $= [\, R < a \,]$	unchanged
(6)	X on t; CNOT sign $\to t$	$R - a$	**0**
(7)	ADD(a)	R	0

Proof. Blocks (1)–(2) form $x+a-N$, which is negative exactly when $x+a < N$; in two's complement the sign qubit *is* that predicate, so (3) records it and (4) restores $R \in [0, N)$. The remaining question is how to clear the ancilla, and the answer is that the flag is recomputable from the output alone. Since $x < N$,

$$t = 1 \iff x + a < N \iff R = x + a \ge a, \qquad t = 0 \iff R = x + a - N < a,$$

so $[\, R < a \,] = \neg t$. Block (5) forms $R - a$, whose sign qubit is therefore $\neg t$; the X in (6) maps $t \mapsto \neg t$ and the following CNOT gives $\neg t \oplus \neg t = 0$. Block (7) restores R. □

The idea: deduce the flag, don't remember it

After block (4) you hold the right answer but you are stuck with a qubit remembering whether you wrapped around. You may not erase it and you may not discard it.
The way out is that **you do not need to remember, because you can work it out again from the answer**. Since x started below N, wrapping happened exactly when $R < a$. So block (5) recomputes the flag from the output and block (6) XORs it against itself.
It is like this: you cannot remember whether you borrowed during a subtraction, but you can look at your answer and deduce that you must have — and then honestly forget. **Every block in this tower ends by recomputing its own flag from its own output.** That single discipline is what keeps the whole construction garbage-free.

The precondition is load-bearing

The proof uses $x < N$ twice. Fed a value $\geq N$ the circuit silently leaves t dirty and the value register wrong by N — and by §1.4 a dirty t does not produce a wrong answer, it produces *no* answer, since the interference downstream dies.
The invariant holds because *every* adder in the tower reduces mod N (§6). Note that this is the same hypothesis that licenses the 3-of-7 control saving in §7: the uncontrolled blocks compose to the identity only when $b < N$. One precondition, two uses.

Remark (traced, $N = 15$). $x = 3$, $a = 7$: 3+7=10 → 10−15= − 5 (sign 1) → t=1 → +15 → R=10 → 10−7=3 (sign 0) → $t = 1 \oplus 1 \oplus 0 = 0$. $x = 9$, $a = 7$: 9+7=16 → 16−15=1 (sign 0) → t=0 → R=1 → 1−7= − 6 (sign 1) → $t = 0 \oplus 1 \oplus 1 = 0$. Both branches: correct output, ancilla clean.

6 Level 3: modular multiplication

The problem this level solves. You have an adder; you need a multiplier. The bridge is the schoolbook one — reversible shift-and-add multiplication goes back to Vedral, Barenco and Ekert [4] — and each shifted multiple is a *classically known constant*. For $x = \sum_i x_i 2^i$,

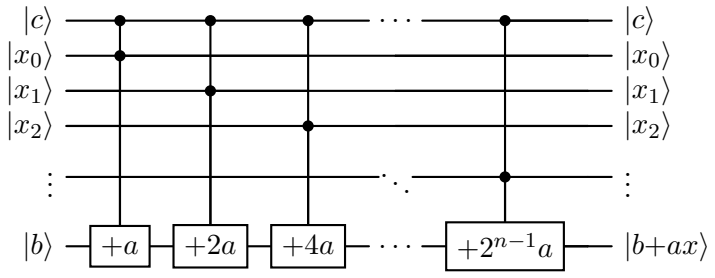
$$a\,x \bmod N = \Big(\sum_{i=0}^{n-1} x_i \left(2^i a \bmod N\right) \Big) \bmod N, \tag{5}$$

i.e. binary long multiplication written reversibly. The bit x_i decides whether its multiple joins the sum, so it becomes a *control*: the data has turned into conditioning.

Theorem 6.1.

$$\mathrm{CMULT}(a):\ |c\rangle|x\rangle|b\rangle \ \mapsto\ |c\rangle|x\rangle|\,b + c\cdot ax \bmod N\,\rangle \qquad \textit{is} \qquad \prod_{i=0}^{n-1} \mathrm{ADD}_{c,\,x_i}^{\bmod N}\big(2^i a \bmod N\big),$$

n doubly-controlled modular additions of compile-time constants.



Each box is one Level-2 modular adder for the constant $2^i a \bmod N$, doubly controlled by the outer control c and the bit x_i. Every partial sum is reduced mod N, which is what maintains the invariant $b < N$ that Theorem 5.1 requires.

Proof. Immediate from (5): the i-th adder fires exactly when $x_i = 1$, and each adder reduces, so all partial sums stay below N. □

The constant is known; the operand is not

It is tempting to think that because a is chosen in advance, the multiplication is "already done". But a multiplication has *two* operands, and precomputation gives you one:

$$\underbrace{a}_{\text{known at compile time}} \times \underbrace{x}_{\text{unknown — it is the register}} \bmod N.$$

The register holds $a^x \bmod N$ for a counting register in superposition over all 2^t values of x. If you knew which x mattered you would compute $a^x \bmod N$ on a laptop and never build the machine. The machine exists precisely because it evaluates at every input at once — so every multiplication in the ladder is fed a *superposition*, and must be a circuit that maps all inputs correctly and simultaneously. For $N = 15$, $a = 7$ the first rung must realise

$$1\to 7, \quad 2\to 14, \quad 4\to 13, \quad 7\to 4, \quad 8\to 11, \quad 11\to 2, \quad 13\to 1, \quad 14\to 8$$

in one shot. Precomputing the whole input/output table instead would need 2^n rows: about 10^{616} for RSA-2048, against 10^{80} atoms in the observable universe. The constant, by contrast, is 2048 bits — 256 bytes. That gap is exactly the gap between what precomputation gives you and what still has to be built.

What precomputation *does* buy is real and is used everywhere: the rung constants (§8), the shifted multiples $2^i a \bmod N$ above, the inverse $\bar{a}$ (§7), and the roughly half of all gates that can be deleted because one adder operand is classical.

Remark ($N = 15$, $a = 7$). The four constants are $2^i \cdot 7 \bmod 15 = 7, 14, 13, 11$. For $x = 3 = 0011_2$ the first two adders fire: $7 + 14 = 21 \equiv 6 \pmod{15}$, and indeed $7 \cdot 3 = 21 \equiv 6$.

7 Level 4: the controlled multiplier, in place

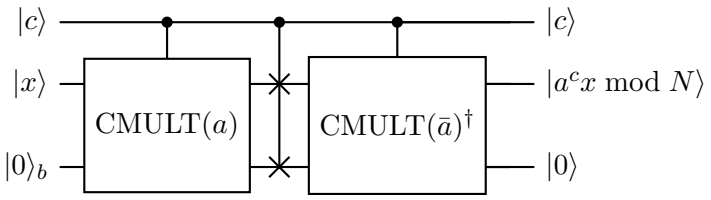
The problem this level solves. CMULT writes its answer into a *fresh* register, but the ladder must apply many multiplications one after another to the *same* register. A circuit that consumed a clean register per rung would need thousands of registers. So the out-of-place multiplier has to be converted into an in-place map $|x\rangle \mapsto |ax \bmod N\rangle$, and the leftover input x has to disappear — which, by §1.4, means it must be *computed away*, not discarded.

The escape is that a is invertible. You can uncompute x from the product. The construction is Beauregard's [8].

Theorem 7.1. *Let* $\gcd(a, N) = 1$ *and* $\bar{a} := a^{-1} \bmod N$ *(classical, extended Euclid). Then*

$$c\text{-}U_a \;=\; \mathrm{CMULT}(\bar{a})^\dagger \cdot c\text{-SWAP} \cdot \mathrm{CMULT}(a)$$

implements $|c\rangle|x\rangle|0\rangle \mapsto |c\rangle|a^c x \bmod N\rangle|0\rangle$ *for all* $0 \le x < N$.



Proof. Take $c = 1$ and follow the register pair:

$$|x\rangle|0\rangle \xrightarrow{\mathrm{CMULT}(a)} |x\rangle|ax\rangle \xrightarrow{\mathrm{SWAP}} |ax\rangle|x\rangle \xrightarrow{\mathrm{CMULT}(\bar{a})^\dagger} |ax\rangle|x - \bar{a}(ax)\rangle = |ax\rangle|0\rangle,$$

since $\bar{a}a \equiv 1 \pmod N$. For $c = 0$ every factor is the identity. Both branches leave the scratch in $|0\rangle$, unentangled, so $c\text{-}U_a$ is realised exactly. □

The controlled SWAP is n Fredkin gates sharing one control. Note that no controlled QFT appears and the constant $\bar{a}$ carries no control — its inverse is classical. For $N = 15, a = 7$: $\bar{a} = 13$, since $7 \cdot 13 = 91 = 6 \cdot 15 + 1$.

Where the outer control goes, and why not everywhere

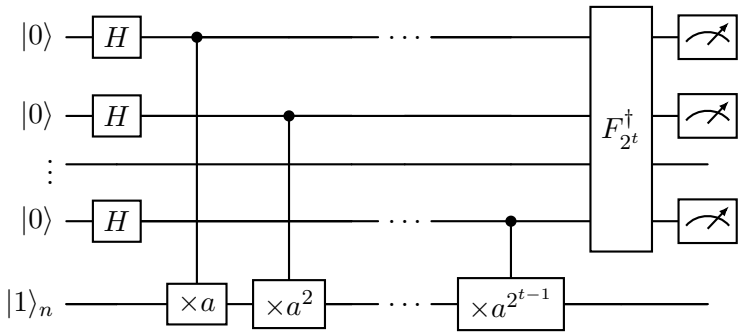
Let $V = G_L \cdots G_1$ and let V_S replace G_j by $c\text{-}G_j$ for $j \in S$. Since $V_S = |0\rangle\langle 0| \otimes R_S + |1\rangle\langle 1| \otimes V$ with $R_S = \prod_{j \notin S} G_j$, we have $V_S = c\text{-}V$ on a subspace W *iff* $R_S = \nVdash$ on W. In words: **a gate needs the control only if leaving it uncontrolled would not cancel out**. Two patterns exhaust the uses:

- *Conjugation.* If $V = ABA^\dagger$ then controlling B alone suffices, since $AA^\dagger = \nVdash$. A conjugating layer never needs the control.
- *Compensating residue.* In Theorem 5.1 only blocks (1), (5), (7) take the outer control: with the control off, blocks (2), (3), (4), (6) conspire to do nothing — $b - N$ goes negative, so t is set, block (4) adds N back, and block (6) clears t. This works only because $b < N$, the same precondition as before.

Measured on $N = 15$, $a = 7$, transpiled to $\{\mathrm{CNOT}, U\}$: the modular adder costs 2 634 CNOTs at depth 4 836 if every gate is controlled, against 206 / 354 for the 3-of-7 construction (12.8×); the full $c\text{-}U_a$, 20 892 / 37 727 against 1 748 / 2 957 (12.0×). An order of magnitude, from one lemma. Both patterns reappear, letter for letter, in the code of Part IV (§25, §26).

8 Level 5: the ladder

What this level supplies. Gift 1 of §1.1. The rungs use classically precomputed constants $c_j = a^{2^j} \bmod N$, obtained by repeated squaring $c_0 = a$, $c_{j+1} = c_j^2 \bmod N$ — this is the identity $U_a^{2^j} = U_{a^{2^j} \bmod N}$, which is why the ladder has t rungs and not 2^t.



Each $\times a^{2^j}$ box is one c-U_a of Theorem 7.1, hence $2n$ modular adders (Theorem 6.1) and n Fredkins; each modular adder is five constant additions (Theorem 5.1); each addition is the CDKM circuit (Theorem 4.2) at $2n$ Toffoli.

The ladder is not one circuit

Every rung uses a *different* constant, so the rungs are t structurally different circuits that happen to sit side by side. For $N = 15, a = 7$ the constants are $7, 4, 1, 1, 1, \ldots$ — so most of that ladder is literally the identity, which is why 15 is everyone's demo integer.

A consequence worth stating, because it surprises people: **the compiled circuit is specific to** a, and Shor's algorithm may need to retry with a fresh base. That is genuine, and it is cheap. Recompiling means repeated squaring plus one extended Euclid — $O(n^3)$ classical bit operations, measured at 50 ms for a 2048-bit modulus, against days of quantum runtime. And it is rarely needed: a bad measurement or a $\gcd(s, r) > 1$ failure is re-run on the *same* circuit, and only a bad base forces recompilation, which happens with probability at most $1/2$ for $N = pq$. Expected number of distinct circuits: about two.

Part III — Closing the loop

9 From permutations to phases

Now follow the state through, and watch classical arithmetic turn into an eigenphase. After the Hadamard layer,

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle|1\rangle.$$

The ladder is the only place any arithmetic happens, and by §8 it computes

$$|x\rangle|y\rangle \longmapsto |x\rangle \,\big|\, a^{\sum_j x_j 2^j} y \bmod N \,\big\rangle = |x\rangle|a^x y \bmod N\rangle, \qquad \text{so} \qquad \frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle.$$

Every gate that produced this was a permutation matrix with entries 0 and 1. Now rewrite the *same state* in the eigenbasis of U_a, using $|1\rangle = r^{-1/2} \sum_s |u_s\rangle$:

$$\frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle \;=\; \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \underbrace{\Big[\frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i x s/r} |x\rangle\Big]}_{\text{a phase ramp of frequency } s/r} \otimes |u_s\rangle.$$

The phases were not written by any gate. They are what the permutation *looks like* in the other basis — and they are visible only because the register was in a superposition, so that a permutation of basis labels became a relative phase between branches. The inverse QFT then reads the frequency of the ramp, and the measurement returns an approximation to s/r for a uniformly random s; continued fractions recover r [1].

The whole story in four lines

1. Phase estimation is cheap only when U has cheap powers; arithmetic makes U_a's powers cheap (Gift 1) *and* makes one power cheap to build (Gift 2).
2. Building it means reversible classical arithmetic — adders, a modular reduction, a shift-and-add multiplier, an in-place trick that needs a^{-1}.
3. Every scratch bit must be uncomputed, or the interference that reads the phase does not happen. Half of every circuit in Part II is that uncomputation.
4. Nothing in the arithmetic is complex-valued. The phases appear on the change of basis, and that change of basis is the algorithm.

10 The bill

$$\underbrace{2n}_{\text{rungs}} \times \underbrace{2n}_{\substack{\text{modular adds}\\ \text{per } c\text{-}U_a}} \times \underbrace{5}_{\substack{\text{additions per}\\ \text{modular add}}} \times \underbrace{2n}_{\substack{\text{Toffoli per}\\ \text{addition}}} \approx 40\,n^3 \text{ Toffoli},$$

which windowed arithmetic [21] plus the 3-of-7 control trick reduces to $\approx 0.3\,n^3$. Windowing is a *different* construction from the one built here — it replaces the controlled blocks by table lookups addressed by the exponent bits — and it is built and measured in Part V (§19).

At $n = 2048$ that is $\approx 2.6 \times 10^9$ Toffoli for the modular exponentiation against $\approx 10^5$ gates for the approximate QFT: **the Fourier layer, which carries all the conceptual content, is** 0.003% **of the circuit.** Everything that determines whether RSA falls is the cost of multiplying two 2048-bit numbers, reversibly, in superposition.

What was machine-checked

MAJ/UMA (Lemma 4.1) on all 8 inputs and as 8×8 orthogonal matrices; the Clifford+T Toffoli against the 8×8 permutation (7 T gates); the CDKM circuit (Theorem 4.2) as a literal gate list on all 2^{2n+2} basis states for $n = 2, 3, 4, 5$ — reversibility, gate counts, and arithmetic — and the $a = 5, b = 3$ trace; the comparator identity (4) for all pairs at $n = 4, 6$; the seven-block modular adder (Theorem 5.1) for all $x, a < N$ at $N = 15, 21, 33, 55$, checking both the output and that t returns to 0; the shift-and-add identity (5) and the conjugation of Theorem 7.1 for all $x < N$ and all a coprime to N at $N = 15, 21, 55$; and the ladder identity $|x\rangle|1\rangle \mapsto |x\rangle|a^x \bmod N\rangle$.

Part IV — Building it: a verified implementation

Parts I–III said what the arithmetic must do and why. This part does it: a complete order-finding implementation in about 150 lines of Qiskit, bottom up, with the test at each level that makes it

believable. It is self-contained — no circuit-library arithmetic, and the QFT built from scratch — and it takes the Fourier route, because that is the shortest correct path from nothing to a factored integer. Part V rebuilds the slow parts three different ways.

11 Level 0: the QFT

Everything in Parts IV and V needs only these imports — no Qiskit circuit-library primitives, no prebuilt arithmetic:

```
import math
from fractions import Fraction
import numpy as np
from qiskit.circuit import ClassicalRegister, QuantumCircuit, QuantumRegister
```

The one primitive everything rests on is the QFT, so build it rather than import it. It is

$$|j\rangle \longmapsto \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i\, jk/2^n} |k\rangle,$$

and the standard circuit follows from the product form of that sum: for each qubit, one Hadamard, then a controlled phase from every less significant qubit, at halving angles. A final layer of swaps reverses the qubit order, which the product form leaves reversed.

```
def qft(n, approx=0):
    """QFT on n qubits, Qiskit convention (qubit 0 = LSB):
        |j> -> 2^(-n/2) sum_k exp(2*pi*i*j*k/2^n) |k>.
    approx > 0 drops controlled rotations by less than pi/2^approx."""
    qc = QuantumCircuit(n, name="qft")
    for j in reversed(range(n)):
        qc.h(j)
        for k in range(j):
            if approx and (j - k) > approx:
                continue                    # angle too small to matter
            qc.cp(np.pi / 2 ** (j - k), k, j)
    for i in range(n // 2):                 # reverse the qubit order
        qc.swap(i, n - 1 - i)
    return qc.to_gate()
```

That is $n(n+1)/2$ gates plus $\lfloor n/2 \rfloor$ swaps: $O(n^2)$. The `approx` parameter is the approximate QFT of §23 — rotations by less than $\pi/2^d$ are below any hardware's resolution, and dropping them costs negligible error while cutting the count to $O(n \log n)$. It is one `continue`, so there is no reason not to have it.

Verify this before anything else

Do not test the QFT by eye. Build its unitary and compare it with the definition:

```
ref = np.array([[np.exp(2j*np.pi*j*k/2**n) for j in range(2**n)]
                for k in range(2**n)]) / np.sqrt(2**n)
assert np.allclose(Operator(qft(n)).data, ref)        # qiskit.quantum_info.Operator
```

Checked for $n = 1..6$ against both the definition and Qiskit's own `QFTGate`: they agree exactly, and `qft(n).inverse()` undoes it. If you drop the swap layer, or reverse the loop order, you get a QFT of the bit-reversed integer — which is still unitary, still looks fine, and will make every addition below wrong. This is the single most common place to lose an afternoon.

12 Level 1: addition by a classical constant

Classically, addition means carries, and carries mean a chain of Toffolis and ancilla qubits. Draper's insight [7] is to avoid them entirely by working in the Fourier basis. Write the QFT of an n-qubit register as a product state:

$$F|y\rangle = \bigotimes_{i=0}^{n-1} \frac{|0\rangle + e^{2\pi i\, y\, 2^{\,i-n}}|1\rangle}{\sqrt{2}}.$$

Every qubit carries the whole value y — in the *phase*, at a different scale. Adding X means advancing each phase by the corresponding amount, so

$$\text{qubit } i \text{ gets } P\big(2\pi X\, 2^{\,i-n}\big), \qquad e^{2\pi i y 2^{i-n}} \longmapsto e^{2\pi i (y+X) 2^{i-n}},$$

and an inverse QFT lands back in the computational basis holding $|(y+X) \bmod 2^n\rangle$. No carries, no ancillas, and since X is classical every qubit needs exactly *one* phase gate, so the addition layer has depth 1.

```
def add_const(qc, X, y):
    """|y> -> |y + X mod 2^n> on the qubit list y (LSB first)."""
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.p(2 * np.pi * X * 2.0 ** (i - n), y[i])
    qc.append(qft(n).inverse(), y)

def c_add_const(qc, ctrls, X, y):
    """add_const controlled on every qubit in ctrls."""
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.mcp(2 * np.pi * X * 2.0 ** (i - n), ctrls, y[i])
    qc.append(qft(n).inverse(), y)
```

Three things to notice

Negative constants just work. $X = -N$ needs no separate subtractor: the phases are periodic, so $-N$ and $2^n - N$ give the same gates. Subtraction is addition of a negative Python integer, and §13 relies on this.

Only the phases take the control. In `c_add_const` the two QFT layers are *uncontrolled*. They form a conjugation $F^\dagger \cdots F$, so with the control off they cancel: controlling them would be pure waste. Keep this in mind — it is the same argument that saves an order of magnitude in §13.

The composite is a permutation. A single $P(\theta)$ is not, and intermediate states carry irrational phases, but $F^\dagger \cdot (\text{phases}) \cdot F$ is exactly the 0/1 permutation matrix $|y\rangle \mapsto |y + X\rangle$. Nothing above Level 1 can tell this apart from a ripple-carry adder.

13 Level 2: modular addition — the hard one

You want $(y + X) \bmod N$, which classically is "add, and *if* the result is $\geq N$, subtract N". A quantum computer cannot branch: it holds a superposition in which some branches overshot and some did not, so there is no single answer to "did it overshoot?".

So compute the condition into a qubit and use that qubit as a control. Give the value register one extra qubit (the *sign* qubit) so intermediates up to $2N$ fit and two's-complement negatives are readable: after subtracting, the top bit *is* the predicate. Then:

	block	value register	flag t
(1)	$+X$ (controlled)	$y + X$ $(< 2N$, fits)	0
(2)	$-N$	$y + X - N$, sign bit $= [\, y + X < N \,]$	0
(3)	CNOT sign $\to t$	unchanged	$[\, y + X < N \,]$
(4)	$+N$ if t	$R := (y + X) \bmod N$	unchanged
(5)	$-X$	$R - X$, sign bit $= \neg t$	unchanged
(6)	clear t from that sign bit	$R - X$	**0**
(7)	$+X$	R	0

Blocks (1)–(4) do the arithmetic. **Blocks (5)–(7) exist entirely to destroy the evidence**, and they are the interesting half. After (4) you hold the right answer but one qubit remembers whether you wrapped — which you may neither erase nor discard.

The escape: *you do not need to remember, because you can deduce it from the answer.* Since $y < N$, wrapping happened exactly when $R \geq X$. So block (5) recomputes the flag from the output and block (6) XORs it against itself. Concretely, $t = 1 \iff y + X < N \iff R = y + X \geq X$, so $[\, R < X \,] = \neg t$, which is the sign bit after (5); XOR that into t and apply X to finish.

```
def c_add_mod(qc, ctrls, X, y, sign, flag, N):
    """|y> -> |(y + X) mod N> when all ctrls are 1; identity otherwise.
    Needs 0 <= X < N, 0 <= y < N, sign and flag in |0>."""
    yw = list(y) + [sign]                    # (n+1)-wide value register
    c_add_const(qc, ctrls, X, yw)            # (1) + X          (controlled)
    add_const(qc, -N, yw)                    # (2) - N
    qc.cx(sign, flag)                        # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)           # (4) + N  if flag
    add_const(qc, -X, yw)                    # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)       # (6) clear flag from the answer
    qc.x(flag)
```

```
    add_const(qc, X, yw)                    # (7) + X
```

Where the control goes — the biggest constant factor in the circuit

Notice what is *not* controlled. Blocks (2)–(5) and (7) are plain, and inside every `add_const` the QFT layers are plain too. Only block (1), block (4)'s $+N$, and block (6)'s X carry a control. The governing lemma: replacing G_j by c-G_j only for $j \in S$ gives a correct c-V *iff* the leftover product $\prod_{j \notin S} G_j$ is the identity on the states you care about. Two patterns satisfy it, and both appear here.

Conjugation. If $V = ABA^\dagger$, controlling B alone suffices, since $AA^\dagger$ is the identity. This is the QFT sandwich in §12: never control a conjugating layer.

Compensating residue. Ask what the uncontrolled blocks do when the control is **off**: $y - N$ goes negative ⇒ sign set ⇒ (3) sets $t \Rightarrow$ (4) adds N back. Then (5) subtracts X, the Toffoli in (6) does nothing (its outer control is off), the `x` flips t back to 0, and (7) restores X. Everything cancels. **This works only because** $y < N$ — the same precondition the correctness proof needs. With the control **on**: after (5) the sign qubit holds $\neg t$, so the Toffoli sets $t \mapsto t \oplus \neg t = 1$ and the `x` clears it.

Measured ($N = 15$, transpiled to $\{\text{CNOT}, U\}$), against the naive circuit built by wrapping an uncontrolled modular adder in `.control(k)` — i.e. controlling every gate:

	every gate controlled		as written here	
	CNOT	depth	CNOT	depth
1 control	2 353	4 351	287	372
2 controls (the real usage, §14)	5 916	10 689	315	418

8× and 19×. Almost all of it is the conjugation pattern — leaving the QFT layers alone. Isolating the residue pattern (blocks controlled or not, QFTs plain either way) is worth a further 1.6× at two controls. Do both; the first is nearly free to get right.

If you prefer, Beauregard's original choice is to control blocks (1), (5), (7) and leave (6) a plain CNOT. Both are correct, and only the residue bookkeeping changes.

The precondition is load-bearing

The proof uses $y < N$ twice. Fed $y \geq N$, the circuit leaves t dirty and the value wrong — and a dirty t does not give a wrong answer, it gives *no* answer. The invariant holds because every adder above reduces mod N; if you ever add without reducing, you have broken it.

14 Levels 3–4: multiplication, in place

Level 3, accumulate. Multiplication is shifted addition, and each shifted multiple $2^i A \bmod N$ is classically known. Bit x_i of the register decides whether its multiple joins the sum — so it becomes a *control*, and the data turns into conditioning:

$$A\,x \bmod N = \Big(\sum_{i=0}^{n-1} x_i \left(2^i A \bmod N\right) \Big) \bmod N.$$

Every partial sum is reduced, which is what maintains the $y < N$ invariant Level 2 demands.

```
def c_mult_acc(qc, c, A, x, y, sign, flag, N):
    """|c>|x>|y> -> |c>|x>|(y + A*x) mod N> : shift-and-add accumulate."""
    for i in range(len(x)):
        c_add_mod(qc, [c, x[i]], (A << i) % N, y, sign, flag, N)
```

Level 4, in place. `c_mult_acc` writes into a *fresh* register, but the ladder must apply rung after rung to the *same* register. Swapping is easy; the problem is that the old input x is still sitting in the scratch, and by the no-garbage rule it must be computed away, not discarded.

The escape is that A is invertible. Multiply, swap, then subtract A^{-1} times the product:

$$|x\rangle|0\rangle \xrightarrow{\times A} |x\rangle|Ax\rangle \xrightarrow{\text{SWAP}} |Ax\rangle|x\rangle \xrightarrow{-A^{-1}\cdot} |Ax\rangle|x - A^{-1}Ax\rangle = |Ax\rangle|0\rangle.$$

```
def c_ua(qc, c, A, x, y, sign, flag, N):
    """|c>|x>|0> -> |c>|A^c x mod N>|0> : multiply, swap, uncompute."""
    c_mult_acc(qc, c, A % N, x, y, sign, flag, N)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                          # classical inverse, gcd(A,N)=1
    c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N)
```

Note what is *not* here: no circuit is reversed. The adjoint of "add $A^{-1}x$ mod N" is "add $-A^{-1}x$ mod N", so the dagger becomes a minus sign on a classical constant. Note also that $\gcd(a, N) = 1$ is doing two jobs at once: it makes $y \mapsto ay$ a permutation (hence unitary, hence something phase estimation can act on *at all*), and it makes A^{-1} exist so the in-place trick works. One hypothesis, two roles.

15 Level 5: the ladder and the full circuit

```
def order_circuit(A, N, t=None):
    """Counting register of t qubits (default 2n), target, n+2 scratch."""
    n = math.ceil(math.log2(N))
    t = t if t is not None else 2 * n
    ctr, tgt = QuantumRegister(t, "ctr"), QuantumRegister(n, "tgt")
    anc = QuantumRegister(n, "anc")
    sf = QuantumRegister(2, "sf")              # sign qubit, flag qubit
    out = ClassicalRegister(t, "out")
    qc = QuantumCircuit(ctr, tgt, anc, sf, out)
    qc.h(ctr)                                  # Hadamard layer
    qc.x(tgt[0])                               # target = |1>
    for i in range(t):                         # the ladder: t rungs
        c_ua(qc, ctr[i], pow(A, 2**i, N), tgt, anc, sf[0], sf[1], N)
    qc.append(qft(t).inverse(), ctr)      # inverse QFT
    qc.measure(ctr, out)
    return qc
```

Qubit budget: t counting + n target + n scratch product + 1 sign + 1 flag = $4n + 2$ with $t = 2n$. For $N = 15$: $8 + 4 + 4 + 2 = 18$.

Why it works. After the Hadamards and the ladder the state is $\frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle$ — and every gate that produced it was a permutation matrix of 0s and 1s. Rewriting the *same state* in the

eigenbasis of U_a, using $|1\rangle = r^{-1/2}\sum_s |u_s\rangle$:

$$\frac{1}{\sqrt{2^t}}\sum_x |x\rangle|a^x \bmod N\rangle = \frac{1}{\sqrt{r}}\sum_{s=0}^{r-1} \underbrace{\left[\tfrac{1}{\sqrt{2^t}}\sum_x e^{2\pi i x s/r}|x\rangle\right]}_{\text{phase ramp of frequency } s/r} \otimes |u_s\rangle.$$

No gate wrote those phases. They are what a permutation of basis labels *looks like* in another basis, visible only because the register was in superposition. The inverse QFT reads the frequency, and the measurement returns $\approx 2^t s/r$ for a uniformly random s.

Why $t = 2n$ — and what happens if you skimp

The measured integer y satisfies $y/2^t \approx s/r$. To pin $r < N$ down uniquely from a rational approximation, continued fractions need the error below $1/2r^2$, and since r can be nearly N that means $2^t > N^2$, i.e. $t \geq 2n$.

Beware the $N = 15$ trap: there $r = 4$ divides 2^t exactly, so $y/2^t$ is exact for *any* $t \geq 2$ and a too-short register still works. $N = 21$ with $a = 2$ ($r = 6$, $n = 5$) is the honest test, and running it at three precisions shows the whole story:

t	qubits	recovered r	what the peaks look like
$5 = n$	17	**18** — wrong	$y/2^t$ lands on 7/20, 3/19: junk fractions
7	19	6 — correct	$21/128 \to 1/6$, $107/128 \to 5/6$
$10 = 2n$	22	6 — correct	clean peaks at $0, 171, 341, 512, 683, 853$

The $t = n$ failure is the instructive one. It did not return nonsense — it returned 18, which *passes* `pow(A, r, N) == 1` because 18 is a multiple of 6. A too-short register gives you a plausible, self-consistent, wrong answer. ($t = 7$ sufficing here is luck: the bound $t \geq 2n$ covers the worst case $r \approx N$, and you do not know r in advance.)

16 The classical half

```
def order_from_counts(counts, A, N, t):
    for bits in sorted(counts, key=counts.get, reverse=True):
        y = int(bits, 2)
        if y == 0:
            continue
        r = Fraction(y, 2**t).limit_denominator(N - 1).denominator
        if pow(A, r, N) == 1:
            return r
    return 0

def find_factor(N, run):                    # run: circuit -> counts dict
    if N % 2 == 0:
        return 2
    for k in range(2, N.bit_length() + 1):  # prime-power check
        d = round(N ** (1 / k))
        if d > 1 and d**k == N:
            return d
    import random
    while True:
```

```
        a = random.randrange(2, N)
        if (d := math.gcd(a, N)) > 1:
            return d                          # lucky gcd
        t = 2 * math.ceil(math.log2(N))
        r = order_from_counts(run(order_circuit(a, N)), a, N, t)
        if r and r % 2 == 0:
            d = math.gcd(pow(a, r // 2, N) - 1, N)
            if 1 < d < N:
                return d
```

`limit_denominator` *is* the continued-fraction expansion — Python ships the number theory. Candidates are checked with `pow(A, r, N) == 1` before being believed; $y = 0$ carries no information and is skipped.

Most of the peaks are useless — by design

The loop looks like defensive coding. It is not: **a majority of the peaks cannot give you** r, and you must expect to discard them. The measurement returns $y \approx 2^t s/r$ for uniformly random s, and continued fractions recover s/r *in lowest terms* — so what you get back is $r/\gcd(s, r)$. Only s coprime to r yields r itself.
At $N = 21$, $a = 2$ ($r = 6$, $t = 10$), the six peaks land as:

s	0	1	2	3	4	5
peak $y \approx$	0	171	341	512	683	853
$y/2^t \to$	0	1/6	1/3	1/2	2/3	5/6
recovered r	1	**6**	3	2	3	**6**
`pow` check	fails	**passes**	fails	fails	fails	**passes**

Two of six — that is $\varphi(6)/6$ — and the other four return proper divisors of r that the `pow` check rejects. This is not hypothetical: in the measured run the most frequent *nonzero* outcome was $y = 512$, which gives $r = 2$, and $\gcd(2^1-1, 21) = 1$ factors nothing. Trusting the top outcome would have failed; scanning all of them by descending frequency, as the loop does, finds $y = 171$ and $r = 6$. The $N = 33$ run repeats the lesson at larger scale: with $r = 10$ only $\varphi(10)/10 = 40\%$ of peaks work, the recovered fractions splitting cleanly into $\{\frac{1}{10}, \frac{3}{10}, \frac{7}{10}, \frac{9}{10}\}$ (accepted) and $\{\frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}\}$ (rejected), and again the top nonzero outcome was one of the rejects. One caveat the check cannot catch: `pow(A, r, N) == 1` accepts any *multiple* of the true order, so a stray y can return 12 instead of 6. Harmless for factoring when the result is even, but do not treat the returned value as certainly minimal.

Retries are cheap and expected: a bad measurement re-runs the *same* circuit; only a bad base a forces rebuilding, and that happens with probability $\leq 1/2$. That bound is close to tight, and easy to see for yourself — at $N = 33$, of the 19 nontrivial bases coprime to 33, nine fail: four have odd order ($r = 5$, for $a = 4, 16, 25, 31$) and five hit $a^{r/2} \equiv -1$, where $\gcd(a^{r/2}-1, N)$ collapses to 1 ($a = 2, 8, 17, 29, 32$). Ten succeed. So 47% of bases send you back to the top of the loop with a *correct* order in hand — order finding worked, and the reduction still failed. Budget for it: this is the common case, not an error.

17 Verifying it — level by level

This is the part most write-ups omit, and it is where the time actually goes. Each level is testable in isolation on a simulator, and **a bug at level k is invisible at level $k+1$ except as noise**, so test in order. Prepare a basis state, run one primitive, measure, compare with Python.

```
def set_value(qc, val, qubits):              # prepare a computational basis state
    for i, q in enumerate(qubits):
        if (val >> i) & 1:
            qc.x(q)
```

The suite below was run against the listings in this note; the results are its actual output.

Level	Test	Result
0	`qft`: unitary vs. the definition and vs. Qiskit's `QFTGate`, $n = 1..6$	exact agreement; `.inverse()` undoes it
0	`_maj`/`_uma` truth tables, all 8 inputs (§18)	MAJ $= (c{\oplus}a,\ b{\oplus}a,$ MAJ) and UMA$\cdot$MAJ $= (c,\ a{\oplus}b{\oplus}c,\ a)$
1	`rc_add`: all (x, y) pairs, $n = 2..5$, control on and off	sum correct, x preserved, carry ancilla clean; exactly $2n$ Toffoli
1	`rc_add_const`: all y, negative constants, $n = 3, 4$	correct mod 2^n; all $n{+}1$ ancillas clean
2	`rc_c_add_mod`: all pairs $y, X < N$ for $N = 9, 15$, control on/off	matches the Fourier version exactly; sign, flag and scratch all clean
4	`rc_c_ua`: $N = 15$, all coprime A, all x, control on/off	$x \mapsto Ax$ mod 15; all scratch clean — the tower is adder-agnostic
0	`lookup`: every address, $m = 1, 2, 3$ (§19)	T[addr] correct; address preserved, unary ancillas clean
2	`add_quantum_mod`: all (y, t) pairs, $N = 9, 15$	$(y{+}t)$ mod N; addend preserved, sign and flag clean
3	`windowed_c_mult_acc`: $N = 9$, $w = 1, 2$, all x, control on/off	agrees with `c_mult_acc` on every input; all scratch clean
3	`run_windowed_acc_mbu` (§19.6): $N = 15$, all k, all x, control on/off	every lookup uncomputed by measurement; x preserved, $y = kx$ mod 15, scratch clean, 180 mid-circuit measurements
0	lookup uncomputed by measurement on a *random non-uniform* address superposition	address restored exactly; leaked norm 0
0	`lookup_ui` (unary iteration, §19.5): every address, both control values, $m = 1..4$	correct; exactly $2(L{-}1)$ Toffoli; m ancillas not 2^m
0	`phase_fixup` (§19.6): every address, every split $l = 0..m$, $m = 2, 3, 4$	acts as exactly diag$((-1)^{F[a]})$; ancillas clean
0	X-basis measurement identity, $L = 4, 8$	every outcome leaves exactly $(-1)^{\text{parity}(m \wedge T[a])}$; output disentangled
5	`windowed_order_circuit(7,15,w=2,t=8)`, 24 qubits	peaks at $\{0, 64, 128, 192\}$, all 2048 shots; $r = 4$; $15 = 3 \times 5$ (58 s)
5	`windowed_order_circuit`, N=21, w=2, t=2n=10, 29 qubits	six peaks at multiples of 170.67 (smeared, $6 \nmid 2^t$); $r = 6$; $21 = 7 \times 3$ (4 062 s)
5	`windowed_exponentiate` (nested, §19.7): every $e < 16$, $N = 15$, $k = 7$	$\lvert e\rangle\lvert 1\rangle\lvert 0\rangle \mapsto \lvert e\rangle\lvert 7^e \bmod 15\rangle\lvert 0\rangle$; scratch clean
5	`nested_order_circuit`, N=15, w_e=w_m=2, 23 qubits	2 multiplications not 4; $r = 4$; $15 = 3{\times}5$ (28 s)
1	`add_const`: all y, several X incl. negative, $n = 3, 4$	single outcome $= (y + X)$ mod 2^n every time
2	`c_add_mod`: all $y, X < N$ for $N = 9, 15$, control on *and* off; measure sign and flag too	$(y + X)$ mod N with control on, identity with it off; **sign and flag always** $\lvert 00\rangle$
4	`c_ua`: all $x < N$, all $A \in \{2, 4, 7, 8, 11, 13\}$, $N = 15$, both control values	$x \mapsto Ax$ mod N; **all $n + 2$ scratch qubits back to** $\lvert 0\rangle$
5	`order_circuit(7,15)`, $t = 8$, 4096 shots	peaks at exactly $\{0, 64, 128, 192\}$, all 4096 shots on peaks
5	post-processing	$r = 4$ for both $a = 7$ and $a = 2$; `find_factor(15)` returns 5

$N = 15$ is too friendly to prove much, so the same suite was run at $N = 21$ ($n = 5$, $4n + 2 = 22$ qubits, orders 2, 3 and 6 — and $6 \nmid 2^t$, so continued fractions do real work):

Level	Test	Result
2	`c_add_mod`: all 441 pairs $y, X < 21$, control on and off	correct, ancillas clean, every case (7 s)
4	`c_ua`: all 11 bases coprime to 21, all $x < 21$, both control values	$x \mapsto Ax \bmod 21$; scratch clean throughout (88 s)
5	`order_circuit(2,21)`, $t = 10$, 22 qubits, depth ∼23 000	six peaks at $0, 171, 341, 512, 683, 853$ as predicted; $r = 6$; $\gcd(2^3-1, 21) = 7 \Rightarrow 21 = 7 \times 3$ (63 s)
5	`order_circuit(8,21)` ($r = 2$, exact)	two peaks, 1049 and 999 of 2048 shots; $r = 2$; again $21 = 7 \times 3$
and once more at $N = 33$ ($n = 6$, 26 qubits, depth ∼43 700)		
2	`c_add_mod`: all 1089 pairs $y, X < 33$, control on/off	correct, ancillas clean (24 s)
4	`c_ua`: all 19 bases coprime to 33, all $x < 33$	correct, scratch clean (575 s)
5	`order_circuit(5,33)`, $t = 12$, 2048 shots	ten peaks at every multiple of 409.6; $r = 10$; $\gcd(5^5-1, 33) = 11 \Rightarrow 33 = 11 \times 3$ (1373 s)
5	`order_circuit(10,33)` ($r = 2$)	$r = 2$; $\gcd(10-1, 33) = 3 \Rightarrow 33 = 3 \times 11$ (1511 s)

The $a = 2$ run is the one worth staring at. The peaks sit at $2^{10}s/6 = 170.67s$, which is *not* an integer, so the distribution smears onto neighbours (682 and 854 pick up 68 and 65 shots beside the main 683 and 853) — exactly the behaviour $N = 15$ can never show you, and exactly what the continued-fraction step exists to absorb.

The two assertions that matter most

Assert the ancillas are clean. At Levels 2 and 4, measure the scratch qubits along with the answer and assert they read $|0 \cdots 0\rangle$. A circuit can compute the right value and still be broken; this is the only cheap way to catch it before it becomes noise.

Assert the peaks, not the factors. At $t = 8$, $r = 4$, the measurement must concentrate on $y = 2^t s/r \in \{0, 64, 128, 192\}$. Do not assert "`find_factor` returned 3 or 5": the classical tail verifies its own candidates, so it can succeed by chance on a *uniform* distribution — exactly what a garbage bug produces. A published from-scratch notebook has this failure: flat histogram over all sixteen outcomes, retry loop still prints 3×5. The algorithm's robustness masks the bug. Test the distribution.

Pitfalls, in the order you will hit them

- **Endianness.** Qiskit is little-endian: `qr[0]` is the LSB, and printed bitstrings read MSB-first. Half of all bugs here are a reversed register.
- **Phase sign and scale.** `2*np.pi*X*2**(i-n)` — get the exponent wrong and you add the wrong constant, silently. Test `add_const` exhaustively; it takes seconds.
- **Negative exponents.** Plain Python is safe (`2 ** -3` is `0.125`), but if n or i arrives as a *numpy* integer — easy, if a bit width came from an array — `2 ** (i-n)` raises `ValueError: Integers to negative integer powers are not allowed`, and `1 << (i-n)` raises on any type. Write `2.0 ** (i - n)` and stop thinking about it.
- **The missing $+1$.** The value register must be $n+1$ qubits (sign qubit included) everywhere in §13. Drop it and the comparison silently reads a data bit.
- **Reducing the constant.** Pass X with $0 \le X < N$ into `c_add_mod` — hence `(A << i) % N` and `(-B) % N` at Level 3/4.
- **Simulation size.** $4n + 2$ qubits at $N = 15$ is 18 — fine. $N = 21$ is 22 qubits at depth $\sim$23 000, about a minute per run on a laptop; $N = 33$ is 26 and roughly $16\times$ worse. This is a wall, not a slope. Test the arithmetic levels with `Statevector.from_instruction` instead of sampling — they are permutations on basis states, so one exact evolution replaces thousands of shots, and the $N = 21$ sweeps above finish in seconds.
- **Transpile once.** Run the pass manager on the circuit once and reuse it across shots.

Running it

```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import SamplerV2 as Sampler

SIM, SAMPLER = AerSimulator(), Sampler(seed=7)
PM = generate_preset_pass_manager(backend=SIM, optimization_level=1)

def run(qc, shots=4096):
    return SAMPLER.run([PM.run(qc)], shots=shots).result()[0].data.out.get_counts()

counts = run(order_circuit(7, 15))           # -> peaks at 0, 64, 128, 192
print(order_from_counts(counts, 7, 15, t=8)) # -> 4
print(find_factor(15, run))                  # -> 3 or 5
```

Part V — Making it fast

Part IV's implementation is correct and about as simple as the algorithm gets. It is also slower than it needs to be in three separate ways, each with its own literature and its own trade. This part builds all three and measures them: ripple-carry arithmetic (a different gate set — the one error correction actually wants), windowing (fewer additions, paid for in qubits), and the two refinements, unary-iteration lookups and measurement-based uncomputation, that make windowing worth having.

18 The other Level 1: ripple-carry arithmetic

Everything so far rests on phase gates, and that is a liability. The smallest rotation in an n-qubit Fourier addition is $2\pi/2^{n+1}$: 0.20 rad at $n = 4$, 10^{-9} at $n = 32$, and 10^{-616} at $n = 2048$ — a number no apparatus will ever apply. Worse, arbitrary-angle rotations are not fault-tolerant gates; each must be synthesised from Clifford+T at a cost that grows with the precision demanded.

The alternative is to do addition the way hardware wants it: carries, in the computational basis, using only X, CNOT and Toffoli. Then the *Toffoli count* is the cost, because it is a proxy for the T count, and T gates are what magic-state distillation actually bills you for. This section builds Level 1 a second time. **Nothing above Level 1 changes.**

18.1 Two blocks that do everything

Schoolbook addition is $s_i = a_i \oplus b_i \oplus c_i$ with carry $c_{i+1} = \mathrm{MAJ}(a_i, b_i, c_i) = a_i b_i \oplus a_i c_i \oplus b_i c_i$. Read that as a cost statement: the sum bits are $\mathbb{F}_2$-linear, so they are CNOTs and effectively free; the carries are quadratic, so each one costs a Toffoli; and they form a chain, so depth is $\Theta(n)$.

Cuccaro, Draper, Kutin and Moulton [9] express the whole adder with two three-qubit blocks acting on a triple (c, b, a) — carry-in, addend, augend:

$$\begin{aligned} \mathrm{MAJ}: &\ \mathrm{CNOT}(a \to b),\ \mathrm{CNOT}(a \to c),\ \mathrm{TOF}(c, b \to a); \\ \mathrm{UMA}: &\ \mathrm{TOF}(c, b \to a),\ \mathrm{CNOT}(a \to c),\ \mathrm{CNOT}(c \to b). \end{aligned}$$

MAJ sends $(c, b, a) \mapsto (c \oplus a,\ b \oplus a,\ \mathrm{MAJ}(a, b, c))$: it computes the carry *into the wire that held* a, which is why no ancilla per carry is needed. Running the chain forward computes all n carries; running UMA back down undoes them. And here is the point of the design:

$$\mathrm{UMA} \cdot \mathrm{MAJ} : (c, b, a) \mapsto (c,\ a \oplus b \oplus c,\ a)$$

— UMA **erases the carry and writes the sum bit in the same three gates**. The uncomputation that §12's no-garbage rule makes compulsory is exactly the pass that delivers the answer. You are not paying twice; you are paying once and collecting on the way out.

```
def _maj(qc, c, b, a, ctrls=()):
    """MAJ: |c,b,a> -> |c^a, b^a, MAJ(a,b,c)>.  Only the b-target takes ctrls."""
    qc.mcx([*ctrls, a], b)              # plain CNOT when ctrls is empty
    qc.cx(a, c)
    qc.ccx(c, b, a)

def _uma(qc, c, b, a, ctrls=()):
    """UMA: undoes MAJ's carry and writes the sum bit into b."""
    qc.ccx(c, b, a)
    qc.cx(a, c)
    qc.mcx([*ctrls, c], b)              # plain CNOT when ctrls is empty

def rc_add(qc, x, y, carry, ctrls=()):
    """|x>|y> -> |x>|(y + x) mod 2^n>.  carry: one clean ancilla, returned |0>."""
    n = len(y)
    _maj(qc, carry, y[0], x[0], ctrls)
    for i in range(n - 1):
        _maj(qc, x[i], y[i + 1], x[i + 1], ctrls)
    for i in range(n - 1, 0, -1):
        _uma(qc, x[i - 1], y[i], x[i], ctrls)
    _uma(qc, carry, y[0], x[0], ctrls)
```

That is $2n$ Toffolis, $4n$ CNOTs and one ancilla, with x preserved and y overwritten by the sum — verified exhaustively below. The interleaved wire order you see in the literature $(c_0, b_0, a_0, b_1, a_1, \dots)$ is only a seating plan that keeps each block's three qubits adjacent; it means nothing mathematically.

18.2 Adding a classical constant

CDKM adds two *quantum* registers, but Level 2 wants "add the known constant X". The cheapest honest fix: write X into n clean ancillas with X gates, run the adder, and unwrite it. That is where the ripple-carry route's extra qubits go.

```
def rc_add_const(qc, X, y, anc, ctrls=()):
    """|y> -> |y + X mod 2^n>.  anc: n+1 clean ancillas, returned clean."""
    n = len(y)
    carry, xreg = anc[0], anc[1:n + 1]
    Xm = X % (2**n)                       # negative X just works, as in Fourier
    for i in range(n):                    # encode the constant -- uncontrolled
        if (Xm >> i) & 1:
            qc.x(xreg[i])
    rc_add(qc, xreg, y, carry, ctrls)
    for i in range(n):                    # unencode
        if (Xm >> i) & 1:
            qc.x(xreg[i])
```

Negative constants work for the same reason as in §12 — `X % 2**n` is the two's complement — so subtraction again needs no separate circuit. With this in place the modular adder of §13 is reproduced *line for line*, only the adder calls changing:

```
def rc_c_add_mod(qc, ctrls, X, y, sign, flag, N, anc):
    yw = list(y) + [sign]
    rc_add_const(qc, X, yw, anc, ctrls)  # (1) + X          (controlled)
    rc_add_const(qc, -N, yw, anc)        # (2) - N
    qc.cx(sign, flag)                     # (3) sign -> flag
    rc_add_const(qc, N, yw, anc, [flag])  # (4) + N  if flag
    rc_add_const(qc, -X, yw, anc)        # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)    # (6) clear flag from the answer
    qc.x(flag)
    rc_add_const(qc, X, yw, anc)         # (7) + X
```

And Levels 3–5 are then genuinely unchanged — same loops, same constants, same modular inverse, with only the extra ancilla register threaded through:

```
def rc_c_mult_acc(qc, c, A, x, y, sign, flag, N, anc):
    for i in range(len(x)):
        rc_c_add_mod(qc, [c, x[i]], (A << i) % N, y, sign, flag, N, anc)

def rc_c_ua(qc, c, A, x, y, sign, flag, N, anc):
    rc_c_mult_acc(qc, c, A % N, x, y, sign, flag, N, anc)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                          # classical inverse, unchanged
    rc_c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N, anc)
```

Checked exhaustively at $N = 15$ (17 qubits): every base coprime to 15, every $x < 15$, control on and off, $x \mapsto Ax \bmod 15$ correct and all $n{+}4$ scratch qubits returned to $|0\rangle$. The abstraction really does hold — swap the adder, keep the tower.

The control trick, again — and why it telescopes

Look at `_maj` and `_uma`: of their six gates, exactly *two* accept the outer control, and both are the CNOT targeting the b wire. Everything else is unconditional. The justification is the residue argument of §13, and here it is unusually clean.

With the control off, what survives of MAJ is $[\mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a)]$ and what survives of UMA is $[\mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c)]$ — the same two gates in the opposite order, i.e. each other's inverse. Since the UMA chain retraces the MAJ chain in reverse, the residues cancel pairwise from the inside out and the whole adder telescopes to the identity.

So a controlled n-bit addition costs $2n$ controlled gates rather than $6n$ — and the constant encoding stays uncontrolled too, since encode/unencode brackets an identity. Verified exhaustively for $n = 2..5$ with the control both on and off.

18.3 What it costs, measured

Both backends, the same doubly-usable modular adder, same modulus, counted before and after transpiling to $\{\mathrm{CNOT}, U\}$:

N	backend	qubits	rotations	Toffoli	CNOT	depth
15 (n=4)	Fourier	7	125	0	287	492
	ripple-carry	13	**0**	71	437	578
63 (n=6)	Fourier	9	245	0	545	948
	ripple-carry	17	**0**	99	637	**844**

Read the trend, not the absolute numbers. Fourier rotations grow as $O(n^2)$ per addition ($125 \to 245$) because of the QFT sandwich; ripple-carry Toffolis grow as $O(n)$ ($71 \to 99$). By $n = 6$ the ripple-carry circuit is already *shallower*, and the gap widens from there. What you buy is a factor of n in gate count and a gate set that survives error correction; what you pay is $n + 2$ extra qubits, taking the full order-finding circuit from $4n + 2$ to $5n + 4$.

Which to use

For *simulating* small N on a laptop, Fourier wins: fewer qubits, and simulation cost is exponential in qubit count, so $4n + 2$ versus $5n + 4$ is the whole ballgame.

For anything aimed at real fault-tolerant hardware, ripple-carry wins and it is not close. Zero arbitrary-angle rotations, a gate count lower by $O(n)$, and a cost model (T count) that actually predicts the resource bill. The 10^{-616} rad rotation is not a rounding problem you can engineer around — it is a statement that Fourier arithmetic at cryptographic sizes must be approximated before it can be attempted at all.

19 Windowing: trading lookups for additions

Level 3 spends one modular addition per bit of x — n of them, each controlled twice. Gidney's windowing [21] removes most of them by the classical trick of merging iterations into a table lookup. Unlike the control placement of §13, which is a fixed constant factor, this one *improves with* n — it is the optimisation that changes the exponent's logarithm, and the reason published Toffoli counts are what they are.

19.1 The idea in three steps

Step 1: a controlled addition is a lookup. Adding the constant k into y controlled on one qubit q is the same as adding $T[q]$ where $T = [0, k]$ — a two-entry table addressed by q. Nothing has changed yet.

Step 2: address more than one qubit. If a two-entry table works, so does a 2^w-entry one. Take w bits of x at a time; the value they contribute to kx is $k \cdot (\text{window value}) \cdot 2^i \bmod N$, which is classical — so tabulate all 2^w possibilities, look up the one the register actually holds, add it *once*, then uncompute the lookup. One addition per *window* instead of one per bit: $\lceil n/w \rceil$ instead of n.

Step 3: fold the control into the address. The outer control c becomes address bit 0, with every $c = 0$ entry set to 0 (Gidney Fig. 5). Adding zero modulo N is the identity, so the modular addition needs *no control at all*. This is why windowing composes so well with the ladder: the expensive part stops being controlled.

The cost of a window is one lookup instead of $w - 1$ extra additions, so the Toffoli count goes as $O\big(\frac{n}{w}(n + 2^w)\big)$. Setting $w = \lg n$ balances the two terms and gives $O(n^2/\lg n)$ — a logarithmic factor better than schoolbook.

19.2 What has to be built

Two new primitives. First a **QROM**: read a classical table at a quantum address. Encode the address one-hot, and the data load becomes pure CNOTs — one per set bit of the table.

```
def _unary_gate(m):
    """binary(m qubits) -> one-hot(2^m qubits), using 2^m - 1 Fredkins."""
    addr, unary = QuantumRegister(m, "a"), QuantumRegister(1 << m, "u")
    qc = QuantumCircuit(addr, unary)
    qc.x(unary[0])
    for i in reversed(range(m)):                    # MSB down to LSB
        step = 1 << i
        for j in range(0, 1 << m, 2 * step):
            qc.cswap(addr[i], unary[j], unary[j + step])
    return qc.to_gate(label="unary")

def lookup(qc, addr, out, table, unary):
    """|addr>|out> -> |addr>|out ^ table[addr]>; unary ancillas returned clean."""
    g = _unary_gate(len(addr))
    qc.append(g, list(addr) + list(unary))
    for j in range(len(unary)):
        Tj = table[j] if j < len(table) else 0
        for b in range(len(out)):
            if (Tj >> b) & 1:
                qc.cx(unary[j], out[b])
    qc.append(g.inverse(), list(addr) + list(unary))
```

Second, the lookup produces a *quantum* value, so Level 2 now needs to add a register rather than a constant. That is a smaller change than it sounds: the phase ramp of §12 just takes its rotations from the bits of t instead of the bits of a Python integer, and Beauregard's seven blocks are otherwise untouched.

```
def add_quantum(qc, a, y, sign=+1):
    """y += sign*a (mod 2^len(y)); a and y both quantum.  The quantum twin of
    add_const: same phase ramp, each rotation controlled by a bit of a."""
    n = len(y)
```

```
    qc.append(qft(n), y)
    for i in range(len(a)):
        for j in range(n):
            if i + j < n:                          # higher terms are trivial
                qc.cp(sign * 2 * np.pi * 2.0 ** (i + j - n), a[i], y[j])
    qc.append(qft(n).inverse(), y)

def add_quantum_mod(qc, t, y, sign, flag, N):
    """|y>|t> -> |(y+t) mod N>|t>.  Blocks (1),(5),(7) go quantum; rest as-is."""
    yw = list(y) + [sign]
    add_quantum(qc, t, yw, +1)            # (1) + t
    add_const(qc, -N, yw)                 # (2) - N
    qc.cx(sign, flag)                     # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)        # (4) + N  if flag
    add_quantum(qc, t, yw, -1)            # (5) - t
    qc.cx(sign, flag)                     # (6) [R < t] == not flag, so this
    qc.x(flag)                            #     pair returns flag to |0>
    add_quantum(qc, t, yw, +1)            # (7) + t
```

With those, Level 3 becomes a loop over windows rather than over bits. It calls `lookup_ui` rather than the `lookup` above: same operation, but built by unary iteration so it needs w ancillas instead of 2^w — see §19.5, which is worth reading before you build anything. The one-hot version above is the easier way to see what a QROM *is*; it is not the one to ship.

```
def windowed_c_mult_acc(qc, c, k, x, y, sign, flag, N, t, anc, w):
    """Drop-in for c_mult_acc, but ceil(len(x)/w) modular additions instead of
    len(x).  c is the top-level control of each lookup, so when c=0 no data
    loads at all and the modular addition -- which is uncontrolled -- adds 0.

    t:   len(y) clean qubits holding one looked-up value (returned clean)
    anc: w clean qubits for the unary-iteration QROM     (returned clean)
    w:   window size -- bits of x consumed per lookup
    """
    n = len(y)
    for i in range(0, len(x), w):
        win = list(x[i:i + w])
        # window value j contributes (k * j * 2^i) mod N
        table = [((k * j) << i) % N for j in range(2**len(win))]
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])
        add_quantum_mod(qc, t[:n], y, sign, flag, N)
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])   # uncompute
```

Level 4 and the ladder then need no new ideas at all — same multiply/swap/uncompute, same classical inverse, with the lookup scratch threaded through:

```
def windowed_c_ua(qc, c, A, x, y, sign, flag, N, t, anc, w):
    """|c>|x>|0> -> |c>|A^c x mod N>|0>.  Same multiply/swap/uncompute as
    c_ua; only the accumulate is windowed."""
    windowed_c_mult_acc(qc, c, A % N, x, y, sign, flag, N, t, anc, w)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                          # classical inverse, unchanged
    windowed_c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N, t, anc, w)
```

Why the $c = 0$ entries being zero is the whole trick

Set $c = 0$ and every table entry is 0, so the lookup writes $|0\rangle$ into t and block (1) adds nothing. Trace the seven blocks with $t = 0$: (2) drives the register negative, so the sign bit sets and (3) sets the flag; (4) adds N back; (5) subtracts nothing, leaving $R = y \geq 0$ so the sign bit is now 0; and (6) computes flag $\oplus$ sign $= 1$, which the X clears. Identity, ancilla clean — *without a single controlled gate in the adder*. The control survives only as a classical fact about the table.

19.3 What it costs, measured

One accumulate $y \mathrel{+}= c{\cdot}kx \bmod N$, transpiled to $\{\mathrm{CNOT}, U\}$:

method	mod. adds	qubits	CNOT	depth	vs. depth
$N = 15$, $n = 4$					
shift-and-add	4	11	1 254	2 119	—
window $w = 2$	2	23	1 184	1 128	1.9×
$N = 63$, $n = 6$					
shift-and-add	6	15	3 474	6 001	—
window $w = 3$	2	37	2 412	2 082	2.9×
$N = 255$, $n = 8$					
shift-and-add	8	19	7 390	13 020	—
window $w = 3$	3	43	4 646	4 873	2.7×
window $w = 4$	2	59	4 432	3 428	3.8×

The addition count falls exactly as $\lceil n/w \rceil$ predicts, and — unlike many asymptotic optimisations — the win is already real at $n = 4$ and grows: 1.06×, 1.44×, 1.67× in CNOTs at $n = 4, 6, 8$, and far more in depth, because the lookups for one window are shallow and the additions they replace were not. What you pay is qubits.

The same holds for the *whole* order-finding circuit, not just one accumulate. At $N = 15$, with the unary-iteration QROM of §19.5, windowing halves the depth for six extra qubits:

full circuit, $N = 15$	qubits	CNOT	depth
plain, $t = 4$	14	10 212	17 239
windowed $w = 2$, $t = 4$	20	7 122	**8 488**
plain, $t = 2n = 8$	18	20 462	34 537
windowed $w = 2$, $t = 2n = 8$	24	14 276	**17 028**

19.4 End to end

Windowed arithmetic factors 15 with nothing else changed. With the unary-iteration QROM of §19.5 the full-precision circuit — $t = 2n = 8$, $w = 2$ — fits in 24 qubits and gives a textbook distribution:

y	shots	$y/2^t$	continued fraction
0	530	0	—
64	490	1/4	$r = 4$ ✓
128	508	1/2	$r = 2$, rejected
192	520	3/4	$r = 4$ ✓

All 2048 shots land on the four ideal peaks, $r = 4$ is recovered, and $\gcd(7^2-1, 15) = 3$ gives $15 = 3\times5$; base $a = 2$ likewise. Note that even in this clean run the $y = 128$ peak returns $r = 2$ and is discarded — §16's scan is doing work here too.

It is worth seeing what §19.5 bought at this level. Built on the one-hot QROM the same circuit needed 30 qubits at $t = 2n$ — 16 GB of statevector, out of reach — so it had to be run at $t = 4$ instead, which is sound at $N = 15$ only because $r = 4$ divides 2^4. Unary iteration turned a compromise into the real thing, and made it faster: 58 s against 100 s.

$N = 15$ is too forgiving to prove much, so the windowed circuit was also run on $N = 21$, where the order is 6 and $6 \nmid 2^t$, so the continued-fraction step has to do real work. At the full $t = 2n = 10$ (29 qubits, depth 14 510, 8 GB of statevector, 68 minutes) it lands on all six ideal peaks $2^{10}s/6 = 170.67\,s$:

y	0	171	341	512	683	853	682	854
shots	326	223	247	337	221	234	74	72
$\to$	—	1/6	1/3	1/2	2/3	5/6	2/3	5/6

— the last two columns being the smearing onto neighbours that a non-integer $2^t/r$ forces, and which $N = 15$ can never show. It recovers $r = 6$, and $\gcd(2^3-1, 21) = 7$ gives $21 = 7 \times 3$. Only 1/6 and 5/6 pass the `pow` check (§16); the other four peaks return divisors of 6 and are discarded. Windowed arithmetic is not doing anything special at $N = 15$; it is just doing arithmetic.

Nested windowing (§19.7) factors 15 too, on 23 qubits at $t = 4$, using *two* modular multiplications where the plain ladder uses four: peaks at $\{0, 4, 8, 12\}$, $r = 4$, $15 = 3 \times 5$, for both $a = 7$ and $a = 2$.

19.5 Making the lookup cheap: unary iteration

The one-hot QROM above is correct but wasteful: it materialises all 2^m address states at once, so it needs 2^m ancillas. Gidney's Fig. 2 walks the address *tree* instead, holding only the AND of the current prefix [17]. Recursively: split on the top address bit, compute node = ctrl $\wedge\, a$ to select the upper half, flip it with one CNOT to get ctrl $\wedge\, \neg a$ for the lower half, then clear it.

```
def _walk(qc, ctrl, addr, out, table, offset, anc):
    """out ^= table[offset + value(addr)], all conditioned on ctrl."""
    if not addr:
        T = table[offset] if offset < len(table) else 0
        for b in range(len(out)):
            if (T >> b) & 1:
                qc.cx(ctrl, out[b])           # leaf: data load is CNOTs
        return
    a, rest = addr[-1], addr[:-1]             # split on the most significant bit
    half, node = 1 << len(rest), anc[0]
    qc.ccx(ctrl, a, node)                     # node = ctrl AND a      (upper half)
    _walk(qc, node, rest, out, table, offset + half, anc[1:])
    qc.cx(ctrl, node)                         # node = ctrl AND NOT a  (lower half)
    _walk(qc, node, rest, out, table, offset, anc[1:])
    qc.cx(ctrl, node)                         # put node back to ctrl AND a ...
    qc.ccx(ctrl, a, node)                     # ... and clear it

def lookup_ui(qc, ctrl, addr, out, table, anc):
    """|addr>|out> -> |addr>|out ^ table[addr]> when ctrl is 1, else identity.
    anc: len(addr) clean ancillas, returned clean.  2(L-1) Toffolis."""
    _walk(qc, ctrl, list(addr), list(out), list(table), 0, list(anc))
```

The recursion depth is m, so it needs m ancillas, not 2^m — and measured Toffoli counts land exactly on $2(L-1)$: $2, 6, 14, 30$ for $m = 1, 2, 3, 4$, against $2, 4, 8, 16$ ancillas saved down to $1, 2, 3, 4$. (With the temporary-AND trick [16] — built in §20 — the uncompute Toffolis become Clifford, giving quoted $L - 1$.) Note this also replaces the control-folding of §19: c is now simply the top-level `ctrl`, so the tree halves as well.

Swapping it in shrinks the windowed circuit substantially — and, unlike the one-hot version, the qubit cost is now barely sensitive to w:

one accumulate, $N = 255$ ($n = 8$)	mod. adds	qubits	CNOT	depth
shift-and-add	8	19	7 390	13 020
window $w = 2$	4	29	4 840	6 497
window $w = 3$	3	30	3 902	4 873
window $w = 4$	2	31	**3 264**	**3 866**

With one-hot at $w = 4$ the same circuit needed 59 qubits; unary iteration brings it to 31, and the CNOT advantage over shift-and-add improves from 1.67× to 2.26× (depth 3.4×). The earlier tables in §19 are the one-hot numbers; these are the ones to build on.

19.6 Uncomputing a lookup for free

We still uncompute each lookup by running it again, at equal cost — and that is avoidable, because the output is about to be discarded. Measure every output qubit in the X basis. On $\sum_a \alpha_a|a\rangle|T[a]\rangle$ this yields a uniformly random outcome m and leaves

$$\sum_a (-1)^{\,\mathrm{parity}(m \wedge T[a])}\, \alpha_a|a\rangle \;\otimes\; |m\rangle,$$

so the output register is disentangled *for free*; all that survives is a phase. Removing it is a phase-only lookup, and *that* can be done in $O(\sqrt{L})$: split the address into l low bits and h high bits, one-hot the low half (2^l Fredkins), then unary-iterate the high half applying CZ into the one-hot register ($2(2^h-1)$ Toffolis). With $l = h = m/2$ the total is $O(\sqrt{L})$ instead of $O(L)$.

```
def phase_fixup(qc, addr, F, one, unary_lo, anc_hi, l):
    """Apply the phase (-1)^F[a] to every basis state |a> of addr, in O(sqrt(L))."""
    lo, hi = list(addr[:l]), list(addr[l:])
    g = _unary_gate(l)
    qc.append(g, lo + list(unary_lo[:1 << l]))
    qc.x(one)                                          # always-true control
    _walk_cz(qc, one, hi, list(unary_lo[:1 << l]), F, 0, list(anc_hi))
    qc.x(one)
    qc.append(g.inverse(), lo + list(unary_lo[:1 << l]))
```

Verified two ways: `phase_fixup` acts as exactly $\mathrm{diag}((-1)^{F[a]})$ with all ancillas clean, for every split $l = 0 \ldots m$ at $m = 2, 3, 4$; and the measurement identity above holds for *every* outcome m (checked amplitude by amplitude at $L = 4, 8$). At $m = 4$ the $\sqrt{L}$ split costs 8 where a direct phase lookup costs 16.

One caveat on the measured costs elsewhere in this section: the CNOT and depth figures in §19–19.7 were taken from the *static* circuits, which uncompute by re-running the lookup. They are therefore a **lower bound** on what windowing delivers — the hybrid version below is cheaper still, by the factors in the table.

19.7 Nesting: windowing the exponent too

Everything so far windows the *multiplicand*. Gidney's headline construction (§3.5 of [21]) windows the *exponent* as well, and that is where the second logarithmic factor comes from. Take w_e counting qubits at a time and let the lookup select the factor $k^{v\cdot 2^i}$ for the window value v; the address becomes the exponent-window bits followed by the multiplicand-window bits, so one table serves both.

Three things fall out at once. There are $\lceil t/w_e \rceil$ multiplications instead of t. The multiplications need *no control at all* — when the exponent window reads 0 the factor is $k^0 = 1$, and multiplying by 1 is the identity. And the swap between accumulators becomes a *relabelling* in the classical loop, so the n Fredkin gates of §14 vanish.

```
for i in range(0, len(e), we):
    ei = list(e[i:i + we])
    ne = len(ei)
    kes = [pow(k, (2**i) * v, N) for v in range(2**ne)]
    kes_inv = [pow(v, -1, N) for v in kes]

    # B += A * k_e   (mod N):  maps (x, 0) -> (x, x*k_e)
    for j in range(0, n, wm):
        mi = A[j:j + wm]
        addr = ei + list(mi)
        table = [((kes[ad & ((1 << ne) - 1)] * (ad >> ne)) << j) % N
                 for ad in range(1 << len(addr))]
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
        qc.append(madd, tmp[:n] + B + [sign, flag])
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])

    # A -= B * k_e^{-1} (mod N): maps (x, x*k_e) -> (0, x*k_e)
    for j in range(0, n, wm):
        mi = B[j:j + wm]
        addr = ei + list(mi)
        table = [((kes_inv[ad & ((1 << ne) - 1)] * (ad >> ne)) << j) % N
                 for ad in range(1 << len(addr))]
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
        qc.append(msub, tmp[:n] + A + [sign, flag])
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])

    A, B = B, A                                  # relabelling swap -- free
```

Verified on the ladder identity directly: $|e\rangle|1\rangle|0\rangle \mapsto |e\rangle|k^e \bmod N\rangle|0\rangle$ for *every* exponent $e < 2^4$ at $N = 15$, $k = 7$, with all scratch returned to $|0\rangle$. Measured on the full circuit at $t = 2n = 8$:

$N = 15$, $t = 2n = 8$	mults	qubits	CNOT	depth
plain shift-and-add	8	18	20 462	34 537
windowed $w_m = 2$	8	24	14 276	17 028
nested $w_e = 2$, $w_m = 2$	4	27	**11 908**	**14 229**
nested $w_e = 4$, $w_m = 2$	2	29	16 292	20 095

1.72× the CNOTs and 2.43× the depth of plain shift-and-add, and a further 1.2× on top of single-level windowing. The last row is the useful warning: at $n = 4$, $w_e = 4$ makes the table 2^6 entries and the lookup costs more than the two multiplications it saved. Windows have an optimum ($w \approx \lg n$), and overshooting is worse than not windowing that level at all.

19.8 What of the paper is actually here

Windowing is a family of constructions, and it is worth being exact about which of them this note builds, since partial implementations of it are easy to mistake for the whole thing.

[21]	status	where / why not
§2, Fig. 2 table lookup by unary iteration	**built**	§19.5, `lookup_ui`
§3.3 windowed modular product addition	**built**	§19, `windowed_c_mult_acc`
§3.4 windowed modular multiplication	**built**	§19, `windowed_c_ua`
§3.5 nested exponent + multiplication windows	**built**	§19.7, `windowed_exponentiate`
§2, Fig. 3 measurement-based uncomputation	**built**	§19.6, `run_windowed_acc_mbu` — hybrid, so it runs as circuit/measure/classical/circuit
§3.2 windowed in-place multiplication	**built**	`windowed_inplace.times_equal`, $x \mathrel{*}= k \bmod 2^L$, exact on every input; the modular ladder still multiplies out-of-place and swaps (Fig. 6)

Wiring it in: the classical controller sits in the middle

The fixup table $F[a] = \text{parity}(m \wedge T[a])$ depends on the measurement outcome, so this cannot be one static circuit — it is genuinely *circuit* → *measure* → *classical* → *circuit*, which is exactly how it runs on hardware. `run_windowed_acc_mbu` does that: it emits a stage, measures, builds F in Python, emits the fixup, and continues.

Verified by wiring it into the real windowed accumulate: every k, every $x < 15$, control on and off, across 180 mid-circuit measurements — x preserved, $y = kx \bmod 15$, all scratch back to $|0\rangle$. Also checked on a lookup addressed by a *non-uniform random* superposition, where any residual phase would show: the address register is restored exactly, with leaked norm 0.

w	$L = 2^w$	recompute	measured	saving
2	4	6	3	2.0×
4	16	30	9	3.3×
8	256	510	45	11.3×
11	2048	4 094	157	26.1×

$2(L-1)$ against $2^l - 1 + 2(2^h - 1)$ with $l = h = w/2$. Uncomputation stops being half the lookup cost and becomes a rounding error — at the $w = \lg n \approx 11$ of a cryptographic run, 26× cheaper, which turns the compute/uncompute pair from 2× a lookup into 1.04×.

20 Emitting temporary ANDs

Every Toffoli in the unary-iteration walk of §19.5 is one half of a compute/uncompute pair — the tree computes ctrl ∧ a, uses it, and clears it. Gidney's observation [16] is that such a pair need not cost two Toffolis. Computing an AND *into a qubit known to be* $|0\rangle$ costs 4 T rather than 7, and *uncomputing* it costs none at all: measure the ancilla in the X basis and repair the phase with a Clifford CZ.

```
def and_compute(qc, a, b, t):
    # t must start in |0>; leaves t = a AND b using 4 T gates, no phase
    qc.h(t); qc.t(t)
```

```
    qc.cx(b, t); qc.tdg(t)
    qc.cx(a, t); qc.t(t)
    qc.cx(b, t); qc.tdg(t)
    qc.h(t); qc.sdg(t)

def and_uncompute(qc, a, b, t, cbit):
    # erase t = a AND b for free: X-basis measurement + Clifford repair
    qc.h(t)
    qc.measure(t, cbit)
    with qc.if_test((cbit, 1)):
        qc.cz(a, b)
        qc.x(t)                                    # return the ancilla to |0>
```

The final gate is $S^\dagger$, not S; three plausible variants of this circuit differ only in that phase and are all wrong, which a truth table alone will not catch — the check has to include the phase. One classical bit suffices for the whole lookup, since each outcome is consumed by the CZ immediately after it is produced and may then be overwritten.

Substituting the pair into the walk gives `lookup_ui_ta`, verified to compute the same map on every address for $m = 1, 2, 3$ with both control values and all ancillas clean. Transpiled to Clifford+T:

m	L	T (plain)	T (temporary AND)	saving
2	4	42	12	3.50×
3	8	98	28	3.50×
4	16	210	60	3.50×
5	32	434	124	3.50×

Exactly 7/2, and for the reason you would predict: half the Toffolis were uncomputes and are now free, and the other half fell from 7 T to 4. Since the T count is what magic-state distillation bills, this is the cheapest real saving in the note — a few lines of code, no change to any interface, and it applies to every lookup in Part V.

21 One counting qubit: $4n + 2 \to 2n + 3$

The counting register is the largest in the circuit and it is barely used: each of its qubits is Hadamarded, controls one rung, and is measured. Worse, in the inverse QFT every controlled rotation is controlled by a qubit that is *about to be measured anyway*. So measure it first and make the control classical — the semiclassical Fourier transform of Griffiths and Niu [13], applied to factoring by Mosca–Ekert [14], Parker–Plenio [15] and Beauregard [8]. The counting register then never exists all at once: one qubit is Hadamarded, drives its rung, receives phase corrections conditioned on every previously measured bit, is measured, and is reset. The loop knows nothing about modular arithmetic — it takes the rungs as callbacks — so it lives in `semiclassical.py` and the elliptic-curve circuit of Part VII reuses it unchanged:

```
def semiclassical_iqft(qc, ctrl, out, rungs):
    t = len(rungs)
    assert len(out) >= t, "one classical bit per rung"
    for i in range(t):
        qc.h(ctrl)
        rungs[t - 1 - i](qc, ctrl)                 # heaviest rung first
        for j in range(i):                         # the inverse QFT, classically
            with qc.if_test((out[j], 1)):
```

```
                qc.p(-math.pi / 2 ** (i - j), ctrl)
        qc.h(ctrl)
        qc.measure(ctrl, out[i])
        with qc.if_test((out[i], 1)):                # reset for the next rung
            qc.x(ctrl)


def order_circuit_1c(A, N, t=None):
    n = math.ceil(math.log2(N))
    t = t if t is not None else 2 * n
    ctr = QuantumRegister(1, "ctr")                  # the whole counting register
    tgt = QuantumRegister(n, "tgt")
    anc = QuantumRegister(n, "anc")
    sf = QuantumRegister(2, "sf")                    # sign, flag
    out = ClassicalRegister(t, "out")
    qc = QuantumCircuit(ctr, tgt, anc, sf, out)
    qc.x(tgt[0])                                     # target = |1>

    def rung(k):                                     # controlled U^(2^k)
        return lambda qc, c: c_ua(qc, c, pow(A, 2 ** k, N),
                                  list(tgt), list(anc), sf[0], sf[1], N)
    semiclassical_iqft(qc, ctr[0], out, [rung(k) for k in range(t)])
    return qc
```

$1 + n + n + 2 = 2n + 3$, the count in the title of Beauregard's paper. Measured:

N	qubits	plain	time	result
15 ($a = 7$, $a = 2$)	**11**	18	6–7 s	$r = 4$, $15 = 3 \times 5$
21 ($a = 2$)	**13**	22	48 s	$r = 6$, $21 = 7 \times 3$

Three things that bite

The bit order is not what you expect. With the rungs run most-significant-exponent first, the *first* measured bit is the *least* significant bit of the phase — which happens to be Qiskit's own bit order, so `order_from_counts` reads this circuit unchanged. Assuming the reverse (as the author first did) silently yields $r = 0$ at $N = 15$; only testing both orderings against a known r settles it.

Fewer qubits does not mean faster simulation. Mid-circuit measurement forces Aer to simulate shot by shot rather than evolve one statevector, so every shot is paid for individually. On hardware this is the cheapest circuit in this note; in simulation part of the qubit saving is given back. It is still a large net win — $N = 21$ in 48 s against roughly 1 000 s for the 22-qubit version.

It is incompatible with exponent windowing. Windowing w_e exponent bits (§19.7) needs those w_e qubits to coexist as lookup address bits; this circuit keeps exactly one alive. The two are ends of one spectrum — counting qubits $= w_e$ — and you cannot have both. Windowing *costs* qubits (24 at $N = 15$) where this *saves* them (11).

22 Coset representation: deleting the modular adder

Everything so far pays five constant additions and a flag qubit for each modular addition (§13), and measurement says that machinery costs 5.2–5.5× the addition it wraps — flat in n, from $N = 15$ to

$N = 1023$. It exists for one reason: to turn addition mod 2^m, which the register does natively, into addition mod N.

Zalka's coset representation [18], as used by Gidney and Ekerå [19], removes the need. Instead of storing k as $|k\rangle$, store the periodic superposition

$$|k\rangle_{\text{coset}} \;=\; 2^{-c_{\text{pad}}/2} \sum_{j=0}^{2^{c_{\text{pad}}}-1} |jN + k\rangle,$$

in a register widened by c_{pad} padding qubits above the high bits. Every branch is congruent to k modulo N, so a *plain, non-modular* addition of a already produces a state whose every branch is congruent to $k + a$. When $k + a \geq N$ the sum simply re-indexes,

$$\sum_j |jN + (k{+}a)\rangle \;=\; \sum_{j'=1}^{2^{c_{\text{pad}}}} |j'N + (k{+}a{-}N)\rangle,$$

which differs from the ideal coset state of $(k{+}a)$ mod N in only the two edge terms out of $2^{c_{\text{pad}}}$. **That is the entire approximation.** Level 2 collapses into Level 1, and the sign and flag qubits go with it:

```
def add(qc, X, reg, N=None):
    """Modular addition -- as a PLAIN addition.  This is the entire point:
    no compare, no subtract, no restore, no flag qubit, no sign qubit."""
    if N is not None:
        X %= N
    add_const(qc, X, list(reg))
```

22.1 The deviation law, measured

Gidney and Ekerå bound the deviation per addition by $n/(c_{\text{sep}}2^{c_{\text{pad}}})$, subadditive under composition, so a sequence of A additions deviates by at most $A\,2^{-c_{\text{pad}}}$ when there are no carry runways. Measured, with $A = 8$ random additions:

N	$2^n - N$	c_{pad}	measured error	bound $A/2^{c_{\text{pad}}}$
15	1	2	0.749	2.000
		3	0.373	1.000
		4	0.129	0.500
		5	0.031	0.250
		6	0.000	0.125
31	1	2	0.507	2.000
		4	0.061	0.500
		6	0.000	0.125

The error halves with each padding qubit, exactly as advertised, and every measurement sits below the bound — which is loose by 2–4×, as a worst-case bound should be.

Where the approximation actually bites

A first attempt to measure this at $N = 21$ returned *zero* error at every padding size, which looks like a broken test and is not. The register holds $m = n + c_{\text{pad}}$ bits, and the headroom above the top branch is $H = 2^{c_{\text{pad}}}(2^n - N) + N - 1 - k$. For $N = 21$, $2^n - N = 11$, so the headroom is enormous and no branch ever wraps. The approximation only bites when the accumulated offset exceeds H, i.e. when N sits close to 2^n — which is why the table above uses $N = 15$ and $N = 31$, where $2^n - N = 1$. **Choosing a friendly modulus hides the very effect you are trying to measure.**

22.2 What it saves

One addition, coset versus the seven-block modular adder:

adder	$N = 15$	$N = 63$	$N = 255$	$N = 1023$
Fourier (CNOT ratio)	1.7×	2.2×	2.5×	2.8×
ripple-carry (Toffoli ratio)	3.0×	3.1×	3.5×	3.9×

Less than the naive 5.3×, because the coset register is c_{pad} qubits wider and the adder must pay for that width; and better with ripple-carry, whose cost is linear in width, than with Fourier, whose QFT sandwich is quadratic. The ripple-carry figure of $4n$ Toffoli for the plain addition matches [19] exactly. Both ratios improve with n, since c_{pad} grows only logarithmically: $c_{\text{pad}} = 2\lg n + \lg n_e + \delta$ with $\delta \in [2, 10]$.

22.3 Multiplying in the coset representation

There is a subtlety that looks fatal and is not. In branch j the register holds $v = jN + x$, so its *bits differ between branches* — and shift-and-add multiplication uses those bits as controls. It works anyway:

$$\sum_i v_i \,(A2^i \bmod N) \;\equiv\; Av \;=\; A(jN + x) \;\equiv\; Ax \pmod N,$$

so every branch accumulates something congruent to Ax, whatever its own j. Verified at $N = 7$ with both registers coset-encoded:

c_{pad}	m	qubits	worst error	bound $A/2^{c_{\text{pad}}}$
3	6	13	0.0760	0.7500
4	7	15	0.0135	0.4375

What is built, and what is not

Built and verified: the coset encoding, the plain-adder modular addition, the deviation law, and the coset multiplier. **Also built:** the full coset order-finding circuit (`coset_order.py`) and oblivious carry runways [20] (`runway.py`). The encoder is a circuit (pad qubit in $|+\rangle$, add $2^h N$, uncompute the pad by comparison) and matches the ideal coset state exactly. Level 4 runs in coset form — the accumulator starts in $|0\rangle_{\text{coset}}$ and the un-multiply returns it there — with runways attached to the accumulator only for the duration of each accumulation (encode, add piecewise, remove). One runway of length m deviates by at most 2^{-m} per addition, and `tests/test_runway.py` measures 0.2513 against 0.25 for $m = 2$. At toy sizes the whole multiplication is far from the ideal state, as the bound $2m \cdot 2^{-c_{\text{pad}}}$ predicts: for $N = 7$, fidelity 0.43/0.63/0.76 at $c_{\text{pad}} = 3/4/5$. End to end ($N = 15$, $A = 7$, $t = 3$) 50/60 shots land on the exact-phase outcomes without runways and 47/60 with them, and both recover $r = 4$.

Gidney's 2025 approximate-residue factoring proposal also uses *yoked surface codes* for idle logical storage; this is a physical error-correction assumption, not a gate in the arithmetic oracle. `yoked_storage.py` reproduces only the fixed-parameter footprint from his Sec. 3.2: at physical error 0.1% and logical error target 10^{-15} per qubit-round, 1 280 cold logical qubits at 430 physical each, 131 hot qubits at 1 352 each, and 126 compute patches at 1 352 each give 897 864 physical qubits, versus 2 078 024 if the cold qubits use hot patches. This is a checked *storage cost model*, not a yoked-code syndrome/decoder implementation or an RNS factoring circuit.

`rns_order.py` is a distinct, deliberately unoptimised *toy* residue-number-system order finder after the high-bit approximation of Gidney (2025), Eqs. (8)–(22). For $N = 15$, $A = 2$, three exponent bits and primes $(7, 11, 13, 19)$, the CRT modulus is $L = 19\,019 \geq 15^3$ and $\Delta_{15}(L) = 1/15 < 2^{-3}$. Coherent modular multipliers compute the residue of the product of the N-*reduced* power factors at each prime. The bits of those residues control additions of the truncated CRT weights into a three-bit output; all residue and adder work is uncomputed. Every eight input exponents agrees with the classical CRT approximation, whose maximum measured modular deviation is 4/15; only 4/8 of the approximated high-bit outputs equal the exact truncated outputs, although the period-four labels remain distinct. With an inverse QFT, all 512 Aer CPU shots land on period-four frequencies and recover order 4. The equal-task, exact Fourier-modular reference uses 13 qubits, 324 Toffolis and 4 800 unsynthesised rotations versus the toy RNS circuit's 18, 2 654 and 43 983: the latter is *dominated* on this toy frontier. These counts do not include fault-tolerant rotation synthesis. This is neither the least-significant-bit CFS 2024/222 algorithm nor Gidney's low-space GHZ-lookup/vented-uncompute implementation; neither paper's cryptographic-size qubit reduction follows from this toy circuit.

`rns_gidney.py` is a separate toy transcription of the `approx_modexp` dataflow of Gidney (2025), App. A.1 and Fig. 3: discrete-log lookup-additions, binary long division modulo $p - 1$, a residue lookup, residue-bit accumulation of the truncated CRT weights (Eq. 21) whose wrap qubit is X-measured and phase-corrected by a comparison, and measurement-based deletion of the lookup output with a phase correction. For $N = 15$, $g = 2$, $m = 3$, primes $(17, 23)$, $f = 2$ ($L = 391$, $L \bmod 15 = 1 < 15 \gg 2$) and window $w_1 = 2$, all eight basis inputs give the Eq. (20) output with every workspace qubit cleared. On a uniform exponent superposition every Aer shot branch has fidelity 1 with the ideal Eq. (26) state, with and without a one-bit mask, and removing either phase correction breaks this. The full inverse-QFT circuit recovers order 4 from 512 Aer CPU shots with 18 qubits, 1 816 Toffolis, 1 739 unsynthesised rotations and 16 mid-circuit X-measurements on the same task (`bench/rns_gidney_toy.json`); the exact Fourier-modular oracle still dominates it at this size. Labelled deviations: Shor-style g^e input rather than Ekerå–Håstad; lookup tables rebuilt from the equations because the paper's Zenodo code was unavailable; only the loop3 startup lookup and $w_4 = 1$; ancilla-free Fourier arithmetic without vents; toy widths. It is not the CFS 2024/222 least-significant-bit algorithm.

`rns_cfs.py` is a separate toy of the *least-significant-bit* multi-product of Chevignard, Fouque and Schrottenloher [22], Algorithm 1: $x \mapsto [[\prod_i A_i^{x_i}]_N]_{2^r}$. It uses the Eq. (27) cofactors and the paper's register widths (q_M on $u + \lceil \log_2 q_M \rceil + 1$ bits with u from Lemma 5, Q_N on $u' + r$ bits with u' from Heuristic 3). Residues follow Lemma 7: a sum of discrete logarithms, then a

23 Where to go from here

Two more upgrades, both easy and worth doing immediately.

Approximate QFT (easy, big win). Already built: pass $d = \lceil \log_2 n \rceil + 2$ as `approx` to `qft` (§11) and the count drops from $O(n^2)$ to $O(n \log n)$ with negligible error [11] — measured, $144 \to 99$ gates at $n = 16$, and the gap widens with n. Make it the default once your tests pass exactly.

One counting qubit — built and measured in §21.

Ripple-carry arithmetic — built and measured in §18. Do this one if you are heading anywhere near fault tolerance.

A fuller Qiskit implementation of the plain (unwindowed) algorithm, with both adders and the one-control circuit, is [26].

Windowing — built and measured in §19. Combined with the control savings it is what takes published Toffoli counts from $\approx 40n^3$ to $\approx 0.3n^3$.

What this does not demonstrate

Simulation cost grows exponentially in qubit count, so noise-free demos stop at a few bits. On real 2020s hardware the binding constraint is not qubit count but noise: even the $N = 15$ circuit's two-qubit depth far exceeds current coherence. What you have built is the algorithm, correct and testable — not a factoring machine. The gap between them is fault tolerance, and at $n = 2048$ it is about 10^9 Toffolis.

Part VI — Reading someone else's implementation

24 The package at a glance

Everything in Part II exists as running code: the `qiskit-shor` package [26] (see also the accompanying walkthrough [27]) implements the full tower on top of the Qiskit SDK and factors integers end to end on a simulator. It is worth reading against the note for two reasons. First, it realises the tower *twice*, over two interchangeable Level-1 backends — the CDKM ripple-carry adder of Part II [9], and a Fourier-space adder [7, 8] that Part II did not need but that dominates practical small-scale demonstrations. Second, everything above Level 1 is written *once*, generically over the adder: the code is organised exactly along the levels of §1.5, and the class hierarchy is the proof of the note's claim that Levels 2–5 use nothing about addition except its interface.

Module	Contents	Level(s)
`gate_utils.py`	`MAJGate`, `UMAGate`, controlled variants, QFT synthesis	0
`rc_adder.py`	`RCAdderCircuit`: ripple-carry additions [9]	1
`qft_adder.py`	`QFTAdderCircuit`: Fourier-space additions [7, 8]	1
`adder.py`	`AdderCircuit` (abstract): modular add / multiply / exponentiate	2–5
`shor.py`	order-finding circuits, `find_order`, `find_factor`	5 + classical

The class `AdderCircuit` is an abstract subclass of Qiskit's `QuantumCircuit`. It declares two primitives, `add_classical` and its controlled version — "add a known constant to a register, optionally

under a control". The two backends supply those primitives; the abstract class builds Levels 2–5 out of them, verbatim from Theorems 5.1–7.1.

Dictionary: the note's registers, the code's names

The code follows Qiskit's little-endian convention: qubit k of a register carries bit x_k of $x = \sum_k x_k 2^k$, so qubit 0 is the least significant. The cast of §5 maps as:

In the note	In the code	Width
value register $\|x\rangle_{n+1}$	`y_reg` + `overflow_bit`	$n+1$
the sign qubit (top bit)	`overflow_bit`	1
the flag t	`ancilla_bit`	1
multiplicand register $\|x\rangle$	`x_reg`	n
counting register	`control_register`	$2n$ (or 1, §27.1)
ripple-carry scratch	`a_reg` (RC backend only)	$n+2$

25 Level 1 in code, twice

25.1 The ripple-carry backend

`gate_utils.py` defines MAJ and UMA exactly as in §4, as reusable Qiskit gates (listings are lightly abridged — docstrings and boilerplate trimmed — throughout Part IV):

```
class MAJGate(SingletonGate):
    """|c,b,a> -> |c+a, b+a, MAJ(a,b,c)>   (addition mod 2)"""
    def _define(self):
        q = QuantumRegister(3, "q")        # q0=c, q1=b, q2=a
        qc = QuantumCircuit(q, name="maj")
        qc.cx(q[2], q[1])                  # CNOT(a -> b)
        qc.cx(q[2], q[0])                  # CNOT(a -> c)
        qc.ccx(q[0], q[1], q[2])           # TOF(c, b -> a)
        self.definition = qc

class UMAGate(SingletonGate):
    """UMA . MAJ |c,b,a> = |c, a+b+c, a>"""
    def _define(self):
        q = QuantumRegister(3, "q")
        qc = QuantumCircuit(q, name="uma")
        qc.ccx(q[0], q[1], q[2])           # TOF(c, b -> a)
        qc.cx(q[2], q[0])                  # CNOT(a -> c)
        qc.cx(q[0], q[1])                  # CNOT(c -> b)
        self.definition = qc
```

and `RCAdderCircuit.add_quantum` chains them precisely as in the diagram of Theorem 4.2 — MAJ up the bits, one CNOT into the overflow qubit at the single instant the final carry sits in the clear, UMA back down:

```
def add_quantum(self, x_reg, y_reg, ancilla_bit, overflow_bit=None):
    """|x>_n |y>_n |0> -> |x>_n |y+x>_{n+1} |0>"""
    n = len(y_reg)
    maj, uma = MAJGate(), UMAGate(RC_version=True)
    self.append(maj, [ancilla_bit, y_reg[0], x_reg[0]])
```

```
    for i in range(n - 1):
        self.append(maj, [x_reg[i], y_reg[i + 1], x_reg[i + 1]])
    if overflow_bit:
        self.cx(x_reg[n - 1], overflow_bit)
    for i in range(n - 1, 0, -1):
        self.append(uma, [x_reg[i - 1], y_reg[i], x_reg[i]])
    self.append(uma, [ancilla_bit, y_reg[0], x_reg[0]])
```

Two departures from the bare Theorem 4.2 are worth pausing on.

Remark (the 3-CNOT UMA). The flag `RC_version=True` selects Cuccaro et al.'s alternative six-gate UMA ($X\,b$; CNOT $a \to b$; TOF $a, b \to c$; $X\,b$; CNOT $c \to a$; CNOT $c \to b$) [9]. It computes the same permutation as the three-gate version — the package's unit tests check them against each other — but its final two CNOTs share no qubit with the *next* UMA's first gates, so consecutive blocks overlap in time and the adder's depth drops by a constant factor. Same matrix, better schedule: a first taste of the fact that gate count and depth are different currencies.

Remark (adding a constant with an ancilla register). CDKM adds two *quantum* integers, but Level 2 needs $y \mathrel{+}= X$ for a *classical* X. The RC backend does the simplest correct thing: it writes X into n clean ancillas with X gates (`bitwise_AND_classical`), runs `add_quantum`, and unwrites X with the same X gates — which is why this backend carries the extra `a_reg` of $n+2$ qubits through every signature, and why its total budget is $5n+4$ qubits against the Fourier backend's $4n+2$ (§27.2). Subtraction, needed for blocks (2) and (5) of Theorem 5.1, is the same routine with the two's-complement of X: the code reduces $X \bmod 2^{n+1}$ first, so `add_classical(X - N, ...)` just works.

The controlled adder is where the code quietly reuses the control lemma of §7. Rather than controlling all six gates of every block, `gate_utils.py` defines `MCXMAJGate` and `MCXUMAGate`, in which *only the CNOT targeting the b wire* acquires the outer control:

$$\begin{aligned} \mathrm{MAJ}_c:&\ c\text{-CNOT}(a\to b),\ \mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a);\\ \mathrm{UMA}_c:&\ \mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c),\ c\text{-CNOT}(c\to b). \end{aligned}$$

With the control off, what remains of MAJ is $\big[\mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a)\big]$ and what remains of UMA is its exact mirror $\big[\mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c)\big]$; since the UMA chain retraces the MAJ chain in reverse order, the whole residue telescopes to the identity — the *conjugation* pattern of §7, found in the wild. One controlled gate per block, $2n{+}1$ in total (the overflow CNOT is controlled too), instead of 6 per block.

25.2 The Fourier backend

The default backend does no carry propagation at all. Draper's observation [7] is that the QFT turns addition into phase accumulation. Write the full QFT on n qubits, in the code's little-endian convention, as

$$F|y\rangle = \bigotimes_{i=0}^{n-1} \frac{|0\rangle + e^{2\pi i\, y\, 2^{\,i-n}}|1\rangle}{\sqrt{2}}.$$

Each qubit carries the value y in the *period* of a phase, at a different scale — in Draper's reading, qubit i holds the binary fraction $e^{2\pi i\,(0.y_{n-i-1}\cdots y_1 y_0)}$ of the $n{-}i$ low bits of y: the value is spread across scales, not across carries. Adding a classical constant X is then n *parallel, uncontrolled* phase gates — qubit i gets $P(2\pi X\, 2^{\,i-n})$, turning $e^{2\pi i y 2^{i-n}}$ into $e^{2\pi i (y+X) 2^{i-n}}$ — and an inverse QFT returns to the computational basis holding $|(y+X) \bmod 2^n\rangle$. That is the entire implementation:

```
def add_classical(self, X, y_reg, a_reg=None, include_QFT=True):
    """|y> -> |y + X mod 2^n>   (no ancillas needed)"""
    n = len(y_reg)
    qft = QFTFullGate(n, approximation_degree=self.qft_approx_degree(n))
    if include_QFT:
        self.compose(qft, y_reg, inplace=True)
    for i in range(n):
        self.p(2 * np.pi * X * 2 ** (i - n), y_reg[i])  # addition in Fourier space
    if include_QFT:
        self.compose(qft.inverse(), y_reg, inplace=True)
```

No carries, no ancillas, and the controlled version (`c_add_classical`) simply upgrades each phase gate to a controlled phase gate — the QFT layers keep no control, being the conjugating $A \cdots A^\dagger$ of the control lemma, §7. This is why the Fourier route needs two registers fewer than the ripple-carry route, and why Beauregard [8] could squeeze the whole algorithm into $2n + 3$ qubits.

Remark (what the Fourier route buys in depth). Two structural facts, one from each source paper, sharpen the economics. *Beauregard [8]:* because X is classical, all the rotations that land on one qubit multiply into a *single* phase gate — the loop above emits exactly one P per qubit, so the addition layer between the QFTs is n parallel one-qubit gates at **depth 1**. *Draper [7]:* even in the quantum–quantum adder (`add_quantum`, $O(n^2)$ controlled rotations) all rotations *commute* — unlike the QFT itself, whose Hadamards impose an ordering — so they schedule into $n{+}1$ time slices, or $O(\log n)$ slices under the small-angle cutoff below. The depth bottleneck of the Fourier route is therefore the QFT sandwich, at depth $O(n)$, not the addition it encloses. Finally, the same gate run backwards subtracts: $\phi\mathrm{ADD}(a)^\dagger$ sends $\phi(b)$ to $\phi(b-a)$ if $b \geq a$ and to $\phi(2^{n+1} - (a-b))$ otherwise, so the top qubit is the comparator $[\,b < a\,]$ — §4's "comparison is free", relocated to Fourier space, and the workhorse of the sign tests in §26.

Doesn't this break "no complex number below Level 5"?

Inside the sandwich, yes: the intermediate states have irrational phases, and a single $P(\theta)$ is certainly not a permutation matrix. But the *composite* $F^\dagger \cdot (\text{phases}) \cdot F$ is exactly the same 0/1 permutation matrix $|y\rangle \mapsto |y + X \bmod 2^n\rangle$ as the CDKM circuit. The slogan of §1.3 survives in a sharper form: the two backends are two factorisations of one permutation — one through $\mathbb{F}_2$ carry algebra, one through the Fourier dual — and everything above Level 1 cannot tell them apart. The trade is not conceptual but economic: the Fourier route pays in *precision* (doubly-controlled rotations by angles as small as $2\pi/2^{n+1}$) instead of in *qubits*, which is why the package also offers the RC backend, whose gate set $\{X, \mathrm{CNOT}, \mathrm{TOF}\}$ is the one fault-tolerant hardware actually wants.

Remark (the approximate QFT). Rotations by $2\pi/2^k$ for large k are far below any hardware's resolution, and dropping them barely disturbs the result: keeping only angles at least $\pi/2^d$ with $d = \lceil \log_2 n \rceil + 2$ costs an error that stays negligible while cutting the QFT from $O(n^2)$ to $O(n \log n)$ gates [11, 12]. The package exposes exactly this cutoff via the flag `approx_QFT`; it is the "approximate QFT" priced in Part III's bill.

26 Levels 2–4: written once, above the interface

26.1 The modular adder

`AdderCircuit.add_classical_modulo` is Theorem 5.1 with one refinement. Since a *and* N are both classical, blocks (1) and (2) — add a, subtract N — merge into a single addition of the constant $a - N$:

```
def add_classical_modulo(self, X, y_reg, ancilla_bit, overflow_bit, N,
                         a_reg=None, reset_ancilla=True):
    """|y>|0> -> |(X + y) mod N>|0>,  assuming 0 <= X < N and 0 <= y < N."""
    y_bits = y_reg[:] + [overflow_bit]        # the (n+1)-wide value register
    n = math.ceil(math.log2(N))

    self.add_classical(X - N, y_bits, a_reg)  # blocks (1)+(2): y + X - N
    self.cx(y_bits[n], ancilla_bit)           # block (3): sign -> t
    self.c_add_classical(ancilla_bit, N, y_bits, a_reg)   # block (4)

    if reset_ancilla:                         # blocks (5)-(7): clear the flag
        self.add_classical(-X, y_bits, a_reg)
        self.cx(y_bits[n], ancilla_bit)
        self.x(ancilla_bit)
        self.add_classical(X, y_bits, a_reg)
```

Read it against the table in §5 line by line: `y_bits[n]` is the sign qubit, `ancilla_bit` is t, and the last four lines are blocks (5), (6), (7) — recompute the flag from the answer, XOR it against itself, restore. (The code performs the CNOT before the X where the note performs the X before the CNOT; both compute $t \mapsto \neg(t \oplus s)$.) The merge means the uncontrolled modular adder costs *four* constant additions rather than five — compile-time constants at work, as promised in the remark of §4.

The controlled version cannot merge blocks (1) and (2), because block (1) must switch with the control while block (2) must always fire. It is also where the code makes a different — and instructive — choice of which blocks carry the control:

```
def c_add_classical_modulo(self, control_reg, X, y_reg, ancilla_bit,
                           overflow_bit, N, a_reg=None, reset_ancilla=True):
    y_bits = y_reg[:] + [overflow_bit]
    n = math.ceil(math.log2(N))

    self.c_add_classical(control_reg, X, y_bits, a_reg)  # (1), controlled
    self.add_classical(-N, y_bits, a_reg)                # (2), uncontrolled
    self.cx(y_bits[n], ancilla_bit)                      # (3)
    self.c_add_classical(ancilla_bit, N, y_bits, a_reg)  # (4), controlled on t

    if reset_ancilla:
        self.add_classical(-X, y_bits, a_reg)            # (5), uncontrolled!
        self.mcx(control_reg + [y_bits[n]], ancilla_bit) # (6), CNOT -> Toffoli
        self.x(ancilla_bit)
        self.add_classical(X, y_bits, a_reg)             # (7), uncontrolled!
```

Two of seven, not three of seven

Where §7 controlled blocks (1), (5), (7), the code controls block (1) and, instead of controlling the additions of blocks (5) and (7), upgrades block (6)'s flag-clearing CNOT to a Toffoli. Both choices are correct, and both are licensed by the same lemma; only the residue argument changes. Control off, code version: block (2) drives the register to $y - N < 0$, sign set, t flips on, block (4) adds N back — the familiar conspiracy. Then block (5) subtracts X *unconditionally*, the Toffoli does nothing (its outer control is off), the X gate undoes the earlier flip of t, and block (7) adds X back: the residue $(-X)$ then $(+X)$ cancels as pure conjugation. Control on: after block (5) the sign qubit holds $\neg t$ (Theorem 5.1's proof), so the Toffoli sets $t \mapsto t \oplus \neg t = 1$ and the X clears it. The bargain: two additions run uncontrolled — saving $O(n)$ controlled gates each — at the cost of upgrading one CNOT to a Toffoli, which is $O(1)$.

Remark (the hidden QFT sandwiches). Beauregard's drawn circuit (his Fig. 5 [8]) keeps the value register in Fourier space for the whole modular adder and drops out only twice — a $\mathrm{QFT}^{-1} \cdots \mathrm{QFT}$ pair around each read of the sign qubit — because a two's-complement sign is a computational-basis fact, invisible to a register that stores its value in phases. The code contains the same structure without ever writing it: every `add_classical` of the Fourier backend closes with an inverse QFT and the next opens with a QFT, so wherever two additions abut, the pair is redundant (Beauregard draws nothing there, and a transpiler cancels it), and wherever a CNOT to the flag intervenes — blocks (3) and (6) — the pair survives as exactly his sandwich. The RC backend has no such subtlety: its register never leaves the computational basis, which is one more reason its gate count is a factor of n better and its qubit count $n+2$ worse.

26.2 The multiplier, out of place and then in place

Level 3 is a four-line loop — Theorem 6.1 transcribed. Note the constant `((A % N) * 2**i) % N` is the shifted multiple $2^i A \bmod N$, computed in Python at circuit construction time — "compile time" in the note's vocabulary:

```
def c_add_quantum_modulo(self, control_reg, x_reg, y_reg, ancilla_bit,
                         overflow_bit, N, a_reg=None, A=1):
    """|c>|x>|y>|0> -> |c>|x>|(y + c*A*x) mod N>|0>"""
    for i in range(len(x_reg)):
        self.c_add_classical_modulo(
            control_reg=control_reg + [x_reg[i]],   # doubly controlled
            X=((A % N) * 2**i) % N,                 # 2^i A mod N, classical
            y_reg=y_reg, ancilla_bit=ancilla_bit,
            overflow_bit=overflow_bit, N=N, a_reg=a_reg)
```

Level 4 — Theorem 7.1 — is the multiply/swap/uncompute sandwich, with the modular inverse handed to Python's built-in `pow`:

```
def c_multiply_modulo(self, control_reg, A, x_reg, y_reg, overflow_bit,
                      ancilla_bit, N, a_reg=None, ...):
    """|c>|x>|0> -> |c>|A^c x mod N>|0>"""
    # CMULT(A):  |x>|0> -> |x>|Ax mod N>
    self.c_add_quantum_modulo(control_reg, x_reg, y_reg,
                              ancilla_bit, overflow_bit, N, a_reg, A=A)
    # controlled SWAP: n Fredkin gates sharing the control
    for i in range(n):
        self.append(SwapGate().control(k), control_reg + [x_reg[i], y_reg[i]])
```

```
    # CMULT(A^{-1})^dagger: uncompute x from the product
    B = pow(A, -1, N)                       # extended Euclid, classical
    self.c_add_quantum_modulo(control_reg, y_reg, x_reg,
                              ancilla_bit, overflow_bit, N, a_reg, A=-B)
```

Remark (the dagger, implemented as a minus sign). Theorem 7.1 calls for $\mathrm{CMULT}(\bar{a})^\dagger$ — run the $\bar{a}$-multiplier backwards. The code never reverses a circuit: it passes $A = -B$ and lets the constant reduction fold $-\bar{a} \bmod N = N - \bar{a}$ into an ordinary forward multiplier. The two are the same operator, because the inverse of "add $\bar{a}x$ to this register mod N" is exactly "subtract $\bar{a}x$ from this register mod N". Adjoints of permutations are permutations, and for adders they are again adders — one more classical fact doing quantum work.

27 Level 5 and the two order-finding circuits

The ladder is the last, shortest loop — rung constants by repeated squaring, one controlled in-place multiplier per counting qubit:

```
def exponentiate_modulo(self, A, x_reg, y_reg, ancilla_reg, N, a_reg=None):
    """|x>_m |y>_n |0>_{n+2} -> |x>_m |(A^x y) mod N>_n |0>_{n+2}"""
    n = math.ceil(math.log2(N))
    for i in range(len(x_reg)):
        self.c_multiply_modulo(
            control_reg=x_reg[i],
            A=pow(A, 2**i, N),              # the rung constant a^{2^i} mod N
            x_reg=y_reg, y_reg=ancilla_reg[:n],
            overflow_bit=ancilla_reg[n], ancilla_bit=ancilla_reg[n + 1],
            N=N, a_reg=a_reg)
```

and the full order-finding circuit of §8 — Hadamards, ladder, inverse QFT, measurement — assembles in a dozen lines (`shor.py`):

```
def order_finding_circuit(A, N, precision=None, adder="qft_adder"):
    n = math.ceil(math.log2(N))
    m = precision if precision is not None else 2 * n   # t = 2n counting qubits

    control_register = QuantumRegister(m)
    target_register  = QuantumRegister(n)
    ancilla_register = QuantumRegister(n + 2)
    output_register  = ClassicalRegister(m, name="output_bits")
    qc = QFTAdderCircuit(control_register, target_register,
                         ancilla_register, output_register)

    for i in range(m):
        qc.h(control_register[i])           # Hadamard layer
    qc.x(target_register[0])                # target = |1>
    qc.exponentiate_modulo(A=A, x_reg=control_register,
                           y_reg=target_register,
                           ancilla_reg=ancilla_register, N=N)
    qc.compose(QFTGate(m).inverse(), qubits=control_register, inplace=True)
    qc.measure(control_register, output_register)
    return qc
```

Count the qubits: $2n$ counting + n target + $(n{+}2)$ scratch (n for the out-of-place product, the sign qubit, the flag t) $= 4n + 2$ — with the RC backend, $n{+}2$ more for `a_reg`, hence $5n + 4$.

27.1 One control qubit: the $2n + 3$ circuit

Beauregard's headline saving [8] is that the $2n$ counting qubits can be replaced by *one*, recycled $2n$ times. The idea, due to Mosca–Ekert [14] and Parker–Plenio [15], rests on the semiclassical Fourier transform of Griffiths–Niu [13]: in the inverse QFT, every two-qubit controlled rotation is controlled by a qubit that is *about to be measured anyway* — so measure it first, and replace the quantum control by a classical one conditioned on the outcome. The counting register then never needs to exist all at once. One qubit is Hadamarded, drives the controlled multiplication for its rung, receives phase corrections determined by all *previously measured* bits, is measured, and is reset for the next rung:

```
    c_bit = control_register[0]             # the one and only counting qubit
    for i in range(m):
        qc.h(c_bit)
        qc.c_multiply_modulo(               # rung i, MSB first
            control_reg=c_bit, A=pow(A, 2 ** (m - i - 1), N),
            x_reg=target_register, y_reg=ancilla_register[:n],
            overflow_bit=ancilla_register[n],
            ancilla_bit=ancilla_register[n + 1], N=N)
        for j in range(i):                  # semiclassical inverse QFT:
            with qc.if_test((output_register[j], 1)):     # classically
                qc.p(-math.pi / 2 ** (i - j), c_bit)      # conditioned phases
        qc.h(c_bit)
        qc.measure(c_bit, output_register[i])
        with qc.if_test((output_register[i], 1)):
            qc.x(c_bit)                     # reset for the next rung
```

The quantum inverse QFT has disappeared entirely, traded for mid-circuit measurement and feed-forward (Qiskit's `if_test` control flow — on hardware, "dynamic circuits"). Total: $1 + n + (n{+}2) = 2n + 3$ qubits, the count in the title of Beauregard's paper. The two variants are toggled by one argument, and they sample from the same distribution: nothing about the *arithmetic* changed, only about where the counting register lives in time.

27.2 The bill, as the package pays it

With $n = \lceil \log_2 N \rceil$ and the package's own accounting [26] (gates counted in the Qiskit basis before transpilation — see the note in the README on why such counts are only comparative):

Backend	qubits	qubits (one-control)	gates	depth
Fourier [7, 8]	$4n + 2$	$2n + 3$	$O(n^4)$	$O(n^3)$
with approximate QFT [11, 12]	$4n + 2$	$2n + 3$	$O(n^3 \log n)$	$O(n^2 \log n)$
Ripple-carry [9]	$5n + 4$	$3n + 5$	$O(n^3)$	$O(n^3)$

The Fourier backend's extra factor of n is the QFT sandwich around every addition; the ripple-carry backend's extra $n{+}2$ qubits are the scratch register for constant encoding. The package also implements, as a standalone method (`c_add_quantum_optimized_depth`), the depth-$O(n)$ controlled Fourier addition of Pavlidis and Gizopoulos [10], which moves the control off the $O(n^2)$ rotation block — the rotations run uncontrolled, conjugated by controlled bit flips of the operand — but this trick does not compose with the modular reduction of §26, so the order-finding circuit does not use it; their full architecture reworks the modular level too, and is beyond both the package and this note.

Beauregard's own ledger [8] prices each level of the Fourier route, with $k_{\max}$ the approximate-QFT cutoff: ϕADD(a) is n+1 qubits at depth $O(1)$; the doubly controlled modular adder n+4 qubits, $O(n\,k_{\max})$ gates, depth $O(n)$; CMULT and c-U_a are $2n$+3 qubits at depth $O(n^2)$; the whole circuit $2n$+3 qubits, $O(n^3 \lg n)$ gates, depth $O(n^3)$. His $2n+3$ was, at the time, the last word in a decade-long qubit race:

Year	Construction	Qubits	Notes
1996	Vedral–Barenco–Ekert [4]	$7n+1$	$5n+2$ with basic optimisation
1996	Beckman–Chari–Devabhaktuni–Preskill [5]	$5n+1$	$4n+1$ with unbounded Toffolis
1998	Zalka [6]	$3n + O(\lg n)$	elementary gates only
2003	Beauregard [8]	$2n+3$	the one-control circuit of §27.1

At $N = 15$ the $2n+3$ circuit is eleven qubits. The 2001 NMR experiment that factored 15 [24] used seven — by compiling for that one modulus, whose rung constants $7, 4, 1, 1, \ldots$ (§8) make most of the ladder the identity. General-purpose and special-cased circuits are different objects, and only the former is priced by the table above.

28 Cross-check: factoring 15 from scratch

The same two papers admit more than one faithful implementation, and it is instructive to read a second. *Building Shor's algorithm from scratch: factoring 15* [25] is an independent Qiskit notebook that assembles the identical Draper–Beauregard stack — its QPE preamble follows Nielsen–Chuang [23] — one class per level of the tower, each verified on deterministic test vectors before the next is built (5 through the QFT and back; (5+3) mod 9 = 8 through the modular adder; (7+3·3) mod 7 = 2 through the multiplier). The correspondence is exact:

Object	`qiskit-shor`	notebook [25]
QFT (Level 0/1)	`QFTFullGate`	`QFTBlock`
ϕADD(a) (§25.2)	`add_classical`	`QAdderClassicalBlock`
modular adder (Thm. 5.1)	`add_classical_modulo`	`add_mod_N_classic`
CMULT (Thm. 6.1)	`c_add_quantum_modulo`	`CMULTaMODN_c`
c-U_a (Thm. 7.1)	`c_multiply_modulo`	`Controlled_U_a`
ladder + QPE (§8)	`order_finding_circuit`	final assembly cell

Three divergences are worth reading closely, because each lands on a point the note has already made.

Remark (three of seven, again). The notebook builds its doubly controlled additions by calling Qiskit's `.control(2)` on the ϕADD gates of blocks (1), (5) and (7) — Beauregard's original prescription, and precisely the 3-of-7 choice of §7. So the two codebases realise the *two* correct control placements side by side: blocks (1), (5), (7) in the notebook; block (1) plus the Toffoli'd flag-clear in `qiskit-shor` (§26). Diff them and the residue argument of the control lemma is the only invariant.

Gift 1, forgone

The notebook's ladder does *not* precompute rung constants. It applies the same $c\text{-}U_a$ gate 2^i times for counting qubit i:

```
for j in range(2**i):  qc.append(Ctrl_U_a_gate, ...)
```

— $1+2+4+8 = 2^t - 1 = 15$ multiplier applications where `qiskit-shor` uses $t = 4$ rungs with `pow(A, 2**i, N)`. This is the exponential sum of §1.1, live in running code. At $N = 15$ it costs a factor of four and nobody notices; at any cryptographic size it is the difference between an algorithm and a demonstration. Gift 1 is two lines of Python — and it is still a gift you must actually accept.

Remark (an undersized counting register). The notebook sets $t = n = 4$ rather than $t = 2n = 8$, noting the shortcut. For this instance it is sound: the order $r = 4$ divides 2^t, so the phase s/r is *exactly* representable in 4 bits and no rounding occurs. For general N the order does not divide 2^t, the measured value only approximates s/r, and the $2n$ bits are what guarantees the continued-fraction step converges to the right denominator [1, 3]. Another instance of 15's exceptional friendliness (§8) — and of why it proves so little.

Read the histogram, not the factors

With $a = 7$, $N = 15$, $r = 4$, $t = 4$, theory owes four sharp peaks: the measurement should return $y = 2^t s/r \in \{0, 4, 8, 12\}$, a quarter of the shots each. The run recorded in the notebook shows instead a *flat* distribution — all sixteen outcomes near 512 of 8 192 shots, and measured phases like 3/16 and 5/16 that no $s/4$ can produce. That flatness is a diagnosis, not statistics: it is the uniform-noise signature §1.4 derives for a counting register left entangled with something — interference killed by $\langle g_{x'} | g_x \rangle = 0$ — somewhere in the final assembly, even though every gate-level unit test passed deterministically. The notebook's retry loop still ends by printing 3×5, because its classical tail — continued fractions, then *verify* $\gcd(a^{r/2} \pm 1, N)$ — filters random candidates until one passes, which uniform noise eventually supplies. Two lessons, both cheap. End-to-end correctness lives in the *distribution*, so test the distribution (the seeded `find_order` test of §29 is exactly such a test). And a pipeline whose classical tail can succeed on white noise will not tell you when its quantum head has failed — the algorithm's own robustness is a bug-mask.

29 Running it

What remains of Shor's algorithm [1] outside the circuit is classical, and `shor.py` contains all of it. `find_order` builds either circuit, transpiles it, samples it, and reads the order out of the measurement distribution — the continued-fraction step is Python's `Fraction`:

```
x = int(best_output, 2)                          # measured phase, t bits
r = Fraction(x / 2**precision).limit_denominator(N - 1).denominator
if pow(A, r, N) == 1:                             # verify classically
    return r
```

— exactly the "continued fractions recover r" of Part III, with the candidate checked by one classical exponentiation. `find_factor` wraps it in the standard loop: strip factors of 2 and prime powers, pick a random base, gcd-check it (the lucky case has already factored N), find the order, and try

$\gcd(a^{r/2} - 1, N)$ when r is even. End to end, on a noise-free simulator:

```
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import SamplerV2 as AerSampler
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_shor.shor import find_factor, find_order

sim = AerSimulator()
pm = generate_preset_pass_manager(backend=sim, optimization_level=1)

order, dist = find_order(7, 15, AerSampler(), pm, num_shots=100,
                         one_control_circuit=True)   # -> 4
factor = find_factor(15, AerSampler(), pm, num_tries=3,
                     num_shots_per_trial=100)        # -> 3 or 5
```

What the package machine-checks

The note's "Verified" boxes have counterparts in the repository's `pytest` suite: unit tests drive every arithmetic method on all (or sampled) basis states through Qiskit's simulator — additions, modular additions, multipliers, for both backends, including that every ancilla returns to $|0\rangle$ — and the end-to-end tests recover the order $r = 4$ of both $A = 2$ and $A = 7$ modulo 15, with both circuit variants, on a seeded `AerSimulator`. The same discipline at two scales: trust nothing you have not enumerated.

What this does not demonstrate

Simulation cost grows exponentially in qubit count, so noise-free demonstrations stop around N of a few bits — and on 2025-era hardware, noise, not qubit count, is the binding constraint: a few hundred physical qubits exist, but the $\sim 2\,000$-gate two-qubit-depth of even the $N = 15$ one-control circuit far exceeds current coherence [26]. The package is the tower of Part II made executable and testable, not a factoring machine; the gap between the two is the fault-tolerance overhead priced in Part III's bill.

Part VII — The other Shor: elliptic-curve discrete logarithms

Why a second half

Everything so far factors integers. The same three layers — Hadamards, an oracle ladder, an inverse QFT — also break elliptic-curve cryptography, and the outer two layers are unchanged. What changes is the middle, and it changes completely.

For factoring, the group is $(\mathbb{Z}/N)^\times$ and the oracle multiplies by a *classical* constant. Multiplication by a constant is cheap: Part II built it out of shifted additions. For the discrete logarithm on a curve, the group is $E(\mathbb{F}_p)$ and the oracle adds a classical *point* — and the chord-and-tangent law needs a division, i.e. a modular *inverse*, of a quantum value. That single fact is the whole story of this part: an inversion costs an order of magnitude more than the multiplication it replaces, and every optimisation below is an attempt to make it cheaper, do it less often, or avoid it altogether.

30 What changes, and what does not

Let E be an elliptic curve over $\mathbb{F}_p$, P a point of prime order r, and $Q = [k]P$ for an unknown k. Recovering k is the *elliptic-curve discrete logarithm problem* (ECDLP), and it is what ECDSA, ECDH and every deployed curve-based scheme rest on.

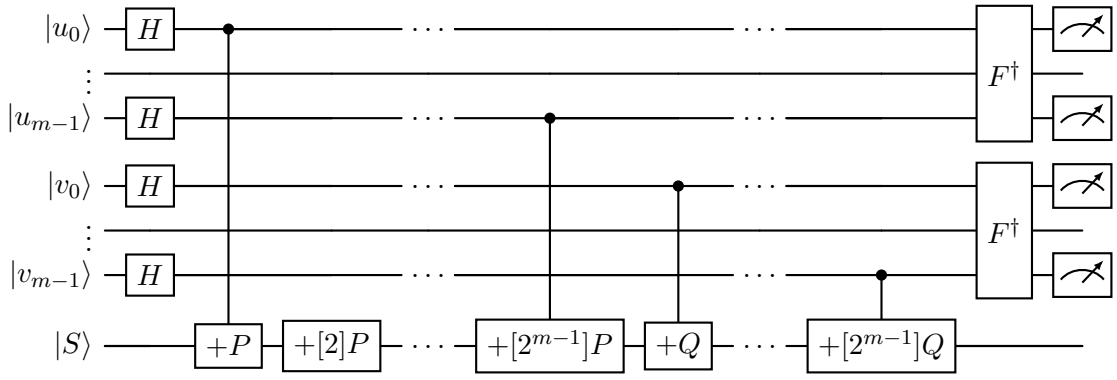
Shor's algorithm for it [28] uses two counting registers instead of one:

$$\frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle|\mathcal{O}\rangle \xrightarrow{\text{oracle}} \frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle|[u]P+[v]Q\rangle \xrightarrow{F^\dagger\otimes F^\dagger} \text{measure } (j_1,j_2).$$

The third register depends on (u,v) only through $u+kv \bmod r$, so its level sets are cosets of the lattice generated by $(-k,1)$ and $(r,0)$. The Fourier transform concentrates on the dual lattice, and a measured pair obeys, with $a_i = \mathrm{round}(j_i r/2^m)$,

$$a_2 \equiv k\,a_1 \pmod r, \qquad \text{so} \qquad k = a_2\,a_1^{-1} \bmod r \tag{6}$$

whenever a_1 is invertible — which discards $j_1 = 0$ automatically. One measurement suffices with constant probability; §41 shows the distribution this produces on a toy curve, and §41.1 the search that [38] adds when 2^m is not a multiple of r.



Each rung adds a *classical* point — the multiples $[2^i]P$ and $[2^i]Q$ are precomputed on a laptop — controlled by one counting qubit. There are $2m \approx 2n$ of them, against the $2n$ rungs of Part II's ladder, so at the level of *counting* nothing has changed. The cost is entirely inside the box marked $+P$.

The accumulator never starts at infinity

The diagram starts the third register at a fixed point S, not at the identity $\mathcal{O}$. Affine addition formulas cannot represent the point at infinity at all: there is no (x, y) for it, and the slope λ is undefined the moment the accumulator equals $\pm$ the addend. Starting at a constant S and reading the answer relative to it keeps the accumulator on an ordinary point throughout, and shifts the computed coset without touching the interference. The same device appears as the offset T in the windowed constructions of §36.8, and as `P0` in the Classiq tutorial [49].

31 Why it works

Section 30 asserted (6). This section derives it. The argument is short, it is the only place in this part where the *quantum* content lives — everything after §32 is reversible arithmetic — and it is checked numerically in `bench/ec_ablation.py`, which evaluates the amplitudes by brute force and compares them against the closed form below.

31.1 The state, in four lines

Write $q = 2^m$ for the size of each counting register and $\omega = e^{2\pi i/q}$. Start from a uniform superposition and apply the oracle:

$$\frac{1}{q}\sum_{u,v}|u\rangle|v\rangle|S\rangle \;\longrightarrow\; \frac{1}{q}\sum_{u,v}|u\rangle|v\rangle\,\big|\,S + [u]P + [v]Q\,\big\rangle = \frac{1}{q}\sum_{u,v}|u\rangle|v\rangle\,\big|\,S + [u + kv]P\,\big\rangle,$$

the last equality being the only place $Q = [k]P$ is used. The third register therefore depends on (u, v) *only* through

$$w \;=\; u + kv \bmod r,$$

so group the terms by w:

$$\frac{1}{q}\sum_{w\in\mathbb{Z}_r}\left(\sum_{(u,v)\,:\,u+kv\equiv w}|u\rangle|v\rangle\right)\big|\,S + [w]P\,\big\rangle. \tag{7}$$

The third register is never measured and never needs to be: (7) is already a sum of orthogonal branches, so the reduced state on (u, v) is a mixture over w, and it is enough to analyse one branch.

Why the offset S cannot matter

S appears only inside the third register's label. It relabels which basis state each branch attaches to; it does not touch the (u, v) amplitudes at all, so it cannot change the measured distribution. The same argument covers w: every branch of (7) gives the same |amplitude|, which the benchmark confirms directly by computing two different branches and comparing (result: `yes`). S is there for the arithmetic — to keep the accumulator off the point at infinity — and for nothing else.

31.2 Two lattices

Fix a branch w. Its (u, v) set is a coset of

$$L \;=\; \{(u,v) \in \mathbb{Z}^2 \;:\; u + kv \equiv 0 \pmod r\} \;=\; \big\langle\, (r,0),\ (-k,1) \,\big\rangle,$$

the *period lattice*. The two generators say the two things that can be done without changing the answer: take r steps in u, or take one step in v and k steps back in u. Its determinant is r, so $[\mathbb{Z}^2 : L] = r$ and the plane splits into exactly r cosets — one per value of w, as it must.

The discrete logarithm is the slope of the second generator. Finding L is finding k; that is the whole problem, restated.

A Fourier transform does not see L, it sees the *dual*

$$L^* \;=\; \{\, y \in \mathbb{Q}^2 \;:\; \langle y, x\rangle \in \mathbb{Z} \;\; \forall x \in L \,\} \;=\; \big\langle\, \tfrac{1}{r}(1,k),\ (0,1) \,\big\rangle,$$

obtained as $(B^{-1})^{\mathsf{T}}$ from L's basis B. Writing a point of L^* as $(a_1/r,\, a_2/r)$ and clearing denominators, membership says exactly

$$a_2 \equiv k\, a_1 \pmod r,$$

which is (6). It is not a consequence of the dual lattice — it *is* the dual lattice, written in coordinates.

31.3 The amplitude

Take $r \mid q$ for a moment and write $q = rM$. Then for each v the admissible u in $[0, q)$ are $u_0(v) + tr$ with $u_0(v) = (w - kv) \bmod r$ and $t = 0, \ldots, M-1$ — exactly M of them. The amplitude at (j_1, j_2) after the transform is, up to normalisation,

$$A(j_1, j_2) \;\propto\; \sum_{v=0}^{q-1} \omega^{v j_2}\, \omega^{u_0(v) j_1} \underbrace{\sum_{t=0}^{M-1} \omega^{t r j_1}}_{\text{(i)}}.$$

Sum (i) is geometric with ratio $\omega^{r j_1}$: it equals M when $r j_1 \equiv 0 \pmod q$, i.e. when $M \mid j_1$, and *exactly zero* otherwise. So only $j_1 = a_1 M$ survives. For those, $\omega^{u_0 j_1} = e^{2\pi i\, u_0 a_1 / r}$ depends on u_0 only mod r — which is why reducing u_0 into $[0, r)$ above was harmless — and $u_0 \equiv w - kv$, leaving

$$A \;\propto\; e^{2\pi i\, w a_1 / r} \sum_{v=0}^{q-1} e^{2\pi i\, v\,(j_2 - k a_1 M)/q}.$$

That second sum is q when $j_2 \equiv k a_1 M \pmod q$ and zero otherwise. Writing $j_2 = a_2 M$ and dividing through by M:

$$\boxed{\; a_2 \equiv k\, a_1 \pmod r \;}$$

and every surviving outcome has $|A|^2 = 1/r$. The distribution is *uniform on the r points of the dual lattice*, and nothing else has any amplitude at all.

r	k	q	outcomes	each	closed form exact	usable $\varphi(r)/r$
4	3	16	4	1/4	✓	0.50
8	3	32	8	1/8	✓	0.50
4	1	64	4	1/4	✓	0.50

31.4 Reading off k, and what it costs

Given (a_1, a_2) with $a_2 \equiv ka_1$, recover $k = a_2 a_1^{-1} \bmod r$ — possible exactly when a_1 is invertible. For prime r that means $a_1 \neq 0$, so $r-1$ of the r outcomes work and one run succeeds with probability $(r-1)/r$: overwhelming, and it does not improve with more qubits because the $a_1 = 0$ outcome is a fixed $1/r$ of the mass. Note r is not secret — it is the curve's published group order — so this is a division, not a search.

The clean case never actually happens

Everything above assumed $r \mid q$. For ECDLP r is *prime*, and q is a power of two, so $r \mid q$ only when $r = 2$. The exact calculation is a device for seeing the structure; the real situation is always the other one.

What goes wrong is only sum (i): with $r \nmid q$ it no longer vanishes, it becomes a Dirichlet kernel. Amplitude leaks onto the integers either side of $a_1 q/r$, which is why the measured peaks of §41.1 sit at $(6, 5)$ and $(2, 3)$ rather than at the exact dual-lattice points $(1.6, 3.2)$ and $(3.2, 6.4)$, and why $a_i = \text{round}(j_i r/q)$ has a rounding in it at all. The leak shrinks as q grows past r. Entries are $P(\text{usable and correct})$, i.e. the chance of an outcome with a_1 invertible *and* $a_2 \equiv ka_1$; the last column is the irreducible $a_1 = 0$:

	$m - \lceil \log_2 r \rceil$				
r	0	1	2	3	$P(a_1{=}0)$
5	60%	67%	74%	77%	20%
7	48%	67%	77%	81%	15%
11	69%	77%	84%	88%	9%
13	63%	75%	84%	88%	8%

At $m = \lceil \log_2 r \rceil$ the answer is barely better than a coin; three extra qubits per register take it to 88%, with the residual 8 being the irreducible $a_1 = 0$ outcome. This is the whole content of [38]'s analysis for the discrete logarithm, and the reason the post-processing of §41.1 searches a small neighbourhood of the rounded value rather than trusting it.

32 The group law, and why it resists reversibility

For $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ on $y^2 = x^3 + ax + b$ with $x_1 \neq x_2$,

$$\lambda = \frac{y_1 - y_2}{x_1 - x_2}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_2 - x_3) - y_2. \tag{8}$$

Three subtractions, one squaring, one multiplication — and one division. In Part II every arithmetic step was addition or multiplication by a compile-time constant. Here the divisor $x_1 - x_2$ is a quantum value, so the inverse must be computed *by circuit*, on a superposition, and then uncomputed. Section 34 does that, and §37 shows it dominating everything else by a factor of three to five.

Level	Object	Made of	In code
5	the two ladders	$2m$ rungs, one per counting qubit	`ec_shor.py`
4	c-$(+P_i)$	4 sub, 1 add, 2 div, 2 mul, 1 sqr, 1 neg	`ec_pointadd.py`
3	$x^{-1} \bmod p$	$2n$ Kaliski rounds + correction	`ec_kaliski.py`
3′	$x \cdot y \bmod p$	n modular doublings + n additions	`ec_mult.py`
2	$\pm \bmod p$, $2x \bmod p$	plain add/sub plus one reduction	`ec_modarith.py`
1	ADD	CDKM chain, or Gidney's AND chain	`ec_adders.py`
0	AND	$4T$ to compute, $0T$ to uncompute	`ec_gates.py`

Level 3 is new and it is the expensive one. Everything below it is Part II's tower with the modulus renamed.

33 Level 2: modular arithmetic over $\mathbb{F}_p$

Two blocks carry everything above them, and both are from Roetteler, Naehrig, Svore and Lauter [29]. Neither is hard; both are worth seeing, because §36.3 makes its savings by attacking exactly the parts described here as "the awkward bit".

Modular doubling. Shift, subtract p, add it back if that went negative:

$$2x \;\longrightarrow\; 2x - p \;\longrightarrow\; \begin{cases} 2x - p & 2x \geq p \\ 2x & \text{otherwise.} \end{cases}$$

The shift is free — it renames wires. The awkward bit is the borrow flag, which must return to $|0\rangle$ or it is garbage in the sense of §1.4. It clears for one CNOT: $2x$ is even and p is odd, so *the answer is even exactly when no reduction happened*, and the answer's own low bit is the flag.

Modular addition. The same shape, but the flag-clearing trick is prettier. After adding and subtracting p, the flag holds $[\,x + y < p\,]$, i.e. "no reduction". And

- no reduction means the answer is $x + y$, which is $\geq x$;
- a reduction means the answer is $x + y - p$, which is $< x$ because $y < p$.

So $[\,\text{answer} < x\,]$ is exactly the complement of the flag, and one comparison against an operand that is still lying there clears it. No extra ancilla, no second copy. In code:

```
def cmodadd(m, ctrl, x, y, p):
    """y <- (y + x) mod p when ctrl (or unconditionally if ctrl is None).
    ..."""
    ctx, n = m.ctx, len(y)
    assert len(x) == n
    anc_x, anc_y = m.anc(1, "ax"), m.anc(1, "ay")
    xe, ye = x + anc_x, y + anc_y
    cp, a = m.anc(n + 1, "cp"), m.anc(n + 1, "a")

    if ctrl is None:
        A.add(ctx, xe, ye, a)
    else:
        A.cadd(ctx, ctrl, xe, ye, cp, a)
    A.sub_const(ctx, ye, p, cp, a)
    A.cadd_const(ctx, anc_y[0], y, p, cp[:n], a)
```

```
    if ctrl is None:
        A.lt_uint(ctx, y, x, anc_y[0], a)
    else:
        A.clt_uint(ctx, ctrl, y, x, anc_y[0], a, cp)
    ctx.x(anc_y[0])
    m.free(cp, a, anc_x, anc_y)
```

The doubling is the same shape and its flag-clearing is one CNOT — the last two lines below, where `xe[0]` is the answer's own low bit:

```
def moddbl(m, x, p):
    """x <- 2x mod p, in place: the register handle stays valid.
    ..."""
    ctx, n = m.ctx, len(x)
    z = m.anc(1, "z")
    xe = A.shift_up(x, z[0])                    # n+1 bits, holds 2x
    _moddbl_body(m, xe, p)
    # value bits now sit at xe[0..n-1]; xe[n] (the old MSB) is clean
    for i in range(n - 1, -1, -1):
        ctx.swap(xe[i], xe[i + 1])
    m.free(z)
```

34 Level 3: inversion by Kaliski's algorithm

The binary extended Euclidean algorithm branches four ways per step: u even, v even, both odd with $u > v$, both odd with $u \le v$. A quantum circuit cannot branch — every branch must run on every input, on a superposition. Kaliski's formulation as reworked by Häner, Jaques, Naehrig, Roetteler and Soeken [30] makes all four cases the *same* straight line:

repeat $2n$ times:

1. **if** v odd **and** (u even **or** $u > v$): swap (u, v), swap (r, s); remember it
2. **if** u, v both odd: $v \mathrel{-}= u$; $s \mathrel{+}= r$
3. $v \mathrel{/}= 2$; $r \mathrel{\times}= 2$ (the doubling only if the round fired)
4. **if** it swapped: swap back

Started at $(u, v, r, s) = (p, x, 0, 1)$ this terminates with $u = 1$, $v = 0$, $s = p$ and

$$r \;=\; -x^{-1}\, 2^{k} \bmod p, \tag{9}$$

where k counts the rounds that did anything. That is a *pseudo*-inverse: right up to a sign and a power of two, both of which the caller fixes — one modular negation, and k modular halvings driven by a counter. Equation (9) is asserted directly in `tests/test_ec_kaliski.py` for every invertible x and every p tested, alongside $u = 1$, $v = 0$, $s \equiv 0$ and the counter itself.

Three bits per round, and why none can be dropped

The round is a permutation of (u, v, r, s) chosen by its decision bits, so those bits must survive or the round is not reversible. Three are kept:

- f — *did this round do anything* ($v \neq 0$ on entry). Not recoverable afterwards: an inert round and the last active round both leave $v = 0$.
- a — *did it swap.*
- b — *were both odd*, so the subtract-and-add fired. Not implied by (f, a): the case $f = 1, a = 0$ covers both "v was even" and "both odd with $u \leq v$".

The comparison $u > v$ is *not* kept. It is consumed while building a and then uncomputed by running the comparator a second time — before u and v change, which is the only window in which that is possible.

In code, with the decision bits built first and the comparison uncomputed before u and v move:

```
def kaliski_round(m, u, v, r, s, p, rec, conditional=True):
    ctx, n = m.ctx, len(u)
    f, a, b = rec

    # --- f: is this round active? ---------------------------------------
    if conditional:
        MA.is_nonzero(m, v, f[0])
    else:
        ctx.x(f[0])                       # [106] Sec 3.3: never test v != 0

    # --- decision bits, from the pre-round u and v ----------------------
    ...

    # --- the straight-line body -----------------------------------------
    for i in range(n):
        ctx.cswap(a[0], u[i], v[i])
    for i in range(len(r)):
        ctx.cswap(a[0], r[i], s[i])

    cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
    A.csub(ctx, b[0], u, v, cp, sc)       # v -= u  (no borrow: u <= v here)
    m.free(cp, sc)
    MA.cmodadd(m, b[0], r, s, p)          # s += r  mod p

    _shift_down(ctx, v)                   # v /= 2  (v is even by now)
    MA.cmoddbl(m, f[0], r, p)             # r *= 2  mod p

    for i in range(n):
        ctx.cswap(a[0], u[i], v[i])
    for i in range(len(r)):
        ctx.cswap(a[0], r[i], s[i])
```

r and s are reduced mod p every round. They have to be: r doubles each round, and the tests confirm it runs past p within a few rounds if left alone. u and v are plain n-bit integers, staying in $[0, p]$ throughout.

Division, and where the garbage goes. What a point addition actually wants is $\lambda = y/x$, and the inversion above leaves $6n$ qubits of per-round records behind. Both are solved at once by the shape of [29]: invert, multiply, copy the answer out, then *run the whole thing backwards*. Only the copy survives; records, counter, u, v and s all return to $|0\rangle$, which the simulator checks at the exact instruction each is freed.

```
def mod_div(m, ctrl, x, y, out, p, inverse_builder=None,
            multiply_builder=None):
    """out ^= ctrl * (y / x mod p).  [106] Fig. 7, the modular division.
    ..."""
    n = len(x)
    lam = m.anc(n, "lam")

    def build():
        inv = m.anc(n, "inv")
        if inverse_builder is None:
            recs, cnt = mod_inv(m, x, inv, p, ctrl=ctrl)
            garbage = [bit for record in recs for bit in record]
            if cnt is not None:
                garbage.append(cnt)
        else:
            garbage = inverse_builder(m, x, inv, p, ctrl)
        if multiply_builder is None:
            import ec_mult as MU
            MU.modmul(m, y, inv, lam, p)
        else:
            multiply_builder(m, y, inv, lam, p)
        return inv, garbage

    # forward: lam = ctrl * y * x^{-1}
    start = len(m.qc.data)
    inv, garbage = build()
    body = list(m.qc.data[start:])

    # lam is already 0 on the ctrl = 0 branch, so the copy needs no control
    for i in range(n):
        m.ctx.cx(lam[i], out[i])

    for ci in reversed(body):                          # undo everything else
        m.qc.append(ci.operation.inverse(), ci.qubits, ci.clbits)
    m.free(lam, inv, *garbage)
```

A control that has to reach all the way down — here, though not everywhere

The division is controlled by the counting qubit q, and for the inversion built above it is not enough to gate only the final copy. On the $q = 0$ branch the x register does not hold $x_1 - x_2$; by that point in the point addition it holds $x_1 + 2x_2$, which vanishes for perfectly ordinary accumulators, so the circuit would be inverting zero — and Kaliski on 0 terminates with $u = p$ and $s = 1$, not the $u = 1$, $s \equiv 0$ that the cleanup here relies on to return those registers to $|0\rangle$.

The fix costs almost nothing. Load the inversion's *setup* under the control — $u \leftarrow p$, $v \leftarrow x$, $s \leftarrow 1$ — and when $q = 0$ every register starts at zero, so $v = 0$ makes all $2n$ rounds inert, r stays 0, and the correction does nothing. The rounds themselves are never controlled; n ANDs and a handful of CNOTs buy the whole thing, and it is checked directly: with $q = 0$ the output register is 0 for every input.

[31] does *not* do this, and does not need to. Its Fig. 7 runs the inversion and the multiplication unconditionally and gates only the copy-out of λ, which is sound there because the inversion is undone by its literal inverse — whatever Inv did to the registers on a degenerate input, $\text{Inv}^\dagger$ undoes. The gating above is the price of *this* construction's cheaper cleanup, not a correction to the paper.

35 Level 4: the eleven-step point addition

The accumulator (x_1, y_1) is quantum, the addend (x_2, y_2) classical, and the control q says whether to add at all. In place, no garbage:

	operation	after it, on the $q = 1$ branch:	
		x register	y register
1	$x \mathrel{-}= x_2$	$x_1 - x_2$	y_1
2	$y \mathrel{-}= q\, y_2$		$y_1 - y_2$
3	$\lambda \mathrel{\oplus}= q \cdot y/x$		
4	$y \mathrel{\oplus}= x\lambda$		$\mathbf{0}$
5	$x \mathrel{+}= 3x_2$	$x_1 + 2x_2$	
6	$x \mathrel{-}= \lambda^2$	$x_2 - x_3$	
7	$y \mathrel{+}= x\lambda$		$\lambda(x_2 - x_3)$
8	$\lambda \mathrel{\oplus}= q \cdot y/x$		
9	$x \mathrel{-}= (2 - q)x_2$	$-x_3$	
10	$y \mathrel{-}= q\, y_2$		y_3
11	$x \leftarrow -x$ if q	x_3	

The circuit is the table, line for line:

```
def point_add_ctrl(m, q, x1, y1, x2, y2, p, multiplier="reference",
                   inversion="kaliski", word_size=None):
    """(x1, y1) <- (x1, y1) + (x2, y2) when q; unchanged when q = 0.
    ..."""
    n = len(x1)
    lam = m.anc(n, "lam")

    MA.modsub_const(m, x1, x2, p)                    # 1  x <- x1 - x2
    MA.cmodsub_const(m, q, y1, y2, p)                # 2  y <- y1 - q y2
    _moddiv(m, q, x1, y1, lam, p, multiplier, word_size, inversion)  # 3
    _modmul_xor(m, x1, lam, y1, p, multiplier, word_size)             # 4
    MA.modadd_const(m, x1, 3 * x2 % p, p)            # 5  x <- x + 3 x2
```

```
    _modsqr_sub(m, lam, x1, p, multiplier, word_size)                  # 6
    _modmul_add(m, x1, lam, y1, p, multiplier, word_size)              # 7
    _moddiv(m, q, x1, y1, lam, p, multiplier, word_size, inversion)    # 8
    _csub_2x2_or_x2(m, q, x1, x2, p)              # 9  x <- -x3
    MA.cmodsub_const(m, q, y1, y2, p)             # 10 y <- y3
    MA.cmodneg(m, q, x1, p)                       # 11 x <- x3

    m.free(lam)
```

Three things in that table are worth pausing on.

Step 4 is an XOR, not a subtraction. At that moment y holds $y_1 - y_2$ and $x\lambda$ is by definition $y_1 - y_2$, so XOR-ing the product in zeroes the register, and the algebra guarantees it. What is saved is the *accumulation* — CNOTs instead of a modular subtraction. The product $x\lambda$ still has to be computed and uncomputed around it, which is two modular multiplications: the step is cheaper than the alternative, not cheap.

Step 8 is the same circuit as step 3, and it clears λ. By then the registers hold $x = x_2 - x_3$ and $y = \lambda(x_2 - x_3)$, so y/x is again λ, and the XOR-accumulating division undoes what step 3 wrote. The whole eleven-step arrangement exists to make that identity true; without it λ would be n qubits of garbage per addition.

Step 9 is two of [30]'s steps merged, following [31]. The original does $x \mathrel{-}= 2x_2$ then a controlled $x \mathrel{+}= x_2$; the merged form subtracts $(2 - q)x_2$ once. Building that operand is free: $2x_2 \bmod p$ and $x_2 \bmod p$ are both classical, so the register is X-gated to one of them and CNOTs from q patch the bits where they differ.

Three exceptional cases, not one

The circuit is correct except on a set of density $O(1/p)$, and it does not detect the exceptions — a check would cost more than it saves. There are three, and only the first is the one usually quoted:

- $x_1 = x_2$ — step 3 divides by $x_1 - x_2$. Doubling, adding an inverse, or infinity.
- $x_3 = x_2$, i.e. $P_1 = -[2]P_2$ — step 8 divides by $x_2 - x_3$. This one is invisible in (8) and appears only once you follow what the *registers* hold. The in-place multiplier of §36.6 reaches the same wall from the other direction, where it would be asked to multiply by zero.
- for the *windowed* circuit of §36.8 adding $\mathcal{O}$: $x_R = 0$. This one is not about the table above — it belongs to [32]'s Algorithm 1, whose own step 6 divides by $x_R - x_{P_i}$, and that is x_R when $P_i = \mathcal{O}$ is encoded as $(0, 0)$. Adding the identity looks like it could never fail.

`ec_classical.point_add_exceptional` decides the first two; the tests exclude them explicitly and report how many, rather than passing quietly on inputs they never exercise.

36 Making it fast

Everything from here on is measured. Each subsection fixes a *task* and varies one implementation choice; comparing a modular doubling against a point addition would say nothing, so nothing here does. Every variant is re-checked against the classical model *inside* the benchmark before its cost is recorded — a cheaper circuit that is wrong is not a datapoint — and variants that are deliberately approximate record a measured failure rate instead. All of it comes from `bench/ec_ablation.py`,

and the tables below are generated from its output rather than typed.

Which Toffoli count, and why it matters here more than in Part V

Both source papers count "Toffolis" as CCX, CCZ and AND gates together [32]. A measurement-based AND† contains *no* Toffoli and no T gate, so it is not counted. That is the convention used in every table below, and it is the one to compare against published figures.
It is not the only defensible convention — one could count every three-qubit operation performed, uncomputes included — and the two differ by nearly 2× on Gidney-style circuits, which is most of what this part builds. `ec_cost` reports both (`toffoli_paper` and `toffoli_equiv`) precisely so the choice is visible rather than buried. Quoting the wrong one would make every optimisation here look roughly half as good as the literature says it is.

36.1 Level 1: which adder, and how to control it

Two adders, chosen per call site by how many ancillas happen to be free — as [32] puts it, the ancilla-hungry one where ancillas are available and the ancilla-free one where they are not.

	addition		controlled addition			qubits	
				copy-then-add, with			
n	CDKM	Gidney	reconstruct	CDKM	Gidney	CDKM	Gidney
8	16	7	32	24	15	17	23
16	32	15	64	48	31	33	47
32	64	31	128	96	63	65	95

Plain addition is the $2n$-versus-n of the textbook comparison, and lands exactly there: $64 = 2n$ against $31 = n - 1$ at $n = 32$. CDKM spends two Toffolis per bit, Gidney one AND per bit, and the difference is that Gidney's targets start clean, so they uncompute for free — the $n - 1$ rather than n being the carry into the top bit, which no adder has to compute.

Controlled addition is where [31] makes its choice. Promoting the adder's CNOTs to Toffolis costs 128 at $n = 32$; copying the operand into clean ancillas *under the control* and then adding unconditionally costs 63, a factor 2.0. The copies all commute, so they also parallelise, which the reconstructed version cannot.

```
def cadd(ctx, ctrl, x, y, copy, anc):
    """y += x if ctrl, via [106] Fig. 4(b): copy under control, then add.
    ..."""
    assert len(copy) == len(x)
    for i, q in enumerate(x):
        ctx.and_(ctrl, q, copy[i])
    add(ctx, copy, y, anc)
    for i, q in enumerate(x):
        ctx.and_dg(ctrl, q, copy[i])
```

The middle column is there to keep that factor honest. Copy-then-add and the Gidney adder are two separate decisions, and the 2.0× bundles them; hold the adder fixed at CDKM and the control style alone is worth 1.33× (96 against 128). The rest is the adder.

What it costs is qubits. The last two columns are the plain adders' widths, $2n+1$ against $3n-1$; for the controlled pair the gap is wider still, 66 against 128 at $n = 32$, since the copy needs a register of its own.

For a classical constant offset, the separate module `gidney_dirty_adder` implements Gidney's dynamic $3n \pm O(1)$ construction with two clean and $n-2$ dirty ancillas. It vents carries by mid-circuit measurement and repairs their phases by classical feed-forward. It is not a drop-in replacement for a unitary adder: the opt-in modular composition and its toy affine oracle below use forward subtraction circuits instead of reversing measurements.

36.2 Level 0: temporary ANDs

Part V measured this for factoring; it applies unchanged, and matters more here because there is so much more arithmetic to apply it to. Over one whole modular inversion:

n	ANDs	AND†s	T (all Toffoli)	T (temporary AND)	saving
5	912	892	14 938	5 958	2.51×
8	2 229	2 197	36 694	14 628	2.51×
12	4 785	4 737	79 254	31 740	2.50×

A flat 2.50×, and the arithmetic says why: ANDs and AND†s arrive in near-equal numbers, so the average is $(4+0)/2 = 2$ against 7, i.e. 3.5× on the ANDs alone, diluted by the Fredkins and dirty-target Toffolis that cannot use the trick.

36.3 Level 2: approximate and pseudo-Mersenne reduction

Shor's circuit only has to work with large constant probability, and [32] spends that freedom on the reductions. Two ideas.

Compare only the top bits. "Is this at least p?" is answered correctly unless the top few bits tie, which happens with probability about $2^{-\text{msbs}}$. That shrinks an n-bit comparison to an msbs-bit one and — more importantly — lets the flag be produced *before* the subtraction rather than recovered after it, collapsing the exact two-step reduction of §33 into a single controlled subtraction.

Exploit the shape of the prime. secp256k1 uses $p = 2^u - f$ with $f = 2^{32} + 977$, tiny against 2^{256}. Then "at least p?" is almost "is bit u set?", and subtracting p becomes "clear bit u, then add f" — and since f is small, adding it need only touch the bottom few bits, because the carry cannot travel far. An n-bit constant subtraction becomes a short one.

The pseudo-Mersenne doubling is the whole idea in eight lines — note that nothing compares anything, and that clearing the overflow bit *is* the subtraction of 2^u:

```
def moddbl_pm(m, x, q, lsbs=None):
    pm = pseudo_mersenne(q)
    assert pm, f"q = {q} is not pseudo-Mersenne"
    u, f = pm
    ctx, n = m.ctx, len(x)
    assert u == n, "expected q just below 2^n"
    lsbs = lsbs or min(n, 2 * max(1, f.bit_length()) + 8)

    z = m.anc(1, "z")
    xe = A.shift_up(x, z[0])                        # n+1 bits; xe[n] is the overflow
    cp, sc = m.anc(lsbs, "cp"), m.anc(lsbs, "sc")
    A.cadd_const(ctx, xe[n], Reg(list(xe[:lsbs])), f, cp, sc)
    m.free(cp, sc)
    ctx.cx(xe[0], xe[n])                            # clears the bit == subtracts 2^u
    for i in range(n - 1, -1, -1):
        ctx.swap(xe[i], xe[i + 1])
    m.free(z)
```

		doubling		controlled addition	
p	reduction	Toffoli	failure	Toffoli	failure
251	exact (Alg. 5 / Alg. 8)	17	0.0%	49	0.0%
251	top 4 bits only (Alg. 6 / Alg. 9)	13	2.4%	39	7.0%
251	pseudo-Mersenne (Alg. 7 / Alg. 10)	5	3.2%	33	3.7%
1021	exact (Alg. 5 / Alg. 8)	21	0.0%	61	0.0%
1021	top 4 bits only (Alg. 6 / Alg. 9)	15	3.0%	45	9.7%
1021	pseudo-Mersenne (Alg. 7 / Alg. 10)	4	3.1%	41	1.3%
4093	exact (Alg. 5 / Alg. 8)	25	0.0%	73	0.0%
4093	top 4 bits only (Alg. 6 / Alg. 9)	17	3.1%	51	9.7%
4093	pseudo-Mersenne (Alg. 7 / Alg. 10)	4	3.1%	49	1.0%

Read the failure column as a rate, not a defect: these circuits are *supposed* to be wrong sometimes. The question is whether the rate tracks $2^{-\text{msbs}}$, and it does — at $p = 4093$, four MSBs give 3.1% and eight give 0.17%, a factor 18 for four bits. The pseudo-Mersenne doubling is the headline: 4 Toffolis against 25 exact, a 6.2× reduction, for a 3.1% failure rate at this width.

Failure compounds, which is why the paper's budget is so large

A single operation failing a few per cent of the time sounds cheap. Chained through the $\approx 1.4n$ iterations of an in-place multiplication it is not. Measured end to end at $p = 31$: a two-bit comparison, which fails about 12% per operation, turns a working multiplier into one that fails 81% of the time, while the full-width comparison stays exact. That is the calculation behind [32]'s choice of 40 to 50 bits at $n = 256$ — per-operation failure near 2^{-40}, so even several hundred iterations stay negligible. A cheap approximation is only cheap once you count how often you use it. The measurement is in `tests/test_ec_opt1128.py`, not an estimate.

One case is not rare. Algorithm 10 cannot reduce $x + y = p$: the sum does not reach bit u. Inside the in-place multiplier that input is certain. r must end the Bézout replay at 0 and only a swap writes r, so every multiplication with $y \neq 0$ presents $r + s = p$ exactly once, at the replay step mirroring the dialog's first swap — near the *end* of the replay, not the start. Algorithms 7 and 10 composed into the multiplier therefore fail on every such input. `ec_approx.cmodadd_pm_q` is Algorithm 11, which catches the case with an all-ones test on the top bits and XORs p out. The all-zeros test that clears its flag is gated here on $ctrl \wedge [y < x]$: the replay's opening iterations make exactly the call ($ctrl = 0$, $y = 0$) on which the ungated test misfires.

36.4 Level 3: Montgomery multiplication over a carry-save tree

[31] rewrites word-level Montgomery so that its two halves have the same shape. The textbook reduction computes $M = c_w p'_w \bmod 2^w$ and then $c \mathrel{+}= Mp$; substituting M gives $c \mathrel{+}= c_w\,(p'_w \bmod 2^w)\,p$, where the bracket depends only on the modulus. So $d = (p'_w \bmod 2^w)\,p$ is a compile-time constant and the reduction becomes a second "$c \mathrel{+}= c_w \cdot d$", structurally identical to the multiplication step. One carry-save tree then serves both. Its 3:2 compressor turns three addends into two with no carry propagation at all,

$$\begin{aligned} S_i &= A_i \oplus B_i \oplus C_i && \text{(three CNOTs)},\\ M_i &= \mathrm{MAJ}(A_i, B_i, C_i) = A_i \oplus \big((A_i \oplus B_i) \wedge (A_i \oplus C_i)\big) && \text{(one AND)}, \end{aligned}$$

with $A + B + C = S + 2M$, so reducing w addends to two costs $O(\log w)$ depth and only the final addition has to propagate a carry.

```
def csa_compress(m, A, B, C):
    """(A,B,C) -> (S, M) with A + B + C == S + M, inputs preserved.
    ..."""
    ctx, W = m.ctx, len(A)
    assert len(B) == len(C) == W
    S, M = m.anc(W, "csaS"), m.anc(W, "csaM")
    for i in range(W):                          # S = A ^ B ^ C
        ctx.cx(A[i], S[i])
        ctx.cx(B[i], S[i])
        ctx.cx(C[i], S[i])
    for i in range(W - 1):                      # M = MAJ, written one place up
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
        ctx.and_(B[i], C[i], M[i + 1])
        ctx.cx(A[i], M[i + 1])
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
    return S, M
```

multiplier	Toffoli	qubits	depth	depth vs. schoolbook
$n = 8$, $w = 2$, $p = 251$				
schoolbook double-and-add	528	46	3 211	—
Montgomery over a carry-save tree	434	88	2 600	1.2×
Montgomery over a QROM lookup	312	121	1 676	1.9×
$n = 12$, $w = 4$, $p = 4093$				
schoolbook double-and-add	1 176	66	7 311	—
Montgomery over a carry-save tree	1 078	268	4 994	1.5×
Montgomery over a QROM lookup	699	261	3 315	2.2×
$n = 16$, $w = 4$, $p = 65521$				
schoolbook double-and-add	2 080	86	13 043	—
Montgomery over a carry-save tree	1 566	324	6 669	2.0×
Montgomery over a QROM lookup	1 054	347	4 780	2.7×

What this table does *not* show, and why

The carry-save multiplier is a *depth* optimisation, and depth is the column where it wins: 2.0× over schoolbook at $n = 16$. It does not beat the QROM-lookup multiplier of [30] on gate count, and at these widths it should not be expected to — a tree over two or four words has almost nothing to flatten, and the rewrite costs real width. The two differ by $2^w \lfloor c_w p'_w / 2^w \rfloor\, p$, which with c_w and p'_w both below 2^w reaches about $2^{2w} p$ — so the reduction over-adds by that much and the accumulator settles at $n + 2w$ bits rather than the $n + 2$ of textbook Montgomery, with a final reduction to mop up. The paper does not dwell on this; the tests measure the width rather than trusting it.

36.5 Level 3: dropping the counter and postponing the reduction

Three changes to §34, all from [31] Sec 3.3. The first two compound and carry the saving; the third is a scheduling change that buys depth with qubits.

Run every round unconditionally. Drop the "is v still nonzero?" test. An inert round does nothing except double r, so instead of stopping at $-x^{-1}2^k$ the register runs on to $-x^{-1}2^{2n}$ — the

same value for *every* input, with no counter to keep and no $2n-k$ corrective halvings to perform. And 2^{2n} is exactly right: fed $x = X2^n$ in Montgomery form the answer is $-X^{-1}2^n$, the Montgomery form of $-X^{-1}$, so the power of two has corrected itself and only the sign is left — one modular negation, against [30]'s counter and its $2n-k$ controlled halvings. It also removes one of the three recorded bits per round, since f existed only to say whether the round fired.

Postpone the modular reductions. Do not reduce r and s mod p each round; let them run into a $2n$-bit register and reduce once at the end. This changes what a round *is*: a modular doubling (compare, conditionally subtract, clear the flag) becomes a plain left shift, which is a renaming of wires and costs no Toffoli at all, and a modular addition becomes a plain addition.

	Toffoli			depth			qubits	
n	[30]	[31]	saved	[30]	[31]	saved	[30]	[31]
5	1 242	817	34%	4 846	2 664	45%	83	84
7	2 328	1 593	32%	9 358	5 342	43%	109	116
8	3 045	2 077	32%	12 271	7 019	43%	125	132
10	4 647	3 237	30%	19 021	11 081	42%	151	164
12	6 585	4 653	29%	27 239	16 059	41%	177	196

At $n = 12$: 6 585 Toffolis become 4 653, a 29% saving, and depth falls by 41%, for about 11% more qubits.

[31] reports 58–60% for division at $n = 192$–521, but on a different metric: the *product* of qubit count and T-depth, which is the trade-off measure that paper uses throughout. The columns above are plain counts, which is why they are not directly comparable — a construction that spends qubits to buy depth looks better under a product metric than under either factor alone. The direction agrees; the magnitudes should not be read against each other.

Fan the swap control out. Each round swaps (u, v) and (r, s) under the decision bit a, twice — $6n$ controlled swaps hanging off a single qubit, which must therefore run one after another. `kaliski_round_opt` copies a into k clean ancillas (`fanout`, default $k = 4$) and controls swap i on copy i mod k, so k swaps can run in the same layer; the copies are cleared with the same CNOTs at the end of the round. The Toffoli count does not move, since $2k$ CNOTs are free in this accounting, and the width grows by $k-1$ — the 84 qubits in the $n = 5$ row below would be 81 at $k = 1$. What the fan-out does *not* reliably buy at these sizes is depth. Within one round it helps (281 against 288 layers at $n = 7$, $k = 4$ against $k = 1$). Across the whole inversion, though, the k ancillas are freed last and recycled as the next round's comparator scratch, so the next round queues behind the un-fan-out and the total grows: 5 342 layers at $k = 4$ against 4 793 at $k = 1$ and 4 708 at $k = 2$, for $n = 7$. A doubling-tree fan-out (depth $\lceil \log_2(k+1) \rceil$) changes this by under 1%, so the cost is the allocation order rather than the copying. The table keeps $k = 4$ to stay comparable with [31], whose fan-out draws on the following multiplier's idle scratch rather than fresh qubits.

The $2n$-bit budget is not a comfortable margin, it is the exact requirement — r reaches 6 bits for $p = 7$ and 32 bits for $p = 65521$, both exactly $2n$. The test measures it rather than assuming it.

36.6 Level 3, rebuilt: the Euclidean dialog

Everything so far treats inversion and multiplication as two jobs. Schrottenloher [32], following [37], observes that they are one, and this is the single largest structural saving in this part.

Run the binary GCD on (u, v) alone and *do not* maintain the Bézout coefficients. Just record the two decision bits per iteration — call the record a *dialog*. It is 1.5 bits per iteration on average, against the $2n$ qubits carrying (r, s) would cost:

repeat $\approx 1.413n$ times:
1. $b_0 := v \bmod 2$; $\quad b_1 := [\,u > v\,]$ — record b_0 and $b_0 b_1$
2. **if** $b_0 b_1$: swap (u, v)
3. **if** b_0: $v \mathrel{-}= u$
4. $v \mathrel{/}= 2$

The iteration count is not arbitrary. Halving v always removes a bit; the subtraction removes a second one half the time. [32] averages the shrink factor of uv over those two cases, $\frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{4} = \frac{3}{8}$, and reads off $2n/\log_2(8/3) \approx 1.413n$ iterations. That step is heuristic — it averages the factor rather than its logarithm, which is not the same thing — but the conclusion holds: measured on this implementation's own dialog, the mean iteration count is $1.383n$ at $n = 32$ and $1.409n$ at $n = 256$, climbing towards $1.413n$. The spread is $\approx 0.6\sqrt{n}$, which is what the $+c\sqrt{n}$ safety margin is for.

```
def dialog_round(m, u, v, rec):
    """One iteration of [1128] Algorithm 2.  `rec` = (b0, b0b1), kept.
    ..."""
    ctx, n = m.ctx, len(u)
    b0, b0b1 = rec
    ctx.cx(v[0], b0[0])                                  # b0 = v mod 2

    t, sc = m.anc(1, "t"), m.anc(n, "sc")
    A.gt_uint(ctx, u, v, t[0], sc)                       # t = [u > v]
    ctx.and_(b0[0], t[0], b0b1[0])                       # b0b1 = b0 AND b1
    A.gt_uint(ctx, u, v, t[0], sc)
    m.free(t, sc)

    for i in range(n):
        ctx.cswap(b0b1[0], u[i], v[i])                   # if b0b1: swap u, v

    cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
    A.csub(ctx, b0[0], u, v, cp, sc)                     # if b0: v -= u
    m.free(cp, sc)

    for i in range(n - 1):                               # v /= 2 (v is even now)
        ctx.swap(v[i], v[i + 1])
```

Now the second idea. The updates to (r, s) that the full extended algorithm would have performed are all *linear* and depend only on those two bits, so they can be replayed afterwards, onto any starting pair:

$$s \leftarrow 2s \bmod p; \qquad \text{if } b_0: \ s \mathrel{+}= r; \qquad \text{if } b_0 b_1: \ \operatorname{swap}(r, s)$$

Read *forwards*, alongside the Euclidean algorithm as the textbook extended version would, this maps $(1, 0) \mapsto (0,\, x^{-1})$ — an inversion — and more generally $(y, 0) \mapsto (0,\, yx^{-1})$, which is an inversion and a multiplication for the price of one pass. That is [32]'s observation, and it is already the saving.

But the circuit reads the record *backwards*, and gets a different map: $(y, 0) \mapsto (0,\, yx)$. Multiplication by x, not by x^{-1}. That is not a defect but the reason the backwards reading is chosen: replayed in reverse the updates need only *doubling* and addition, where the forward reading needs modular *halving*. And since the whole thing is a circuit, running it in the other direction recovers the division. One component, both of the point addition's multiplicative steps.

The replay is seven lines, and the whole saving is that none of them looks at x:

```
def bezout_replay(m, r, s, recs, q, dbl=None, cadd=None, compressed=False):
    """(r, s) updated by [1128] Algorithm 3, reading the record in reverse.
    ..."""
```

```
    if dbl is None:
        state = {"s": s}

        def do_dbl():
            state["s"] = MA.moddbl_rotate(m, state["s"], q)
    else:
        state = {"s": s}

        def do_dbl():
            dbl(m, state["s"])
    cadd = cadd or (lambda mm, c, a, b: MA.cmodadd(mm, c, a, b, q))

    def replay_round(b0, b0b1):
        do_dbl()                                        # s <- 2s mod q
        s = state["s"]
        cadd(m, b0[0], r, s)                            # if b0: s += r mod q
        for i in range(len(r)):
            m.ctx.cswap(b0b1[0], r[i], s[i])            # if b0b1: swap r, s

    if compressed:
        full = len(recs) - len(recs) % 3
        for i in reversed(range(full, len(recs))):
            replay_round(*recs[i])
        for start in reversed(range(0, full, 3)):
            group_recs = recs[start:start + 3]
            group = _record_wires(group_recs)
            m.emit_inverse(compress_triple_circuit, m, group)
            for rec in reversed(group_recs):
                replay_round(*rec)
            compress_triple_circuit(m, group)
    else:
        for rec in reversed(recs):
            replay_round(*rec)
    return state["s"]
```

Assembling the three phases — dialog, replay, dialog backwards — gives the in-place multiplication, and the rebuild consumes the record on its way out, so nothing is left behind:

```
def inplace_mul(m, x, y, q, iters=None, dbl=None, cadd=None,
                register_sharing=False, c_pad=2.3, c_iter=2.4, compress=False):
    """|x, y> -> |x, y*x mod q>.  [1128] Algorithm 4.

    ..."""
    n = len(x)
    iters = iters or eea_iterations(n, c_iter)
    if register_sharing:
        return _inplace_mul_register_sharing(
            m, x, y, q, iters, dbl, cadd, c_pad, c_iter)
    if not compress:
        return _inplace_mul_plain(m, x, y, q, iters, dbl, cadd)

    recs, u, v = dialog(m, x, q, iters, compress=True)   # 1: x -> dialog
    clear_dialog_end(m, u, v)

    full = len(recs) - len(recs) % 3
```

```
    spares = [Reg([_record_wires(recs[i:i + 3])[5]])
              for i in range(0, full, 3)]
    m.reserve(*spares)
    s = m.anc(n, "bz")                                  # 2: replay onto (y, 0)
    s = bezout_replay(m, y, s, recs, q, dbl, cadd, compressed=True)
    for a, b in zip(y, s):                              # r is 0, s is the answer
        m.ctx.swap(a, b)
    m.free(s)

    hi = m.anc(1, "vhi")                                # 3: dialog -> x
    u2 = m.anc(n + 1, "u")
    v2 = Reg(list(x) + list(hi), "v")
    m.ctx.x(u2[0])                                      # the dialog ends at u = 1
    for i in reversed(range(full, len(recs))):
        m.emit_inverse(dialog_round, m, u2, v2, recs[i])
        m.free(*recs[i])
    for start in reversed(range(0, full, 3)):
        group_recs = recs[start:start + 3]
        group = _record_wires(group_recs)
        m.emit_inverse(compress_triple_circuit, m, group)
        for rec in reversed(group_recs):
            m.emit_inverse(dialog_round, m, u2, v2, rec)
        m.free(Reg(group))
    A.encode_const(m.ctx, u2, q)                        # and starts at u = q
    m.free(u2, hi)
```

Direction matters, and it is easy to get wrong

Applying Algorithm 3's update rules in the record's original order is *not* the inverse of applying them in reverse, and it is not the transpose either — it is simply a different linear map. Writing "$(1, 0)$ gives x^{-1}" next to "$(y, 0)$ gives yx" is self-contradictory, since the second is linear in y and contains the first at $y = 1$; an earlier draft of this section said exactly that, and the code is what caught it.

The classical model in `ec_classical.py` carries all three maps and the tests pin down which is which: `bezout_replay` (reversed) multiplies by x, `bezout_replay_inv` divides by it, and `bezout_inverse_classical` — an independent coefficient-tracking implementation that needs modular halving where the replay needs only doubling — yields x^{-1} directly. The third exists purely as a cross-check, which is why the direction claim is not taken on faith.

Compared against what it replaces — Kaliski inversion plus an out-of-place multiplication, which is what a point addition otherwise needs:

	invert-then-multiply		dialog + replay		saving	
p	Toffoli	qubits	Toffoli	qubits	Toffoli	qubits
7	1 095	64	612	39	44%	39%
11	1 910	83	957	48	50%	42%
31	2 889	99	1 378	57	52%	42%
61	4 068	115	1 875	66	54%	43%

At $p = 61$ that is 54% of the gates and 43% of the qubits, and the margin widens with n. The record itself is squeezed further by a trick worth its own paragraph: each pair $(b_0, b_0 b_1)$ can only be $(0, 0)$,

$(1,0)$ or $(1,1)$ — never $(0,1)$, since $b_0 b_1$ cannot be set unless b_0 is — so three pairs carry $3^3 = 27 < 32$ states and fit in five bits instead of six. One qubit freed per three iterations, taking the record from $2.83n$ to $2.355n$ asymptotically, and (unlike a shift-based encoding) leaving it a *fixed* size, which is what lets the rest of the circuit be laid out statically.

That packing is implemented (`compress_triple_circuit`, with its inverse used in the replay) but it is *standalone and opt-in*: `inplace_mul(..., compress=True)`. The multiplier measured in the table above, and used everywhere else in this part, is the unpacked one. At these sizes packing saves no peak width — the peak sits in the replay's scratch, not in the record — and the encode/decode network costs Toffolis: 672 against 612 at $p = 7$, and 1 975 against 1 875 at $p = 61$, at the same 39 and 66 qubits. The packing pays only where the record is the widest live thing, which is the cryptographic regime of §37.1.

36.7 Level 4, rebuilt: projective coordinates

The other way to make an inversion cheap is not to do it. In Jacobian coordinates $(X : Y : Z) \sim (x, y) = (X/Z^2, Y/Z^3)$ the addition law is inversion-free, and with the addend in affine form ($Z_2 = 1$, which it is, being a classical point) every term involving Z_2 drops out:

$$U_2 = x_2 Z_1^2, \qquad S_2 = y_2 Z_1^3, \qquad H = U_2 - X_1, \quad R = S_2 - Y_1,$$
$$X_3 = R^2 - H^3 - 2X_1 H^2, \quad Y_3 = R(X_1 H^2 - X_3) - Y_1 H^3, \quad Z_3 = Z_1 H.$$

Eleven multiplications, no division — [31] reports this as the fewest of any quantum projective addition, against a previous best of twelve.

The catch is that projective coordinates are not a unique representation. $(P + Q) - Q$ returns a *different triple* for the same point, so the input cannot be erased and every addition leaves its accumulator behind. The fifteen intermediates *can* all be cleaned — each by reversing the step that made it, with that step's inputs still live — and this implementation cleans them, but the input point is $3n$ qubits of irreducible garbage per addition.

The eighteen steps, and then the unwinding — note the last two lines, which are the whole subtlety:

```
def jacobian_add(m, X1, Y1, Z1, x2, y2, X3, Y3, Z3, p):
    """(X3:Y3:Z3) <- (X1:Y1:Z1) + (x2, y2).  [106] Algorithm 4.
    ..."""
    n = len(X1)
    A = m.anc

    ...

    Z1sq = A(n, "Z1sq"); _, b1 = m.step(sqr, Z1, Z1sq)                    # 1
    U2 = A(n, "U2");     _, b2 = m.step(MU.modmul_const, m, Z1sq, x2, U2, p)   # 2
    Z1cu = A(n, "Z1cu"); _, b3 = m.step(mul, Z1sq, Z1, Z1cu)              # 3
    H = A(n, "H");       _, b4 = m.step(sub_into, U2, X1, H)              # 4
    ...
    def mk2V():
        for q, r in zip(V, twoV):
            m.ctx.cx(q, r)
        MA.modadd(m, V, twoV, p)
    _, b11 = m.step(mk2V)                                                 # 11

    T0 = A(n, "T0");     _, b12 = m.step(sub_into, Rsq, Hcu, T0)          # 12
    _, b13 = m.step(sub_into, T0, twoV, X3)                       # 13  OUTPUT
```

```
    T1 = A(n, "T1");      _, b14 = m.step(sub_into, V, X3, T1)              # 14
    T2 = A(n, "T2");      _, b15 = m.step(mul, R, T1, T2)                   # 15
    T3 = A(n, "T3");      _, b16 = m.step(mul, Y1, Hcu, T3)                 # 16
    _, b17 = m.step(sub_into, T2, T3, Y3)                            # 17  OUTPUT
    _, b18 = m.step(mul, Z1, H, Z3)                                  # 18  OUTPUT

    # unwind everything but the three outputs.  Note T0 goes back via step 12,
    # not step 13 -- step 13 wrote X3, which stays.
    for body in (b16, b15, b14, b12, b11, b10, b9, b8, b7, b6, b5, b4, b3, b2, b1):
        m.undo(body)
    m.free(T3, T2, T1, T0, twoV, V, Rsq, R, S2, Hcu, Hsq, H, Z1cu, U2, Z1sq)
```

The order of unwinding is not simply "backwards"

X_3 is an output and must survive, so the intermediate T_0 cannot be released by reversing the step that consumed it. It is released by reversing the step that *produced* it, from R^2 and H^3, which are still live at that moment. Getting this wrong destroys the answer; `Machine.undo` exists precisely so that a recorded step can be reversed out of order, and the tests sweep the projective representatives Z of every input point — $Z \in \{1, 2\}$ by default and all of $[1, p)$ under `SHOR_EC_FULL=1` — a sweep that catches representation bugs an affine-only test cannot see.

36.8 Level 5: windowing, zig-zag, and the semiclassical transform

Three counting arguments, none of which changes a single arithmetic circuit.

Windowing [**?**] groups w control qubits into one address and looks up P_i from a table of 2^w precomputed points, so $2n + 2$ additions become $2\lceil (n+1)/w \rceil$. A lookup costs 2^w Toffolis, an order of magnitude below an addition, so w is chosen large — both [32] and [36] use $w = 16$.

Zig-zag [47] attacks the projective garbage. Write ρ for the number of accumulator registers — not m, which above is the counting-register width. Additions are arranged in runs of $\rho, \rho - 1, \ldots, 1$: fill all ρ registers, then walk back un-computing all but the last of the run — legal, because each state's predecessor is still live — returning $\rho - 1$ registers, and repeat. Total forward additions $\rho(\rho + 1)/2$, so ρ registers serve them all.

The semiclassical transform [**?**] is the same trick as §21: every controlled rotation in the inverse QFT is controlled by a qubit about to be measured, so measure it first and make the control classical. Both scalar registers collapse to *one recycled qubit*. This is the assumption behind both papers reporting only the cost of the point arithmetic.

The windowed addition itself, with the three lookups and the two in-place multiplications that replace §35's two divisions. The implementation also selects the Euclidean dialog and square-subtract backends before building this circuit:

```
def windowed_point_add(m, addr, x2, y2, points, p, iters=None, gcd="dialog",
                       square="reference"):
    """[1128] Algorithm 1.  `points[i]` is the window entry for address i.
    ..."""
    ctx, n = m.ctx, len(x2)
    w = len(addr)
    if gcd == "dialog":
        div = lambda: E.inplace_div(m, x2, y2, p, iters)
        mul = lambda: E.inplace_mul(m, x2, y2, p, iters)
    elif gcd == "pingpong":
        import ec_pingpong as PP
```

```
        div = lambda: PP.pp_div(m, x2, y2, p, iters)
        mul = lambda: PP.pp_mul(m, x2, y2, p, iters)
    elif gcd in ("jump2", "jump2-codec"):
        import ec_jump2 as J2
        codec = gcd == "jump2-codec"
        div = lambda: J2.j2_div(m, x2, y2, p, iters, codec)
        mul = lambda: J2.j2_mul(m, x2, y2, p, iters, codec)
    else:
        raise ValueError(f"unknown gcd backend {gcd!r}")
    if square == "reference":
        csq = _csub_square
    elif square in ("symmetric", "karatsuba"):
        import ec_karatsuba as K
        csq = (K.csub_square_symmetric if square == "symmetric"
               else K.csub_square_karatsuba)
    else:
        raise ValueError(f"unknown square backend {square!r}")
    assert len(points) == 1 << w, "the window must be full"
    assert points[0].inf, "entry 0 must be the point at infinity"

    ...

    one = m.anc(1, "one")
    ctx.x(one[0])
    c = m.anc(1, "c")
    MA.is_nonzero(m, addr, c[0])                        # 1

    x1, y1 = m.anc(n, "x1"), m.anc(n, "y1")
    _lookup(m, one[0], addr, x1, xs)                    # 2
    _lookup(m, one[0], addr, y1, ys)
    MA.modsub(m, x1, x2, p)                             # 3
    MA.modsub(m, y1, y2, p)                             # 4
    _lookup(m, one[0], addr, x1, xs)                    # 5
    _lookup(m, one[0], addr, y1, ys)

    div()                                               # 6  y2 <- y2 / x2

    xx = m.anc(n, "3x")
    _lookup(m, one[0], addr, xx, x3s)                   # 7
    MA.modadd(m, xx, x2, p)                             # 8
    _lookup(m, one[0], addr, xx, x3s)                   # 9
    m.free(xx)

    csq(m, c[0], y2, x2, p)                             # 10 if c: x2 -= y2^2
    mul()                                               # 11 y2 <- y2 * x2
    MA.cmodneg(m, c[0], x2, p)                          # 12 if c: x2 <- -x2
    ...
```

	point additions			zig-zag registers			garbage qubits	
n	none	w=11	w=16	none	w=11	w=16	none	w=16
192	386	36	26	28	8	7	16 128	4 032
256	514	48	34	32	10	8	24 576	6 144
384	770	70	50	39	12	10	44 928	11 520
521	1 044	96	66	46	14	11	71 898	17 193

At $n = 256$: 514 additions become 34, a 15× reduction, and the registers holding projective garbage fall from 32 to 8. The two compose — windowing cuts how many additions there are, zig-zag cuts how many of their outputs are live at once — and together they take the garbage from 24 576 qubits to 6 144.

36.9 Implemented ECDSA.Fail backends and a measured toy frontier

The windowed addition accepts four interchangeable in-place multiplication/division backends: the original Euclidean dialog (`dialog`), Jump-2 macro-steps (`jump2`), Jump-2 with base-5 transcript encoding (`jump2-codec`), and comparison-free ping-pong dialog (`pingpong`) [33]. Jump-2 groups binary-GCD updates without changing the algebra; its base-5 codec stores three five-symbol decisions in seven qubits instead of nine. Ping-pong is an *alternative* walk, not a layer stacked on Jump-2. The controlled square-subtract at step 10 selects the generic multiplication, a symmetric square, or a one-level Karatsuba decomposition (`reference`, `symmetric`, `karatsuba`). Symmetry avoids computing both off-diagonal products; Karatsuba computes two half-squares and one sum-square. At small field widths ancilla and uncomputation costs can outweigh the saved products.

The table compares *full* arithmetic-oracle ECDLP circuits, including Hadamards, windowed point additions, inverse QFTs and measurements, on the same $p = 7$ curve, order-five subgroup, $w = 2$ and three control bits per scalar. Each row recovers the discrete log on Qiskit Aer and its histogram passes the exact-distribution total-variation-distance check with 4 000 shots. Resources count the circuit before Aer transpilation, using the paper's Toffoli convention; depth treats each inverse QFT as a single logical instruction, so it is a circuit-level comparison rather than a routed hardware depth. "Frontier" means no tested row is no worse in *all three* of width, Toffolis and depth and strictly better in one:

dialog + square	qubits	Toffoli	depth	frontier
dialog+reference	53	6 008	27 433	no
dialog+symmetric	53	5 964	27 785	no
dialog+karatsuba	57	6 748	32 961	no
pingpong+reference	48	2 816	15 833	yes
pingpong+symmetric	50	2 772	16 149	yes
pingpong+karatsuba	57	3 556	21 309	no
jump2+reference	45	2 824	12 311	yes
jump2+symmetric	50	2 780	12 663	yes
jump2+karatsuba	57	3 564	17 859	no
jump2-codec+reference	45	2 824	12 311	yes
jump2-codec+symmetric	50	2 780	12 663	yes
jump2-codec+karatsuba	57	3 564	17 859	no

The toy codec rows tie the corresponding unencoded Jump-2 rows: at this width, three symbols do not amortize a seven-wire packing circuit, so identical resource triples appear twice on the frontier. Neither this tie nor a win at $n = 3$ predicts cryptographic-size ordering.

The data and the measured Aer device/method are saved by `bench/ec_pareto.py` in `bench/ec_pareto.json`; the notebook draws the same three-axis frontier. These toy-circuit numbers are

not the cryptographic resource estimates of either paper. Fig. 1 record packing and probabilistic register sharing remain separate opt-in techniques, not part of the measured windowed circuits. The full-circuit reference-affine oracle below is likewise *not* a point on this windowed frontier. Gu et al. [34] propose a different, reversible divsteps prime-field inversion architecture for affine point addition. The `divsteps` selector now implements its exact signed (z, f, g) recurrence and coefficient updates on toy primes up to 127, with a fixed-length schedule chosen by enumerating toy-field units. Branch decisions are retained on qubits; the modular result is halved once for each recorded active step to remove the 2^t factor, then the gate walk is reversed to clean the history and arithmetic ancillas. Exact low-space CDKM arithmetic and temporary reuse of the already-zero terminal remainder reduce the toy peak without claiming Gu's two-register packed layout. Exhaustive toy tests check Appendix A's integer determinant, scaling, branch decoder and reverse recurrence; a separate binary-long-division Euclidean reference checks SI II.2's quotient arithmetic. The latter is *not* a packed-Euclidean quantum backend. `divsteps-decoded` reuses one pair of decision qubits across all rounds: each round erases its decisions from the updated quantum state and recovers them before that round is reversed. It retains one active/frozen bit per round. Because the circuit stores coefficient *residues*, not the signed integer coefficients assumed in Appendix A, its decoder is an exact round-specific finite-state lookup compiled from every unit of each toy field and the disabled control state, rather than the paper's scalable valuation-window arithmetic. Conflicting lookup keys are rejected at construction time; full-history `divsteps` remains available. `divsteps-arithmetic` instead preserves signed integer coefficients in two's-complement registers and reconstructs decisions *arithmetically*. The updated coefficient detects a swap via $f'_v \equiv 2 \pmod 4$; after a negative/zero-counter nonswap, valuation $j = v_2(f'_v)$ selects the bit $\mathrm{bit}_{j-1}(g'_v(f'_v/2^j)^{-1} \bmod 2^j)$ from Appendix A, Eq. (A11). Reversible truncated integer products include carries when computing this odd inverse; the computation and its work qubits are uncomputed after toggling the branch bit. This is a circuit over signed registers, *not* a lookup of toy states. The toy-only maximum step count and signed width are exhaustively bounded by the chosen field. The valuation window is fitted *per round*: round r only tests the depths j that some toy input actually reaches there after an ambiguous nonswap. For $p = 5, 7, 11$, these are $(1, 1, 2, 2, 1)$, $(1, 1, 2, 2, 1, 1)$ and $(1, 1, 2, 2, 3, 3, 2, 1)$, where 1 means no Eq. (A11) window. The same exhaustive inverse checks pass. This cuts the $p = 5$ ECDLP oracle from 37,912 to 32,248 Toffolis. In the final round of an exact arithmetic schedule, the shared swap/parity bits can remain set across inverse copy-out: no intervening operation reads them, and the symmetric decoder toggles would cancel. Omitting both toggles reduces the $p = 5$ oracle further to 31,736 Toffolis and depth 84,237, without changing its 63-qubit width or its sampled TVD 0.032. A controlled superposition of all four nonzero inputs returns exactly to the initial state on Aer CPU; all 16 full-oracle basis inputs and the complete inverse-QFT ECDLP also pass. This skip is restricted to the exact arithmetic schedule, not the bounded-round or lookup-decoded schedules. The active/frozen bit remains stored once per round. The signed coefficient is reduced modulo p only when copying out the inverse, before the same 2^{-t} correction and reverse cleanup. Thus this implements the arithmetic decoder, but does not reproduce the paper's packed layout, measurement-based erasure or 256-bit resource schedule. `packed-euclid` is a separate, genuinely packed Euclidean inversion: Luo et al.'s Algorithms 1–3 [35], which realise Gu's SI II.2 walk. Two $(n+3)$-qubit words, Work1 and Work2, hold the remainders, the partial quotient and the Bézout coefficients from opposite ends. Coherent length counters $\ell_t, \ell_q, \ell_{r'}$ and a shift counter ℓ_s locate the boundaries. Each of the $4\lceil 1.5795\, n \rceil$ reversible steps does four things: it moves the shift boundary; it subtracts or adds the shifted divisor in the remainder interval and records the quotient bit; it applies the matching coefficient update to the prefix; and, when a quotient finishes, it swaps the words and rewrites the counters from the highest-one and right-length scans. The inverse is copied out with the parity-dependent sign, and the whole walk is uncomputed. A bit-position classical model reproduces Luo's Table 4 ($13^{-1} \bmod 37$). It matches Luo's integer recurrence

and gives exact inverses for every odd prime below 128. The gate-level step equals the model on every reachable state for $p \leq 13$. Controlled inversion returns every ancilla to zero for every input. Counter widths are toy sized, justified by these exhaustive checks rather than by a proof. Gates are plain Toffolis with a borrowed-ancilla multi-control ladder, so the Toffoli count below is an unoptimised toy circuit, *not* the paper's resource estimate. On the $p = 7$ curve with 2 control bits per scalar, the 57-qubit packed-euclid ECDLP circuit recovers $k = 4$ (TVD 0.017 against 0.041). An optional `ec_divsteps.mod_inv(round_limit=, failure=)` limits the signed arithmetic walk while retaining coherent, clean failure signaling. It emits a zero inverse and a separate failure flag for an unfinished step or an invalid field input (including zero and out-of-range values); the disabled control branch is still the identity. The schedule is *not* selectable in the affine point adder, which would need an explicit failure contract for both divisions. Exhaustive toy-unit traces in `bench/ec_gu_schedule.json` count 3/4, 2/6, and 2/10 failures at one step short of the exact $p = 5, 7, 11$ schedules, respectively; the exact ceilings have zero toy-unit failures. These are *controlled-inversion component* measurements and are excluded from the complete-oracle Pareto frontier. A bounded affine adder is blocked at toy scale. `bench/ec_gu_bounded_adder.json` traces both divisions of each controlled addition on the actual task. It models a clean contract in which a failed addition is skipped and its flag is kept and measured. At one step below exact, 32/32 ($p = 5$) and 20/32 ($p = 7$) task additions fail. The resulting distributions are 0.477 and 0.438 in TVD from the exact ones, against 4096-shot bounds of 0.040 and 0.041. At the exact limit, it only adds failure logic to the default. The flag cannot be cleaned in place. The second division uncomputes λ, so the flag must cover both divisors before P is overwritten. After a successful addition, recomputing it would need the opposite, negated divisors, which have different step counts. Truncated carries and test-free quotient positions are therefore not used.

The following *separate affine frontier* holds the complete ECDLP task fixed at $p = 5$, order 3, $k = 2$, and two control bits per scalar. All six affine oracles pass every one of their 16 arithmetic basis-state checks. On Aer CPU (limit 63 qubits), the full-history divsteps circuit's 1 024-shot full superposition/QFT/measurement run on 60 qubits recovers $k = 2$ and has TVD 0.032 against an exact-distribution shot-noise bound 0.078. Separately, on the $p = 7$, order-five toy curve with two control bits per scalar, the 63-qubit complete affine divsteps ECDLP circuit recovers $k = 4$ over 4 096 Aer shots (TVD 0.017 against 0.041). This is a second end-to-end validation, *not* another point on the $p = 5$ frontier. The decoded circuit also recovers $k = 4$ on the $p = 7$ curve with 3 control bits per scalar and 55 qubits (TVD 0.039 against 0.054); the different control count keeps this validation out of the p=5 resource comparison. The `kaliski` and `kaliski-opt` rows have *not* been sampled on Aer for this task. The last column reports three-metric resource nondominance, not end-to-end Aer certification for the unsampled rows:

affine inverse	qubits	Toffoli	depth	Aer sampling	frontier
kaliski	67	10 976	46 911	not sampled	yes
kaliski-opt	71	13 336	51 421	not sampled	no
divsteps	60	26 872	75 109	TVD 0.032	yes
divsteps-decoded	52	27 064	75 505	TVD 0.032	yes
divsteps-arithmetic	63	31 736	84 237	TVD 0.032	no
packed-euclid	57	3 522 984	3 524 181	TVD 0.032	no

The p=5 affine task is not comparable with the p=7 windowed frontier above. The arithmetic decoder's extra signed registers and generic reversible integer products cost more on this toy task than the round-specific lookup; an exact arithmetic decoder alone is *not* the paper's complete optimised divsteps implementation. The packed-euclid row needs fewer qubits than the full-history and arithmetic divsteps rows, but about a hundred times more Toffolis, because of its generic coherent addressing. Fused coordinate arithmetic, Gu's optimised packed gate schedule and the bounded-

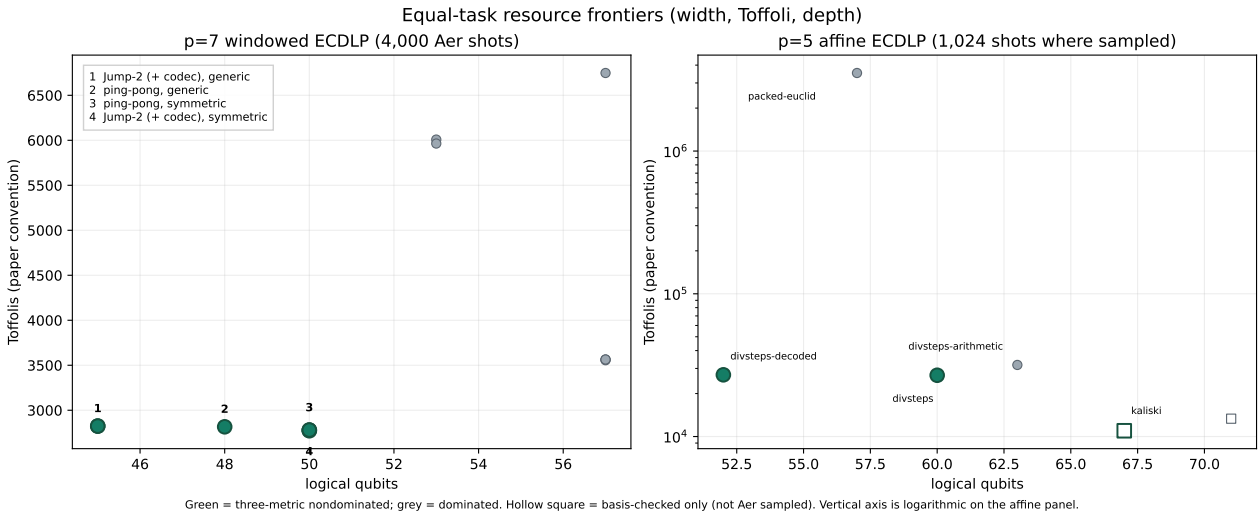


Figure 1: Two *separate*, equal-task three-axis resource frontiers from `bench/ec_pareto.json` and `bench/ec_gu_affine.json` (regenerate with `bench/plot_optimisation_frontiers.py`). Horizontal position is logical width, vertical position is the pre-synthesis Toffoli count, and frontier membership additionally requires nondominance in logical depth. Green points are resource-nondominated; grey points are dominated. Tied Jump-2/codec points overlay each other. The hollow Kaliski square is basis-checked but too wide for Aer sampling on this host; a resource-frontier point is *not* automatically Aer-certified. The affine panel uses a logarithmic Toffoli axis to retain the unoptimised packed-Euclidean walk. Different tasks and component-only bounded schedules are never compared as frontier points.

error 256-bit schedule remain unimplemented; no cryptographic-size resource claim follows. The $p = 7$, two-bit signed-decoder complete oracle requires 64 qubits, above this Aer CPU installation's 63-qubit limit; its inverse, division and in-place point addition have basis-state validation but are not described as an Aer ECDLP result. The Aer measurements above ran on CPU/MPS; CPU Aer suffices for the end-to-end toy validation reported here. Likewise, ECDSA.Fail's lane-lifetime refinements, general constant propagation and post-generation dead-code elimination are not represented by a selectable, end-to-end measured point-adder backend here. The separate probabilistic register-sharing experiment does not imply those refinements.

37 Everything at once

The subsections above each hold one thing fixed. This one does not: it stacks them, on the operation that actually matters — one point addition — and then on the whole circuit.

Two different kinds of number appear below, and they should not be read against each other. The point-addition table and the keybox are *component comparisons*: each construction is built and measured on its own, with the arithmetic options that construction names. The whole-circuit table at the end is a single *reference* build: `ecdlp_circuit(..., oracle="arith")` with its defaults, which are the affine addition of Alg. 3 with the reference multiplier and the [30] Kaliski inversion — no windowing, no dialog, no projective coordinates, none of the Level 1–4 optimisations. It is there to show the size of a complete, simulated, correct circuit, not the best one the components above could assemble. (`ec_shor` does accept the optimised backends, `arithmetic="windowed"`, `multiplier=`, `inversion=`, and the test suite runs them end to end; they are not what that row measures.)

curve	construction	Toffoli	qubits	depth	vs. affine
Alg. 3, Alg. 4 are [31]'s; Alg. 1 is [32]'s.					
$p = 7$	affine, in place (Alg. 3)	2 756	64	11 886	—
	Jacobian, out of place (Alg. 4)	1 544	75	8 511	44%
$p = 11$	affine, in place (Alg. 3)	4 769	83	21 023	—
	Jacobian, out of place (Alg. 4)	2 642	98	15 126	45%
$p = 7$	windowed w=2, dialog mult. (Alg. 1)	1 523	49	6 907	45%

Three constructions of the same map, and they stack differently. The affine addition is in place and leaves nothing behind, and pays for that with an inversion. The Jacobian addition removes the inversion entirely and is 45% cheaper, but leaves $3n$ qubits of input point as garbage — which is only affordable once windowing and zig-zag have cut how many such inputs are live at once. The windowed addition is cheapest of the three at 45% below affine, and it gets there differently again: not by changing coordinates, but by replacing invert-then-multiply with a single Euclidean dialog, and by amortising three table lookups over a whole window of additions.

How the savings compose — and where they do not

Read down the stack for one point addition at $n = 256$, taking each factor from the measurements above:

- *Gate-level, and only partly independent.* The Gidney adder ($\approx 2\times$ on the inner additions) and the temporary AND ($2.50\times$ on T) are *not* two savings: they are one saving counted at two levels. The adder wins by choosing a gate whose target starts clean, and the temporary-AND accounting is exactly what makes that gate's uncompute free. Multiplying the two would double-count. Copy-then-add controls are a smaller further factor than the table suggests: the $2.0\times$ there bundles the control style together with the switch to Gidney's adder inside it. Hold the adder fixed and the control style alone is worth $1.33\times$ (96 against 128 at $n = 32$, CDKM on both sides) — it buys parallelism and depth more than gates. Pseudo-Mersenne reduction ($6.2\times$ on doublings) is the one that cleanly composes, because it acts on a different part of the circuit again: the reductions rather than the additions.
- *Structural, and they replace rather than compose.* The dialog (54% off the multiplicative step) and projective coordinates (45% off the addition) are two different answers to the same problem — the inversion — so you take one, not both. Every published construction picks a side.
- *Count-level, and these do multiply the whole thing.* Windowing divides the number of point additions by 15 at $n = 256$; zig-zag divides the live projective garbage by 4; the semiclassical transform deletes both counting registers. None of them touches an arithmetic circuit, which is precisely why they compose cleanly with everything above.

The last group dominates. An optimisation that halves a point addition is worth half as much as one that runs fifteen times fewer of them, which is why both papers spend their first section on windowing and their last on gate counts.

And the whole circuit — the reference-affine arithmetic oracle, not a stack of the optimisations above — on the toy curve $y^2 = x^3 + 5x + 4 \bmod 7$ with $\mathrm{ord}(G) = 5$:

ECDLP circuit	qubits	Toffoli	gates	depth
real arithmetic, two counting registers	69	16 488	105 339	70 967
permutation oracle, two counting registers	12	—	603	282
permutation oracle, semiclassical QFT	7	—	619	287

69 qubits and 105 339 gates to compute a discrete logarithm in a group of order five. That ratio is the honest summary of this part: the arithmetic is enormous, and the counting-level tricks are the only reason cryptographic sizes are discussed at all. Note the last two rows — the semiclassical transform takes 12 qubits to 7 by recycling one control qubit, which is precisely why both papers report only the point arithmetic and treat the scalar registers as free.

37.1 Space at cryptographic size

The one place where a projection is worth making, because it is what the papers' headline qubit counts are built from. The dialog record dominates the space of [32]'s construction. Every column here is the `ec_cost` closed form, not a circuit measurement, and the "Fig. 1 packing" column prices the standalone packing of §36.6, which the measured multiplier does not use:

n	record, raw	with Fig. 1 packing	per n	with register sharing
256	802	669	$2.613n$	579
384	1 180	984	$2.562n$	859
521	1 582	1 319	$2.532n$	1 157

The middle column converges to the paper's $2.355n$ from above — the gap is the $O(\sqrt{n})$ safety margin on the iteration count, which is a constant fraction at these sizes and vanishes asymptotically. Register sharing is available as an opt-in layout: it shrinks u and v and reuses their cleared wires. Its overflow probability depends on the padding and input distribution; the basis-state simulator detects overflow rather than truncating. The implementation keeps fixed five-bit record packing, so these measured circuit widths do not claim the paper's tighter $2.12n + O(\sqrt{n})$ projection.

The paper's own closed form for the total — its asymptotic $2.355n$ record plus $2n$ for the pair of modular integers the reconstruction holds, i.e. $4.355n + O(\sqrt{n})$ — gives 1151 qubits at $n = 256$ against the paper's 1192, and 1407 against 1446 for the gate-optimised variant. That is the paper's formula evaluated, *not* the table above summed: the table's middle column carries its $O(\sqrt{n})$ margin explicitly and would give a larger figure. Neither is a check on the other.

The full-algorithm count of $2^{26.11}$ Toffolis is likewise the paper's equation (1) evaluated at the paper's own per-addition figure — restated for scale, not independently confirmed. Nothing in this package measures a 256-bit point addition.

38 The 2026 refinements

Everything so far builds [31] and [32] as printed. Between those and the lowest published count — IonQ's 1.196×10^6 Toffolis per point addition plus three lookups [41] — lie three changes, and a fourth line of work (ECDSA.Fail [33]) reached comparable numbers by different means. Each is built here as a *switch* beside the construction it refines, never in its place: with the switch off, the circuit is the old one gate for gate, and a regression test pins every number above.

38.1 Uncomputing by measurement, and checking that it worked

A windowed addition loads its table point three times. Unloading a lookup by running it again costs as much as loading it; measuring the loaded register in the X basis costs nothing, but leaves the phase $(-1)^{m \cdot T[a]}$ on the address, which a *phase* lookup of $F[a] = m \cdot T[a]$ must repair — about $2\sqrt{L}$ ANDs with Gidney's power products, against L for the lookup. IonQ goes further: every unload of a window shares one address, so their tables are XORed and a single repair serves all of them. That is how the lookup cost of an addition falls from ten recomputed lookups to three loads and one repair: 1 310 700 Toffolis to 197 364 at $w = 16$.

Measurement is the one thing the simulator of §41 cannot see: it pushes basis states and skips phases. So the logical gates carry their measurement-based *price* and a unitary *definition* (the plain XOR), which keeps every exhaustive test of this part valid, and a separate sparse state-vector simulator executes the real thing — measure, build the repair from the actual outcome, apply it — under fixed outcome sequences. The uncompute is right if and only if every basis input comes back with the same phase; an unrepaired phase is exactly a difference between inputs. An approximate repair (IonQ's comparator on the top bits) fails that on precisely the inputs where its truncated comparison disagrees, which the tests confirm input by input.

38.2 The GCD, five ways

Two thirds of a point addition is the in-place multiplier, and most of that is the walk and replay of the binary GCD. IonQ's change is a change of *representation*: store $\tilde{v} = v + (1 - a)u$ with $a = v \bmod 2$. Both registers stay odd, so the next parity is free ($\tilde{v}[1] \oplus u[1]$, two CNOTs), and the update $\tilde{v} \leftarrow (\tilde{v} \pm u)/2$ is a *conditionally inverted* addition — one uncontrolled adder between two rows of CNOTs:

```
def ci_add(ctx, e, x, y, anc):
    for q in y:
        ctx.cx(e, q)
    add(ctx, x, y, anc)
    for q in y:
        ctx.cx(e, q)
```

$n - 1$ ANDs where [32] spends $2n$ on a controlled subtraction. The recorded bits turn out to be exactly the dialog's (checked on every x for three primes), so the replay and the packing of §36.6 carry over. ECDSA.Fail's "ping-pong" walk removes the comparison and the swap altogether by keeping both operands signed and odd and updating them alternately; its Jump-2 walk takes up to two halvings per step. Built at $n = 256$ with secp256k1's prime, each part counted and combined with the paper's round budget:

in-place multiplication, $n = 256$	rounds	walk/round	replay/step	Toffolis
dialog [32], exact, full width (before)	401	1 285	2 306	1 955 276
dialog, fused 77-bit compare, width schedule, PM	401	525	1 108	865 588
cond. inverted [41], dialog replay	401	378	1 108	747 194
cond. inverted [41], IonQ replay	401	378	1 041	720 565
ping-pong [33]	704	257	785	917 565
Jump-2 [33]	261	1 284	1 878	1 161 099

The ping-pong walk is the cheapest per round, but at $2.75n$ rounds its replay dominates; composed from this package's general modular cells it does not reach ECDSA.Fail's fused ones. Its round budget is also not a worst case: exhaustively, $n = 16$ needs $3.9n$.

38.3 Squaring

Step 10 subtracts λ^2. With the general multiplier that is two multiplications (compute and uncompute). IonQ's dedicated squarer writes $x^2 = -x + \sum_i (2x_i - 1)2^{2i+1}(h_i + x_i)$ with $h_i = \lfloor x/2^{i+1} \rfloor$: every term is an *unconditional* signed subtraction on a window of $n + 1 - i$ bits, for $\frac{1}{2}n(n+3)$ ANDs in all. A pseudo-Mersenne fold ($2^n \equiv f$) then reduces it into the accumulator: 1 050 623 Toffolis become 79 160 at $n = 256$.

```
def sqr_int(m, x, z):
    ctx, n = m.ctx, len(x)
    assert len(z) == 2 * n + 1
    pad = m.anc(1, "pad")

    # z <- -x as an (n+1)-bit two's-complement number
    zero = m.anc(1, "z0")
    anc = m.anc(n, "sq")
    A.sub(ctx, Reg(list(x) + list(zero)), Reg(z[:n + 1]), anc)
    m.free(anc, zero)
```

```
    for i in range(n):
        ctx.cx(z[n + i], z[n + i + 1])              # sign-extend by one bit
        h = list(x[i + 1:])
        for q in h + [pad[0]]:
            ctx.cx(x[i], q)                         # x_i ? NOT(0||h) : (0||h)
        operand = Reg(h + [pad[0], x[i]], "op")     # signed, sign = x_i
        window = Reg(z[2 * i + 1:n + i + 2], "win")
        assert len(operand) == len(window) == n + 1 - i
        anc = m.anc(max(len(window) - 1, 1), "sq")
        A.sub(ctx, operand, window, anc)            # z -= signed operand
        m.free(anc)
        for q in h + [pad[0]]:
            ctx.cx(x[i], q)
    m.free(pad)
```

38.4 One point addition, at $n = 256$

windowed point addition, $n = 256$, $w = 16$	before	after
two in-place multiplications	3 910 552	1 441 130
square-subtract	1 050 623	79 160
table lookups	1 310 700	197 364
five modular additions	5 115	5 115
total	6 276 990	1 722 769
published: [32] / [41]	2 588 962	1 392 608

A 3.6× reduction per addition, and with the first window replaced by a lookup and the last three dropped (recovered by a classical search, as [36] and [41] do), 28 additions instead of 34: 213 417 660 Toffolis for the whole algorithm become 48 237 532. The "after" column uses the approximate settings at the papers' parameters; their failure rates are measured at toy sizes, not at $n = 256$.

What reproducing the sources turned up

IonQ's replay needs its $0 \leftrightarrow p$ swap *before* the complemented adder: without it, every $e = 1$, $y = 0$ step leaves an ancilla dirty. ECDSA.Fail's ping-pong budget $L = 2.75n$ is not a worst case. And at toy sizes the round budgets distort any comparison of GCDs — which is why the table above is assembled at $n = 256$ rather than read off a toy circuit.

One more, found while building at $n = 64$: the GCD's top-bit comparisons act on registers the width schedule pads with $2.3\sqrt{n}$ bits of leading zeros, so a comparison on k top bits sees only $k - 2.3\sqrt{n}$ significant ones. IonQ's "$40 + 2.3\sqrt{n}$" comparison bits (77 at $n = 256$) is that correction. A flat 24 bits at $n = 64$ fails on 15 of 40 multiplications, and every $n = 256$ count here uses 77.

39 At cryptographic size

The tables of §38 are assembled: each part built at $n = 256$, then multiplied by a round count. This section drops the assembly. The builders that the tests check exhaustively at toy primes are run on secp256k1 itself with 16-bit windows, and every count below is read off the circuit they produce. Two further subsections then close, one at a time, the two gaps the first table shows: Toffolis against IonQ, and qubits against everyone.

39.1 Building a 256-bit circuit on a laptop

Built flat, as a list of gates, one 256-bit point addition is tens of millions of instructions, and the whole algorithm thirty times that. But it is made of few distinct pieces called many times on registers of the same shape: the replay alone calls one modular adder eight hundred times. `hier.tracing()` uses that without editing a builder. For the duration of a `with` block it wraps a list of existing builder functions in a decorator. A wrapped call computes a *shape* — register widths, which arguments share qubits, the classical parameters — builds the sub-circuit once per shape into a private machine, and appends it to its caller as one opaque gate carrying its counts (inverted, when the caller runs it backwards). A complete windowed point addition then builds in seconds. Two checks license the numbers: at toy size the hierarchical count equals the flat one on every field, for eight configurations; and the tree exports to Qualtran bloqs whose `QECGatesCost` gives the same Toffolis.

one windowed point addition, secp256k1, $w = 16$	Toffolis	qubits
Qualtran's `ECAdd` [44], for reference	8 346 972	1 798
dialog [32], exact arithmetic, MBU lookups, masked tables	5 163 617	2 104
+ 77-bit comparisons, width schedule, pseudo-Mersenne replay	2 972 327	2 251
+ dedicated squarer and fold	2 000 864	2 251
cond. inverted walk and replay [41], squarer	1 711 194	2 251
ping-pong walk, 704 rounds [33], squarer	2 117 534	2 155
dialog, register sharing and Fig. 1 packing, squarer	2 006 184	1 873

The whole algorithm builds too: a first-window lookup, then 28 windowed additions with the last three windows dropped, 47 978 966 Toffolis on 2 251 qubits with the control qubits recycled. On the same tree Qualtran counts 249 846 912 Cliffords and 29 873 480 measurements. Two gaps remain. The best row is 23% above IonQ's figure, and every row needs far more qubits than any paper reports.

39.2 Matching IonQ's Toffoli count

Splitting the IonQ-style addition by component locates the first gap:

one windowed point addition, $n = 256$	before	after
table lookups and their repair	197 364	197 364
GCD walks and their undoing	593 432	593 432
Bézout replays	833 312	564 086
of which signed additions	569 600	307 574
square-subtract	79 160	79 160
everything else	7 926	4 571
one windowed point addition	1 711 194	1 438 613

The walk is already IonQ's; the replay is not. Each of its eight hundred steps adds $\pm r$ into s modulo p, and the cell that did it — Algorithm 11 between two complements, with the $0 \leftrightarrow p$ swap and its undoing — costs 712 Toffolis, of which only 256 are the addition. The rest are top-bit tests: does the sum need reducing, is it exactly p, was the input 0. IonQ's modular adder (their Algorithm 2) decides the reduction with no test at all. It adds with carry-out, and the carry *is* the flag:

```
def modadd_pm_phase(m, x, y, q, kappa=None, delta=None):
    import ec_mbu as MB
    pm = pseudo_mersenne(q)
    assert pm, f"q = {q} is not pseudo-Mersenne"
```

```
    u, f = pm
    ctx, n = m.ctx, len(y)
    assert u == n
    kappa = min(n, kappa or phase_kappa(q))
    delta = min(n, delta or 32)

    ax, ay = m.anc(1, "ax"), m.anc(1, "ay")
    sc = m.anc(n, "sc")
    A.add(ctx, x + ax, y + ay, sc)                    # ay = carry-out
    m.free(sc)
    cp, sc2 = m.anc(kappa, "cp"), m.anc(kappa, "sc2")
    A.cadd_const(ctx, ay[0], Reg(list(y[:kappa])), f, cp, sc2)   # 2^u -> f
    m.free(cp, sc2)

    def recompute(qc, qs):
        from ec_gates import Ctx
        xx, yy, fl, anc = qs[:n], qs[n:2 * n], qs[2 * n], qs[2 * n + 1:]
        A.lt_uint(Ctx(qc, "and"), top_bits(yy, delta), top_bits(xx, delta), fl,
                  anc[:delta])

    def phase(qc, qs):
        from ec_gates import Ctx
        xx, yy, anc = qs[:n], qs[n:2 * n], qs[2 * n + 1:]
        out = anc[delta]
        qc.x(out)
        qc.h(out)                                     # |->: the CNOT kicks back
        A.lt_uint(Ctx(qc, "and"), top_bits(yy, delta), top_bits(xx, delta), out,
                  anc[:delta])
        qc.h(out)
        qc.x(out)

    MB.mbu_flag(m, ay[0], list(x) + list(y), recompute, phase, nanc=delta + 1)
    m.free(ax, ay)
```

If $x + y \geq 2^{256}$ the carry is set, and subtracting p is "drop the carry, add $f = 2^{32} + 977$", which touches only the low $\kappa = 65$ bits. What remains is to clear the carry, and the identity that makes this cheap is that after the correction it equals $[y' < x]$ for the *new* y': with a reduction $y' = x+y-p < x$, without one $y' = x + y \geq x$. So the carry is measured in the X basis, and only on outcome 1 — half the time — is the phase $(-1)^{[y'<x]}$ applied, by a comparator on the top $\delta = 32$ bits. The cell costs 352 Toffolis in the worst case and 336 counting only what executes, which is IonQ's convention. Its approximations are all of order f/p or $2^{-\delta}$: sums in $[p, 2^{256})$, which raise no carry and stay unreduced; a carry escaping the low κ bits; ties in the top δ bits. At toy primes the tests find it wrong on exactly the sums in $[p, 2^n)$ and nowhere else, and phase-correct everywhere else with every measurement actually performed.

One of those cases is not rare. The replay *must* pass through a zero: dividing, it starts from $(r, s) = (0, z)$, the first swap moves the 0 into s, and the next subtraction complements it — but $\overline{0} = 2^{256} - 1$ is not a residue. Multiplying, the same step runs backwards as an addition with $x+y = p$ exactly, the one sum the carry cannot see. Both happen at a step fixed by the 2-adic valuation of x, so within the first k steps with probability $1 - 2^{-k}$. IonQ's parameter table says "rounds with $x + y = p$: 37": the first 37 steps keep the careful cell and the rest use the new one. The tests measure the failure rate halving with each careful step and reaching zero at 37, in both directions.

One more detail is worth as much as the adder. The package built multiplication as division run

backwards: exact, but it throws the saving away, because the inverse of "measure, and sometimes repair" is "recompute", which always runs. So multiplication gets its own replay, each step inverted by hand (double, then subtract $\pm r$), and its uncomputes stay measurements. The replay's signed additions fall from 569 600 to 307 574 Toffolis per point addition, and each halving from 73 to 64 with the shorter correction.

one windowed point addition, $n = 256$	Toffolis	executed	qubits
cond. inverted walk and replay, Algorithm 11 cells (as above)	1 711 194	1 711 194	2 251
+ IonQ's adder in the replay, 37 careful steps	1 441 968	1 430 352	2 233
+ the same adder in the addition, approximate negation	1 438 613	1 426 965	2 233
+ Fig. 1 packing, replay in x's qubits	1 443 933	1 432 285	1 845
row 3 + signed windows (2^{15}-entry tables)	1 340 563	1 328 915	2 233
row 4 + lean careful cell, CNOT ends, shared walk, phase fold	1 436 519	1 424 871	1 457
the same with signed windows (§39.3)	1 338 469	1 326 821	1 457
+ SELECT-SWAP on the $3x$ lookup (§39.4)	1 322 341	1 310 693	1 457
published [41]	—	1 392 608	1 457

That is 1 438 613 Toffolis, or 1 426 965 counting what executes, within 2.5% of IonQ's 1 392 608. Built end to end on these cells, the 28-addition algorithm is 40 346 698 Toffolis (40 020 554 executed), against IonQ's $28 \times (1.196 \times 10^6 + 3 \cdot 2^{16}) \approx 39.0 \times 10^6$. The qubits are another matter: 2 233, or 1 845 with the record packed and the replay run in x's qubits, against IonQ's 1 457. The last two rows reach both at once (§39.3).

The last row goes past IonQ. Signed windows [30, 31] read the w address bits as an odd digit $d = 2i+1-2^w$. The lookup then runs over $|d|$ on $w-1$ bits, and the sign is two controlled negations of y, merged across consecutive windows. Each table has 2^{15} entries instead of 2^{16} and, the digits being odd, never contains O. Built at $n = 256$, one addition costs 1 340 563 Toffolis, 1 328 915 executed: 4.6% below IonQ's figure, and 2 745 400 fewer over the 28 additions. The whole-circuit count above still uses unsigned windows; §39.3 builds the whole circuit with signed ones.

Signed windows need every point the accumulator visits to have odd order, since the negation assumes $y \neq 0$. secp256k1's group has prime order, so this holds there. At toy size it holds only on curves of odd order, and the check there is the one that matters most: the whole output distribution. Run on every input of a curve of order 43, the signed circuit's exact frequency distribution is at total variation 0.370 from the ideal. That comes entirely from the 28% of inputs that meet an exceptional addition, which is unavoidable at this size. It finds the logarithm in one run with probability 43% (ideal 77%), above IonQ's bound of 10%.

39.3 IonQ's count and IonQ's qubits, in one circuit

The last two rows of the table are one circuit. It starts from row 4, 1 845 qubits, and applies the method of §39.5: find the step that sets the peak, take the scratch out of it, rebuild. On IonQ's cells the peak moved four times, and no fix added a Toffoli.

- *The careful cell* of the first 37 replay steps. Algorithm 11 allocates a copy register for a controlled addition even when the addition is uncontrolled, and keeps its adder's scratch to the end so that the top-bit comparisons can use a few of those qubits. Freeing the scratch as soon as the addition is done gives the same 703 Toffolis on 259 scratch qubits instead of 647 (`PMPhase(lean=True)`).
- *The replay's opening and closing additions.* Multiplying, the replay starts by setting $s = y$, an exact modular addition of r into a register that is 0. Dividing, it ends by clearing $\tilde{s}$, a

subtraction of r from a register that equals r. Both answers are copies, so both are n CNOTs (`cnot_ends`) instead of 1 023 Toffolis on 516 scratch qubits.

- *The square-subtract's fold.* It subtracts the square's low half and then f times its high half, bit by bit, while holding the $2n + 1$-qubit square: 8 subtractions, each the exact generic one. The phase-approximate subtraction does each on 258 scratch qubits and 352 Toffolis (`csub_square_pm(arith=...)`).
- *The end of the walk.* u kept its $n + 2$ qubits while the record grew beside it. Register sharing ([32] Sec 3.1, `CondInv(share=True)`) frees u's high qubits, and the two that $\tilde{v}$ borrows, as the width schedule narrows; the record grows into them, and the walk backwards claims the same qubits back at the mirror-image round. Unlike the dialog's sharing it never frees x's own qubits, so the replay can still run in them. It adds no gates.

The peak is then the replay: the packed record (669 qubits), the multiplicand and the accumulator (512, the accumulator in x's qubits), the window (16), and one cell's scratch — 1 457 qubits in all. One addition costs 1 436 519 Toffolis (1 424 871 executed), and with signed windows 1 338 469 (1 326 821 executed) on 1 457 qubits: fewer Toffolis than IonQ's 1 392 608 at IonQ's own width, 1 457 logical qubits (IonQ's target was [32]'s 1 462, which includes the 16 window qubits, as the counts here do). Built end to end with signed windows, the whole algorithm is 37 535 781 Toffolis (37 209 637 executed) on 1 457 qubits, against IonQ's 39.0×10^6 on 1 457.

The price is depth: 33 606 905 Toffolis deep, against 29 490 972 for the unsigned circuit on 2 233 qubits. Most of it is row 4's: one addition is 1 114 102 deep in row 3, 1 255 253 once the record is packed and the replay runs in x's qubits, and 1 296 204 with everything above. Reusing a qubit orders the two steps that use it.

39.4 SELECT-SWAP lookups

Each point addition loads three table words: (x, y) twice, $2n$ bits each, and $3x$ once, n bits. Each load is a unary iteration over the table's address, $2^{15} - 2$ ANDs with signed windows, and each unload is free: measured, and repaired by the one merged phase lookup of the addition. SELECT-SWAP [45] trades a load's ANDs for qubits. A lookup over the high $w - k$ address bits writes 2^k consecutive words at once, and k layers of Fredkin gates controlled by the low bits move the addressed word into place: $2^{w-k} - 2$ ANDs plus $(2^k - 1)b$ Fredkins for b-bit words, on $(2^k - 1)b$ extra qubits.

The other words need no uncomputation of their own. After the swaps each holds a word fixed by the whole address, so they are measured in the X basis at once, and their phase kickback joins the merged repair of the addition's unloads, which costs the same with them as without. The extra qubits are live only during the load:

```
def select_swap_lookup(m, addr, out, table, k, group=None):
    w, b = len(addr), len(out)
    k = min(k, w)
    if k == 0:
        lookup(m, addr, out, table)
        return
    lam = 1 << k
    lo, hi = list(addr[:k]), list(addr[k:])

    def word(v):
        return table[v] if v < len(table) else 0
    junk = m.anc((lam - 1) * b, "ssw")
    blocks = [list(out)] + [list(junk[j * b:(j + 1) * b]) for j in range(lam - 1)]
```

```
    wide = [sum(word((v << k) | j) << (j * b) for j in range(lam)) for v in range(1 << (w
    - k))]
    lookup(m, hi, [q for blk in blocks for q in blk], wide)
    for i in reversed(range(k)):                    # block 0 <- word a_lo
        s = 1 << i
        for j in range(s):
            for qa, qb in zip(blocks[j], blocks[j + s]):
                m.ctx.cswap(lo[i], qa, qb)
    G = []                                          # the junk, as a function of addr
    for a in range(1 << w):
        idx, base = swap_network_order(k, a & (lam - 1)), (a >> k) << k
        G.append(sum(word(base | idx[j]) << ((j - 1) * b) for j in range(1, lam)))
    unlookup(m, addr, junk, G, group=group)
    m.free(junk)
```

Whether that is free depends on what else is live then. At a load of (x, y) the accumulator, the loaded pair, the window and the lookup's walk hold 1 054 qubits, so two words per load ($k = 1$) already needs 1 565, above the 1 457 of §39.3. At the $3x$ load only 798 are live, and two words fit in 1 053. The table builds the signed addition of §39.3 with k swap bits for the point loads and for the $3x$ load; every setting was checked at toy size on every input, and with its measurements performed.

k (point loads, $3x$ load)	qubits	Toffolis	executed	Toffoli depth
$(0, 0)$	1 457	1 338 469	1 326 821	1 198 152
$(0, 1)$	1 457	1 322 341	1 310 693	1 182 024
$(1, 1)$	1 565	1 290 597	1 278 949	1 150 276
$(1, 2)$	1 565	1 282 917	1 271 269	1 142 347
$(2, 2)$	2 588	1 268 581	1 256 933	1 126 989
$(3, 3)$	4 635	1 261 413	1 249 765	1 117 009

With only the $3x$ load on SELECT-SWAP the addition costs 1 310 693 executed Toffolis on 1 457 qubits, 16 128 fewer than without it and 5.9% below IonQ's figure at the same width. Built end to end, the whole algorithm is 37 084 197 Toffolis (36 758 053 executed) on 1 457 qubits, 33 155 321 deep. Beyond that each step buys less: 1 271 269 on 1 565 qubits, then 1 256 933 on 2 588.

39.5 Fewer qubits

[32] makes the qubit count a statement about one moment. The peak of a point addition is the Bézout replay, holding the packed GCD record, the multiplicand and the accumulator — $2.355n + O(\sqrt{n})$ plus $2n$ — and whatever scratch its arithmetic draws; every other step can borrow what the replay will need. So the method is to take the scratch out of each cell that runs at the peak, rebuild, find the new peak, and repeat. It moved four times. From the replay it went to the square-subtract, whose exact modular subtraction held $2n$ scratch beside the $2n$-qubit square. Then to the dialog walk, whose controlled subtraction copied its operand first. Then back to the replay, now dominated by the constant adder. Last, to the squarer itself.

The cells that come out are the ones [32] names for its space-optimised variant — CDKM adders wherever no ancillas are free — plus three that shrink the scratch further. A comparator on the CDKM carry chain uses one ancilla instead of k. An all-ones test in chunks of about $\sqrt{k}$, each chunk's AND parked in a flag, uses about $2\sqrt{k}$ ancillas instead of $k - 1$. A constant adder runs CDKM only over the 33 bits of f and adds the carry into the bits above as an *increment* on borrowed qubits [46]: for a register g in any state, $w - g - \overline{g} = w + 1 - 2^{|w|}$, so two subtractions of a dirty register increment w and leave g as it was. Each new cell gives the output of the cell it replaces on every input tried,

failures included. Where the walk and the squarer have headroom — the replay's accumulator is not yet allocated — they keep Gidney adders up to an ancilla budget.

One of these cells corrected this package. Its CDKM adder with a control was documented at the textbook $3n$ Toffolis but put the control on every MAJ and every UMA gate, which is $4n$. The carries can be computed unconditionally and unwound by a UMA that writes the sum bit only under the control — two Toffolis per bit on the way down, one on the way up:

```
def _cuma(ctx, ctrl, c, b, a):
    """UMA that writes the sum bit only when ctrl.  On entry (after an
    *uncontrolled* MAJ) b = y ^ x, c = c_i ^ x, a = c_{i+1}."""
    ctx.ccx(c, b, a)                    # a <- x
    ctx.ccx(ctrl, c, b)                 # b ^= ctrl (c_i ^ x)
    ctx.cx(a, c)                        # c <- c_i
    ctx.cx(a, b)                        # b = y ^ ctrl (x ^ c_i)
```

```
def cdkm_cadd(ctx, ctrl, x, y, carry):
    n = len(y)
    assert len(x) == n and n >= 1
    _maj(ctx, carry, y[0], x[0])
    for i in range(n - 1):
        _maj(ctx, x[i], y[i + 1], x[i + 1])
    for i in range(n - 1, 0, -1):
        _cuma(ctx, ctrl, x[i - 1], y[i], x[i])
    _cuma(ctx, ctrl, carry, y[0], x[0])
```

one windowed point addition, secp256k1, $w = 16$	qubits	Toffolis
cond. inverted, pseudo-Mersenne, IonQ replay (the start)	2 251	1 711 194
+ Fig. 1 packing of the record	2 119	1 716 514
+ replay in x's qubits	1 863	1 716 514
+ CDKM replay cells	1 713	2 042 914
+ CDKM walk adder	1 713	2 284 770
dialog replay instead of IonQ's	1 557	2 338 844
dialog, register sharing, Fig. 1, CDKM replay cells	1 557	2 333 400
+ CDKM square-subtract and point-addition adders	1 389	2 335 990
+ CDKM dialog walk	1 309	2 577 278
+ lean replay cells	1 299	2 868 531
+ CDKM squarer	1 246	2 935 857
+ Gidney adders where there is headroom	1 246	2 853 603
+ SELECT-SWAP on the $3x$ lookup (§39.4)	1 246	2 821 091
+ Luo's register-shared EEA instead of the dialog [33]	1 107	86 033 905
(cond. inverted walk, same cells: its record is not shared)	1 468	2 659 019

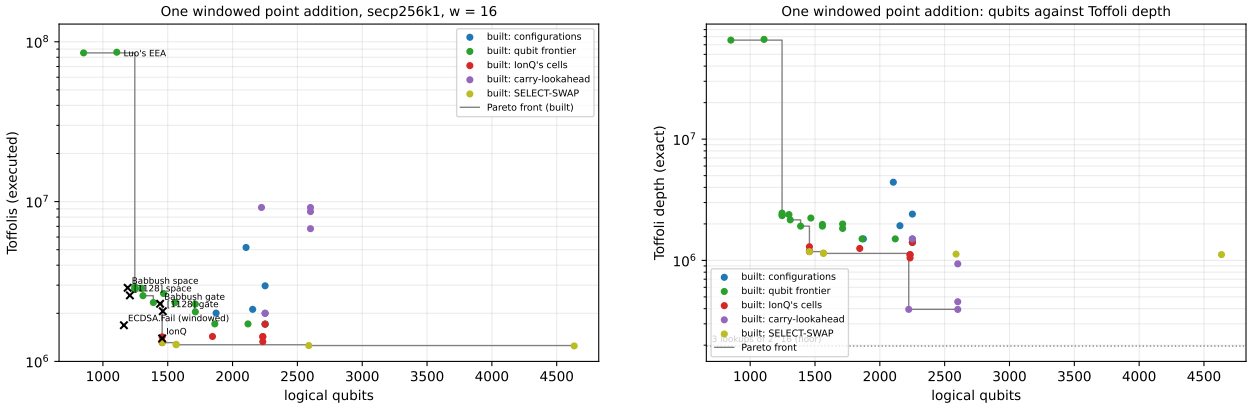
The best row is 1 246 qubits, $4.87n$: the record's 669 (x's own 256 among them), 512 for the multiplicand and the accumulator, 16 for the window, and a few dozen of scratch, most of it the register that holds f. [32] reports $4.355n + O(\sqrt{n})$, or 1 192 qubits plus the window. The price is Toffolis: 2 821 091 here, against 1 711 194 at the 2 251 qubits where the table starts, and against [32]'s 2.39×10^6 for its space-optimised circuit. IonQ reports its Toffoli count at 1 457 qubits; on IonQ's cells, §39.3 reaches both.

39.6 Depth

A qubit count is paid for as long as the computation runs, so the other axis that matters is time, and on an error-corrected machine time is Toffoli depth (times the reaction time). It is measured here exactly: `hier.exact_depth` expands the cached hierarchy gate by gate as it schedules, without ever building the flat circuit, and at toy sizes it equals the flat circuit's depth. Every construction so far ripples, and the GCD's rounds are sequential anyway, so their depth is 0.8–0.95 times their Toffoli count. [31] makes the other choice: carry-lookahead adders [42] of depth $\approx 4\log_2 n$ for $\approx 7n$ Toffolis. Put where the depth is — the replay's doubling and controlled addition, the walk's comparison and subtraction, the squarer — they change the round cost from $O(n)$ to $O(\log n)$. A second, easily missed ingredient: n Toffolis sharing one control take n steps, because a qubit takes part in one gate at a time. So each shared control, of a controlled copy or of the replay's swap, is first fanned out to n copies with CNOTs, which cost no Toffoli depth.

one windowed point addition, $n = 256$, $w = 16$	qubits	Toffolis	Toffoli depth
dialog, pseudo-Mersenne replay (ripple adders)	2 251	2 000 864	1 503 533
+ carry-lookahead replay	2 601	6 769 556	936 530
+ carry-lookahead walk	2 601	8 655 996	456 379
+ carry-lookahead squarer	2 601	9 184 472	394 281
+ register sharing, Fig. 1	2 223	9 189 792	394 917

The lookups do not move: unary iteration over 2^{16} entries is 2^{16} steps deep, so three of them set a floor of 196 608 that no arithmetic reaches below. The two frontiers, every configuration built in this part:



40 The rest of the catalogues

[31] and ECDSA.Fail [33] name more constructions than the ones that set the headline numbers above. Each one below is built as its own module, tested exhaustively or on large samples at toy sizes, and measured here. Signed windows are the one that moved a headline, and §39.2 already counts them.

40.1 Carry-lookahead addition

The Draper–Kutin–Rains–Svore adder [42] (`ec_cla`) computes all carries as a prefix over (generate, propagate) pairs in $O(\log n)$ rounds. Its propagate rounds write clean ancillas, so they are temporary ANDs whose uncompute is free. Only the generate and carry rounds stay Toffolis:

	carry-lookahead			Gidney ripple		
n	Toffolis	depth	qubits	Toffolis	depth	qubits
8	27	11	25	7	7	23
32	179	19	117	31	31	95
64	395	23	243	63	63	191
128	835	27	497	127	127	383
256	1 723	31	1 007	255	255	767

That trade, roughly 7× the Toffolis for $\log n$ depth, is what §39.6 spends.

40.2 Projective coordinates with a quantum addend, and twisted Edwards curves

[31]'s Algorithm 4 is an out-of-place Jacobian addition. With the addend looked up rather than classical (`ec_proj_q`) it costs, at $n = 32$:

one addition, $n = 32$	qubits	Toffolis
affine in place, ctrl (ec_pointadd)	540	262 797
Jacobian + classical affine (ec_proj)	742	157 680
Jacobian + quantum affine	806	174 900
Jacobian + quantum affine, ctrl	903	175 250
Jacobian + quantum Jacobian	998	257 460
Jacobian + quantum Jacobian, ctrl	1 095	257 810
to_affine (once per run)	571	139 066

A cheaper addition does not make a cheaper oracle. A projective triple depends on the path that produced it, so the chain must be undone before the phase estimation reads anything. Undoing it doubles the additions, and at toy size the whole projective oracle costs about twice the affine one. Extended twisted Edwards coordinates (`ec_edwards`, [31] Sec 4.4) have a *complete* law, with no exceptional cases, and need 7 multiplications where Jacobian needs 11. With a classical addend:

	Edwards (Ed25519-shaped)		Weierstrass Jacobian	
n	Toffolis	qubits	Toffolis	qubits
32	44 438	550	143 796	742
64	172 090	1 094	566 132	1 478

40.3 Batch inversion

Montgomery's trick inverts k field elements with one inversion and $3(k-1)$ multiplications. Litinski [44] shares one inversion across k independent affine additions this way. Built clean, with every intermediate uncomputed, it costs 2 inversions plus $5(k-1)$ multiplications against $2k$ inversions (`ec_batch`):

n	k	what	Toffolis, separate	batched	qubits, separate	batched
64	2	inversions	703 732	516 346	1 054	1 118
64	4	inversions	1 407 464	845 306	1 310	1 502
64	8	inversions	2 814 928	1 503 226	1 822	2 510
256	4	inversions	22 169 576	13 414 394	5 156	5 924
256	4	point additions	65 369 012	46 815 704	5 672	7 445

Batching always wins on Toffolis and always costs qubits. It serves the Kaliski path only: in the dialog, a multiplication costs as much as an inversion.

40.4 The register-shared Euclidean algorithm

ECDSA.Fail's narrowest circuit (§5.3.5 of [33]) replaces the dialog with Luo et al.'s extended Euclidean algorithm. The two remainders and the two cross-cofactors share two banks of $n+3$ qubits: the remainders shrink, the cofactors grow into the lanes they free. Each quotient is built one bit at a time by testing shifted multiples, and a rotation cursor aligns $2^{\ell}\rho'$ without copying it. `ec_luo` builds it from a bit-level model that reproduces Luo's worked example row by row. The division is fully reversible: the EEA forward, a multiplication by the inverse, the EEA backward, and the product cleared:

		inversion		division		dialog division	
n	$2n+6\lfloor\log_2 n\rfloor+19$	qubits	Toffolis	qubits	Toffolis	qubits	Toffolis
32	113	113	374 232	177	779 682	284	32 686
64	183	183	1 383 179	311	2 890 248	546	120 780
128	317	317	5 259 375	573	11 012 304	1 064	456 874
256	579	579	20 433 078	1 091	42 836 318	2 088	1 749 162

The inversion lands on Luo's formula exactly. The division needs about half the dialog's qubits for about twenty times its Toffolis. Plugged into the windowed point addition with the lean cells it gives 1 107 qubits, a row of the table in §39.5. That division still holds four field registers at its peak; the next subsection takes it to three.

40.5 Luo's point addition on three field registers

The division above peaks at four field registers: x inside the bank W_2, y, the second bank W_1, and the quotient. [35] gets to three by *venting* y. Once $z = y/x$ sits in W_1's lanes, y is a function of the other registers, so it is measured in the X basis and its register is reused, clean, as the bank of the backward EEA. The measurement leaves a phase $(-1)^{b\cdot y}$ for outcome b. After the backward EEA it is cancelled by recomputing $y = xz$ into the same register, applying Z^b, and uncomputing:

```
def div3(m, x, y, p, arith, steps=None, lean=True):
    ctx, n = m.ctx, len(x)
    bk = L.Banks(m, n, x=x, lean=lean)
    L.eea_forward(m, bk, p, steps=steps)                        # W1 = 0, W2 = x^-1
    xinv, a = L._le(bk, bk.W2, 1, n), L._le(bk, bk.W1, 1, n)
    idle = bk.lt + bk.lq + bk.lr + [bk.P1, bk.P2, bk.Sign] + bk.W2[n:] + bk.W1[n:]
    with L._lent(m, idle):
        mul_acc(m, arith, xinv, y, a)                           # A = y / x
    key = ("luo3", next(_keys))
    MB.vent(m, [xinv, a], y, _quotient(p), key)                 # y = A / x^-1
    bk2 = L.Banks([list(y) + bk.W1[n:]] + list(bk[1:]))          # Y is the bank now
    m.emit_inverse(L.eea_forward, m, bk2, p, None, steps, True, True)
    with L._lent(m, bk.meta() + bk.consumed + bk.W1[n:]):
        mul_acc(m, arith, x, a, y)                              # y again ...
        MB.zfix(m, y, key)                                      # ... cancels the phase
        mul_unacc(m, arith, x, a, y)
    for q1, q2 in zip(y, a):
        ctx.swap(q1, q2)
    bk.free(m)
```

Multiplication is the mirror image, and each costs two EEAs and three multiplications. Here the vent is a logical gate. The basis-state simulator runs it as the XOR it stands for, and `run_live` performs the measurement and applies Z^b from the actual outcome; every input then ends with the

same phase. Built without the Z^b, the same circuit fails that check, so the check can see what the vent does.

The multiplications now take p as a classical constant, because the bank that held it holds z. [35] does its modular reductions with Gidney's constant-workspace classical–quantum adder [39], for any p, built in §40.6. On it the division and multiplication are exact for every odd p, at exactly $3n + 6\lfloor \log_2 n \rfloor + 19$ qubits. For secp256k1 the lean pseudo-Mersenne cells of §39.5 also serve, a little cheaper: they need about 38 clean qubits, fewer than the EEA leaves idle while the multiplications run, so the width stays the EEA's. The table uses them. Luo et al.'s point addition (their Fig. 14) adds a classical point under a control. Their Shor circuit uses signed windows instead, with the looked-up point loaded one coordinate at a time into the one free register — their five lookups, this package's `serial_load`:

three field registers, secp256k1	qubits	Toffolis	X-measurements
in-place division	835	42 103 470	256
in-place multiplication	835	42 103 470	256
controlled addition of a classical point (Fig. 14)	836	85 039 296	512
signed windowed addition, $w = 16$	851	85 204 396	1 792
whole ECDLP-256, 28 additions	851	2 385 774 798	
published [35]: point addition	835	70.29×10^6	
published [33]: narrowest point additions	825–851		

The division lands on [35]'s $3n + 6\lfloor \log_2 n \rfloor + 19 = 835$ exactly, and the windowed addition adds the 16 window qubits: 851, the top of ECDSA.Fail's published range. The price is Toffolis: 85 204 396 per addition, and 2 385 774 798 for the whole algorithm on 851 qubits, 64 times the circuit of §39.4.

40.6 Gidney's classical-quantum adder

Adding a classical constant d into a quantum register is where the other constructions spend their workspace: the constant is written into n clean qubits and added with a quantum–quantum adder. [39] does it with three clean qubits and about $4n$ Toffolis, by *venting* carries. Its key identity is that the carries of an addition are reproduced by adding d into the bitwise complement of the result,

$$\mathrm{carry}(\sim x', d, c_0) = \mathrm{carry}(x, d, c_0), \qquad x' = x + d + c_0,$$

so once bit k is summed, the carry into it is a function of the finished result. A streaming adder therefore holds each carry in one of two clean qubits only while it is needed, then measures it in the X basis. That costs $n - 1$ ANDs and leaves, for each outcome $m_k = 1$, a phase flip by the carry c_k to be done later.

Those phases are done with a dirty register g, using $Z_q CX_{c\to q} Z_q CX_{c\to q} = Z_c$: apply Z^m to g, XOR the carries into g, apply Z^m again, XOR them again. That leaves g as it was and applies exactly $(-1)^{m\cdot c}$. What remains is a circuit that XORs every carry of an addition into a dirty register. [39] takes one from [40] and gives it as a figure; the one here is derived. Write $w_k = c_k \oplus x_k$ and $y_k = x_k \oplus d_k$. Then

$$w_{k+1} = (x_k \oplus x_{k+1}) \oplus y_k\, w_k,$$

whose only product is $y_k w_k$. Let P be the ascending pass $g_{k+1} \oplus= (x_k \oplus x_{k+1}) \oplus y_k g_k$, and Q the same pass without the constant terms. For any g, $P(g) = w \oplus Q(g)$, and Q is linear and invertible, so $P(Q^{-1}(g)) = g \oplus w$: run Q descending, then P, then XOR x_k in to turn w into c. That is about $2n$ Toffolis and no ancilla, and d enters only through X and CNOT gates, so a control on the constant costs nothing:

```
def carry_xor(ctx, x, d, g, ctrl=None, cin=None):
    nb, K = len(x), len(g)
    assert 1 <= K <= nb
    xs = list(x)

    def orig(k, out):                                  # out ^= x_k (original)
        if k >= nb:
            return
        ctx.cx(xs[k], out)
        if k < K:                                      # x_k holds y_k = x_k ^ d_k now
            _flip(ctx, out, _bit(d, k), ctrl)

    for k in range(K):                                 # y_k in place, k < K
        _flip(ctx, xs[k], _bit(d, k), ctrl)
    for k in range(K - 1, 0, -1):                      # Q^{-1}: g_{k+1} ^= y_k g_k
        ctx.ccx(xs[k], g[k - 1], g[k])
    # P, k = 0: g_1 ^= d_0 ^ x_1 ^ y_0 (c_0 ^ d_0)
    _flip(ctx, g[0], _bit(d, 0), ctrl)
    orig(1, g[0])
    if cin is None:
        if _bit(d, 0):
            if ctrl is None:
                ctx.cx(xs[0], g[0])
            else:
                ctx.ccx(ctrl, xs[0], g[0])
    else:
        _flip(ctx, cin, _bit(d, 0), ctrl)              # cin ^ d_0 in place
        ctx.ccx(cin, xs[0], g[0])
        _flip(ctx, cin, _bit(d, 0), ctrl)
    for k in range(1, K):                              # P: g_{k+1} ^= x_k ^ x_{k+1} ^ y_k g_k
        orig(k, g[k])
        orig(k + 1, g[k])
        ctx.ccx(xs[k], g[k - 1], g[k])
    for k in range(1, K + 1):                          # w -> c: g_k ^= x_k
        orig(k, g[k - 1])
    for k in range(K):
        _flip(ctx, xs[k], _bit(d, k), ctrl)
```

If the streaming adder also XORs each carry into g as it makes it, one carry-XOR finishes the repair: $3n$ Toffolis with $n-1$ dirty qubits and two clean ones. Without dirty qubits, each half of the target serves as the other's dirty register, for $4n$ Toffolis and three clean qubits. [35]'s modular addition is then a quantum–quantum CDKM addition, a subtraction of p, a controlled addition of p back, and a comparison that clears the flag. Its doubling is a subtraction of p and a controlled addition back. Every addition of $\pm p$ is Gidney's adder, borrowing the addend as its dirty register. At $n = 256$:

cell, $n = 256$	Toffolis	per n	clean qubits
classical-quantum adder, n - 1 dirty	762	$2.98n$	2
classical-quantum adder, 3 clean	1 015	$3.96n$	3
modular addition b += c a	2 813	$10.99n$	5
modular doubling, dirty-assisted	1 535	$6.00n$	4

The modular addition costs $11.0n$ and the doubling $6.0n$, against [35]'s $11n$ and $6n$. A vent cannot be run backwards: its inverse would be a recomputation, not a free measurement, and its Z fix would

come before the measurement. So the subtraction and the halving are built forwards, from the same steps in reverse with each classical addition of d replaced by one of $2^w - d$, and the package refuses to invert a vent. On these cells the three-register blocks are exact for every odd p. The tests check this on every input at $p = 7, 11, 13$, at exactly $3n + 6\lfloor \log_2 n \rfloor + 19$ qubits, with every vent measured and every Z applied from its outcome. At $n = 256$ the division is 835 qubits and 44 200 815 Toffolis, and the windowed addition 851 qubits and 90 935 988 Toffolis, against 42 103 470 and 85 204 396 on the pseudo-Mersenne cells.

The same adder would take the 33 qubits that hold f out of the fewest-qubit dialog of §39.5. It cannot be dropped in, though: the dialog divides by running its multiplier backwards, so those cells would first need their inverses built forwards.

40.7 ECDSA.Fail's fire census

Most of ECDSA.Fail's commits removed gates. Its exact passes (constant propagation and cancellation, `ec_opt.peephole`) are exact here too. Its census is not. It runs a sample of inputs, records which Toffolis ever fired, and deletes the others (`ec_opt.census`, `strip_unfired`). On the $q = 61$ dialog multiplier, whose 1 680 Toffolis drop to 991 when the census covers *every* input, which makes it exact dead-code removal:

sample size	Toffolis left	fresh inputs wrong
10	752	98.0%
30	891	65.2%
100	957	10.2%
300	984	2.8%

The dead code at this size is the round budget's padding. At $n = 256$ the same padding is the $2.4\sqrt{n}$ rounds the schedule adds, and a census could only find what those rounds never do on the inputs it saw.

41 Verifying it

A wrong ECDLP circuit does not return a wrong answer. It returns a *flat histogram* — and since the classical post-processing verifies its own candidates against the curve, it will happily print the correct k from pure noise. Testing the output is therefore worth almost nothing; three other things are worth a great deal.

Exact simulation at full width. Every arithmetic circuit here is a permutation of basis states, so pushing one basis state through in $O(\text{gates})$ time is exact — and scales to hundreds of qubits where a statevector dies at thirty. Ancillas are checked back to $|0\rangle$ *at the instruction where they are freed*, not merely at the end, which is what catches a circuit that computes the right value and leaves a dirty flag behind. This is sound only for circuits whose phases carry no information; the sole phase and Hadamard gates outside the AND gadget are in the state preparation and the Fourier transform, which are run on Aer instead.

```
def simulate(qc, init=None, checks=(), strict=True):
    """Push one basis state through ‘qc‘ and return the final bit assignment.

    ..."""
    idx = {q: i for i, q in enumerate(qc.qubits)}
    bits = [0] * qc.num_qubits
```

```
    if init:
        for k, v in init.items():
            bits[idx[k] if k in idx else k] = int(v) & 1

    ...
        name = op.name
        w = [idx[q] for q in qargs]

        if name in _SKIP or name in _DIAGONAL:
            return
        if name == "x":
            bits[w[0]] ^= 1
            return
        if name == "swap":
            bits[w[0]], bits[w[1]] = bits[w[1]], bits[w[0]]
            return
        if name == "ecand":
            if strict and bits[w[2]] != 0:
                raise SimError("AND target was not |0>: temporary AND invalid")
            bits[w[2]] = bits[w[0]] & bits[w[1]]
            return
        if name == "ecand_dg":
            if strict and bits[w[2]] != (bits[w[0]] & bits[w[1]]):
                raise SimError("AND-dagger target did not hold a AND b")
            bits[w[2]] = 0
            ...
```

Statevector cross-checks. The fast simulator only sees basis states, and a bug in it would hide from every test that uses it. So the small circuits are also evolved on genuine superpositions and checked for exact fidelity *and* for the absence of leaked amplitude — the statement that ancillas are unentangled, which basis-state testing cannot see. A stray diagonal gate would appear here as a phase, and phases are exactly what Shor depends on.

The listings, too. Every listing in this part is extracted from `shor_qiskit/ec_*.py` by AST at build time, not copied by hand, and the whole part is regenerated and diffed against the document by `bench/ec_check_tex.py`. So a listing cannot drift from the function it claims to show: change the code and the document either follows or the check fails. Where a listing is shortened, the elision is visible as a bare `...` — nothing is silently trimmed.

Property-based tests over random curves. Random prime, random non-singular (a, b), random points and scalars. That is where curve-shape assumptions hide: $a = 0$, $b = 0$, curves with a point of order two, primes where 2 is or is not a square. The harness shrinks failures — staying inside each field's declared domain, and re-checking its own result before reporting it — and it has been validated against injected bugs, where it isolates the exact offending modulus.

what	result
classical models (Kaliski, Montgomery, dialog, Jacobian)	exact
adders, comparators, modular arithmetic	exhaustive $p \leq 61$, random to $n = 128$
Kaliski inversion and division	exhaustive; records and counter clean
affine point addition	every point pair, both control branches
projective point addition	every point pair; every Z under `SHOR_EC_FULL`
in-place multiplication (dialog)	every (x, y) for $p \leq 31$ ($p \leq 61$ full)
ECDLP end to end	every logarithm on two toy curves

41.1 The distribution, and the control that makes it mean something

On $y^2 = x^3 + 5x + 4 \bmod 7$ with $G = (0, 5)$ of order 5, and $Q = [2]G$, the measured peaks sit at $(j_1, j_2) \in \{(0,0), (6,5), (2,3), (3,6), (5,2)\}$ carrying 20%, 10%, 10%, 10%, 10% of the shots. Rounding by $a_i = \text{round}(j_i \cdot 5/8)$ turns those into $(a_1, a_2) = (1,2), (2,4), (3,1), (4,3)$ — and every one of them satisfies $a_2 = 2a_1 \bmod 5$, which is (6) with $k = 2$.

```
def ecdlp_postprocess(counts, order, bits, search=1):
    """Recover k from measured (j1, j2) pairs of the two ECDLP registers.

    `counts` maps (j1, j2) -> number of shots.  Returns [(k, weight)] sorted by
    weight, heaviest first.
    ..."""
    if isinstance(counts, dict):
        items = list(counts.items())
    else:
        items = [(pair, 1) for pair in counts]

    votes, q = {}, 1 << bits
    for (j1, j2), c in items:
        for da in range(-search, search + 1):
            for db in range(-search, search + 1):
                a1 = (round(j1 * order / q) + da) % order
                a2 = (round(j2 * order / q) + db) % order
                if gcd(a1, order) != 1:
                    continue
                k = a2 * pow(a1, -1, order) % order
                w = c / (1 + abs(da) + abs(db)) ** 2
                votes[k] = votes.get(k, 0.0) + w
    return sorted(votes.items(), key=lambda kv: -kv[1])
```

The control that catches a broken circuit

The correct k must be the *top* candidate, not merely *a* candidate that verifies. On both toy curves it is, for every logarithm, holding at least 30% of the vote against a uniform 20% and 7.7% respectively. And feeding the same post-processing a flat, random histogram returns a top share of 21.2% against a uniform 20% — no peak, no signal.

That control exists because an early version of this post-processing did not have it. Ekerå's limited search [38] widens the rounding when 2^m is not a multiple of r, and with the radius set too generously it enumerated the entire group: every k "verified", on every input, including noise. The bug was invisible in the pass/fail of an end-to-end test and obvious the moment the vote shares were printed next to the uniform baseline.

41.2 Complete extended-Edwards arithmetic at toy size

The existing toy `ec_edwards.py` backend implements the standard extended twisted-Edwards mixed addition of Hisil–Wong–Carter–Dawson and RFC 8032 [48]. The now-verified formulas in [31] Sec. 4.4, Fig. 11 are the *same* mixed addition: for a fixed affine addend (x_2, y_2), compute

$$A = (Y_1 - X_1)(y_2 - x_2), \quad B = (Y_1 + X_1)(y_2 + x_2), \quad C = 2dT_1x_2y_2, \quad D = 2Z_1,$$

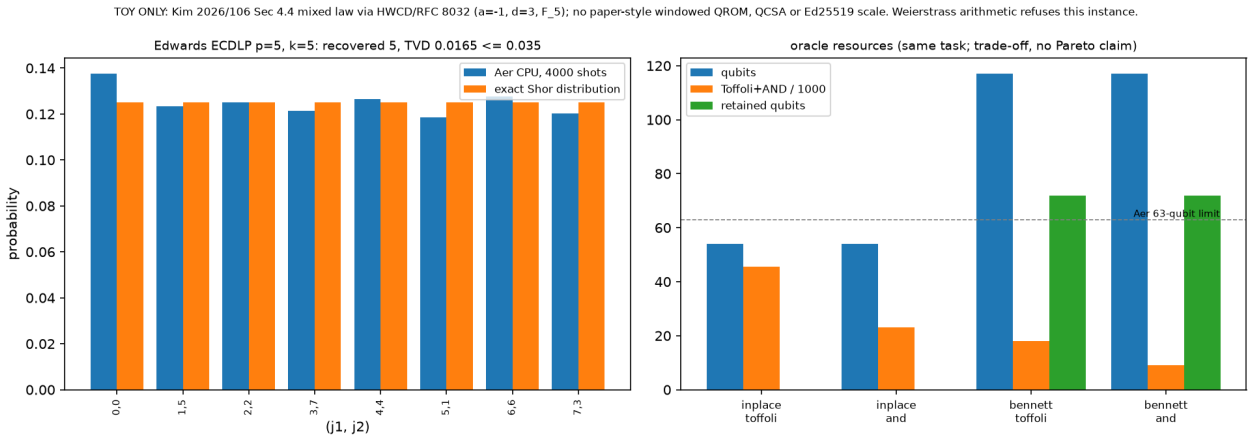
$$E = B - A, \quad F = D - C, \quad G = D + C, \quad H = B + A, \qquad (X_3, Y_3, Z_3, T_3) = (EF, GH, FG, EH).$$

These are the paper's three known-constant products and four output products, seven at the algorithm level; reversible scratch uncomputation adds operations to the compiled circuit. The existing backend selects constants by controlled modular arithmetic instead of the paper's *windowed quantum lookup of three constants* $(y_2 - x_2, y_2 + x_2, 2dx_2y_2)$, and uses a different toy multiplier, so its gate counts cannot be equated to the paper's Table 4 Ed25519 resource estimates. The point is represented by $(X : Y : Z : T)$ with $XY = ZT$, $x = X/Z$, $y = Y/Z$. For d a nonsquare over $\mathbb{F}_p$ and $p \equiv 1 \pmod 4$, the model checks every pair of toy-curve points, including identity, inverse and doubling. Out-of-place controlled addition leaves its input triple available and erases intermediate arithmetic work. The in-place variant uses a forward add and the inverse of addition by the negative point to erase its old affine input, trading extra gates for the absence of retained projective point registers.

For the complete eight-point curve $-x^2+y^2 = 1+3x^2y^2$ over $\mathbb{F}_5$, $P = (1, 2)$ and $Q = [5]P = (4, 3)$, all 64 (u, v) oracle basis inputs and their cleanup pass. The full in-place Hadamard–oracle–inverse-QFT–measurement circuit recovers $k = 5$ on Aer CPU/MPS, with all 42 scratch qubits read as zero; the saved 4 000-shot runs have TVD 0.0165 (Toffoli-only) and 0.0122 (temporary-AND) versus a 0.035 shot-noise limit (`bench/ec_edwards_toy.json`). The equally scoped Bennett oracle is basis checked, *not* Aer sampled: its 117 qubits exceed this host's 63-qubit limit. The existing Weierstrass arithmetic adders on the birationally isomorphic toy curve cannot find a starting offset that avoids their exceptional inputs, so the lookup-table oracle is a distribution reference only, *not* a comparable arithmetic cost or a Pareto point.

style	gates	oracle qubits	Toffoli/AND	retained point qubits	Aer
in-place	Toffoli	54	45 528	0	sampled
in-place	temp. AND	54	23 104	0	sampled
Bennett	Toffoli	117	18 026	72	basis only
Bennett	temp. AND	117	9 136	72	basis only

The paper-convention "Toffoli/AND" column counts an AND compute but not its measurement-assisted uncompute; it is not a physical gate-time estimate. Neither two different gate implementations nor an unsampled wide circuit establish a paper-level resource advantage. Reproduce the figure and JSON with `PYTHONPATH=shor_qiskit:tests:benchvenv/bin/pythonbench/ec_edwards_toy.py`.



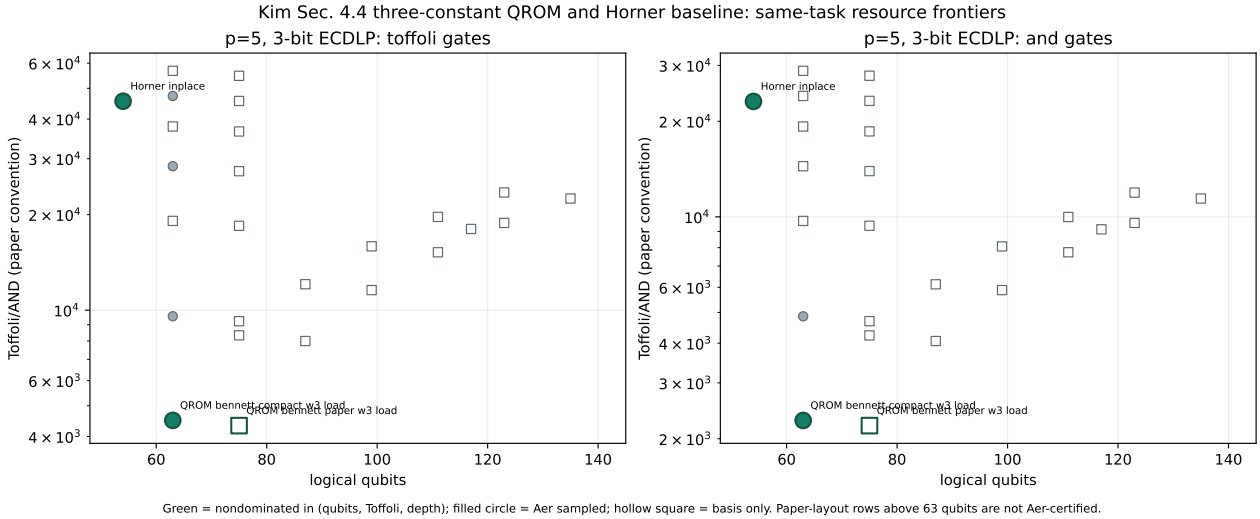
Toy Edwards benchmark. Left: sampled-versus-exact ECDLP output peaks. Right: same-task oracle width, logical Toffoli/AND count (in thousands) and retained input wires. The 63-qubit line distinguishes Aer-tested in-place circuits from basis-checked Bennett circuits; no Weierstrass arithmetic Pareto comparison is inferred.

For context, [31] Table 4 reports its *projected 255-bit Ed25519 point-addition* costs as 64 947 qubits, 3 577 T-depth, 42 377 full depth and $2^{25.06}$ total gates. These are paper synthesis estimates for a different field size and arithmetic stack, *not* local Aer measurements or points on the toy Pareto chart.

41.3 Kim Sec. 4.4: coherent three-constant lookup

The distinct opt-in `ec_edwards_kim.py` path implements the *windowed* part of [31] Sec. 4.4 as well as its mixed law: a single reversible unary-iteration QROM walk writes $(y_2 - x_2, y_2 + x_2, 2dx_2y_2)$ for the selected precomputed window point into three n-qubit registers; the seven-product arithmetic consumes them, and the same lookup uncomputes all three. Window address zero selects the Edwards identity, whose constants are $(1, 1, 0)$, with no exceptional-case branch. A first lookup can initialise the accumulator directly, rather than performing an identity-plus-point addition. This *is not* a toy-point permutation oracle: the selected constants feed the actual modular arithmetic. The compact circuit reuses intermediate registers; a separate Fig. 11-like fresh-register layout is basis-checked but too wide for this host's 63-qubit Aer limit.

For the *same* complete $\mathbb{F}_5$, 3-bit-per-scalar ECDLP task above, all 64 control-register inputs pass with clean scratch for all 26 adder/layout/window choices in Toffoli mode; the temporary-AND variants are separately costed and the selected narrow variants sampled on Aer (`bench/ec_edwards_kim.json`). A no-first-lookup $w = 1$ compact variant performs the same six additions as the controlled-Horner baseline: 56 760 versus 45 528 paper-convention Toffolis at 63 versus 54 qubits. Windowing, rather than merely changing the addend selector, accounts for the reduction. With a first direct lookup, $w = 3$ requires only one remaining mixed addition: the compact in-place variant uses 63 qubits and 9 568 Toffolis (4 862 temporary-AND convention), while compact Bennett uses 63 qubits and 4 496 (2 284 in AND mode). Each of these four circuits recovers $k = 5$ on Aer CPU/MPS in 4 000 shots, all 51 scratch qubits zero on every shot, with TVD 0.0165 below the 0.035 shot-noise bound. Bennett's arithmetic saving comes from a *single window per scalar* at this toy size; larger multi-window Bennett variants retain projective inputs and may exceed the local simulator limit. The fresh-register layout at $w = 3$ has 75 qubits and 4 328 Toffolis, but is *basis-checked only*. These numbers count logical Toffoli/AND/CSWAP equivalents and are not [31]'s 255-bit QCSA synthesis or physical runtimes.

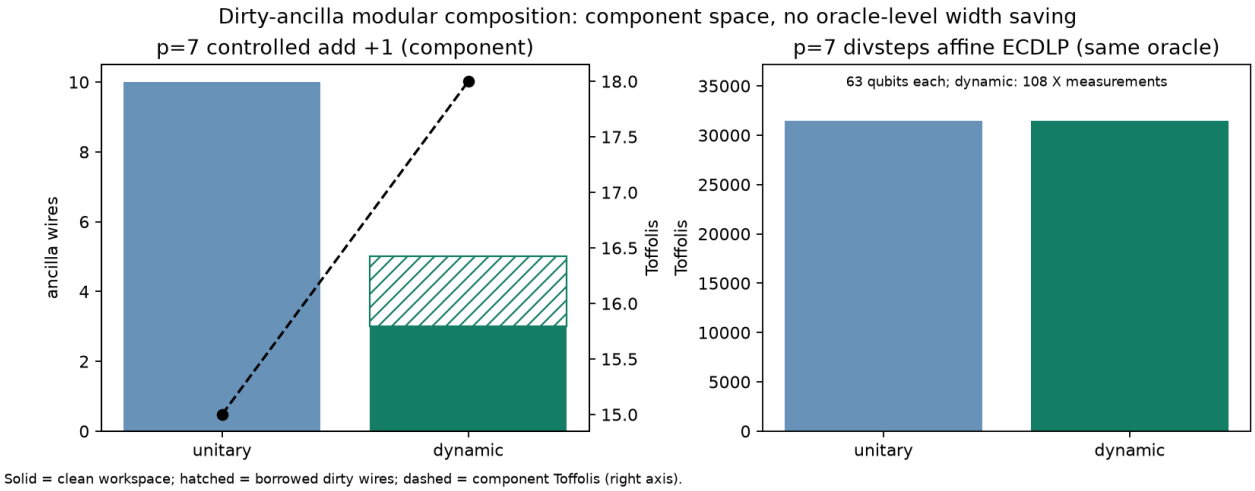


Same-task Edwards resource frontiers. Separate Toffoli and temporary-AND panels compare all 26 variants per mode on qubits, paper-convention Toffolis and circuit depth. Filled circles were sampled end to end on Aer; hollow squares were only basis-checked. Three resource-nondominated rows in each mode are the 54-qubit Horner in-place baseline, the 63-qubit compact Bennett QROM $w = 3$ variant, and the unsampled 75-qubit fresh-register Bennett $w = 3$ variant. An unsampled resource point is *not* an Aer-certified performance claim. The other ECDLP and order-finding graphs retain their distinct tasks and are not merged into this frontier.

41.4 Dynamic dirty-ancilla modular addition in a toy affine oracle

Gidney's measured-carry constant adder [39] has now been composed into an *opt-in* controlled addition $y \mapsto y + [c]k \bmod p$ in `ec_dynamic_modarith.py`. A signed offset first computes the reduction flag; a second conditional addition restores the modulus where needed; a carry-out computation erases the flag. The dirty workspace is returned in its original state even when entangled with other registers. Every measured carry is corrected by classical feed-forward before reuse. The inverse operation is a *new forward* addition of $p - k$, not an attempt to invert a measurement; this matters when the affine point-add circuit erases its input. The hybrid oracle selects the dynamic path only at four constant-addition steps and leaves the other eleven arithmetic steps unitary. It does not make the whole affine construction a low-space dynamic circuit.

For the $p = 7$, $k = 1$ controlled modular-add *component*, the unitary construction costs 15 paper-convention Toffoli/AND operations and ten clean ancillas; the dynamic construction costs 18 Toffolis, three clean ancillas, two borrowed dirty ancillas and eleven measured carries. These are *component* workspace counts, not complete-oracle widths. For the same $p = 7$ divsteps affine ECDLP oracle, the recorded comparison is 63 qubits and 31 488 versus 63 qubits and 31 456 paper-convention Toffoli/AND operations (unitary versus hybrid), with 108 additional X-basis measurements for the hybrid. The 32-operation reduction is small; the whole oracle does *not* save a qubit. Both oracle variants pass all 16 control-register basis inputs and all measurement-branch phase/cleanup checks. The hybrid's 1 024-shot Aer CPU/MPS run recovers $k = 4$, with TVD 0.0479 against a 0.0791 shot-noise bound. The alternative Kaliski hybrid has 67 qubits and is basis/branch checked only, exceeding this host's 63-qubit simulator limit. Counts and sampled metadata are in `bench/ec_dynamic_modarith.json`. Since a measured-carry gate's physical cost differs from the paper-convention count, none of these numbers establish a physical runtime improvement.



Dynamic modular composition. Left: clean workspace (solid), borrowed dirty workspace (hatched), and Toffolis (dashed, right scale) for one controlled add. Right: equal-task whole-oracle costs, with unchanged width. These are separate comparisons, not a new cross-task Pareto frontier. Reproduce with `venv/bin/pythonbench/plot_optimisation_frontiers.py`.

42 What is not here

Named rather than glossed:

- **A fully dynamic low-space arithmetic oracle.** The opt-in composition above covers constant additions, not the other eleven arithmetic steps; it leaves the complete-oracle width unchanged.
- **Cryptographic-scale Ed25519 resource reproduction.** The complete toy arithmetic and three-constant windowed lookup above validate [31] Sec. 4.4 at toy size, but do not reproduce the paper's 255-bit QCSA arithmetic, Table 4 synthesis or physical costs.
- **Register sharing** ([32] Sec 3.1) is built beside the dialog of §36.6, not inside it, and beside IonQ's walk (§39.3): its failure mode is a register overflow, which the ancilla checks turn into a counted failure. The dialog of §36.6 itself stays exact.
- **A fully dirty-workspace dialog adder** [39]. The lean adder of §39.5 borrows dirty qubits above the top bit of f but keeps f in clean workspace; the separate opt-in modular constant-adder composition above does not replace the dialog's cells.
- **IonQ's Karatsuba square-subtract** (their Algorithms 7–11), with its fused slice additions and phase comparators. The square-subtract here is the dedicated squarer and a fold; a Karatsuba split composed from this package's general cells was built and measured, and costs more.
- **Physical-level estimates** for QLDPC architectures. The surface-code model of Gidney 2025 is built and reproduces its headline from its inputs; QLDPC figures are recorded, not modelled.

The Montgomery implementation now exposes both layouts from [31]: the *depth-optimised* form allocates fresh accumulator outputs per carry-save addition, while the *qubit-optimised* form reuses the accumulator in place. The former uses more qubits and is shallower; both are checked against the same Montgomery reference by the tests.

Where this part sits

The affine construction is [29] as improved by [30], with [31]'s modifications: 2017–2020 with a 2026 finish. The projective addition and the carry-save Montgomery multiplier are [31]. The Euclidean dialog, the approximate and pseudo-Mersenne arithmetic, and the windowed addition are [32]. What is deliberately still absent is the *scalable* residue-number-system factoring architecture that cuts qubit counts by changing what a number *is*, rather than how it is added. The separate toy high-bit RNS demonstration above does not change the ECDLP oracles or reproduce CFS's low-bit quantum circuit.

The honest limit of what runs on a laptop: the circuit with real arithmetic is verified on every basis state, which for a permutation is the whole truth, but at 69 qubits for a seven-element field it cannot be run in superposition. The end-to-end demonstration uses a permutation oracle of identical circuit shape. Both halves are tested; the join between them is an argument, not a simulation.

Part VIII — Factoring at size, and why elliptic curves fall first

Parts I–VI factor with the textbook modular exponentiation: $4n$ Fourier adders per multiplication and $2n$ counting qubits, verified exhaustively on numbers a laptop can simulate. Part VII built the elliptic-curve discrete logarithm up to secp256k1. This part does the same for RSA with the construction that holds the current record, Gidney's 2025 approximate residue arithmetic [43]. It builds that construction at $n = 1024$–4096 and puts the two side by side, which is what the comparison "ECDSA or RSA, which falls first" actually needs.

43 Gidney 2025, built

43.1 What the circuit computes

Shor's algorithm with Ekerå–Håstad's short discrete logarithm needs $V = g^e \bmod N$ for an exponent e of $m = n/2 + n/s$ qubits. With e read w_1 bits at a time, g^e is the exact integer product of $W_1 = \lceil m/w_1 \rceil$ classical factors below N. So it is determined by its residues modulo any set P of small primes whose product L exceeds N^{W_1}, and those residues can be computed one prime at a time in registers of $\ell \approx 21$ qubits. For each prime p:

1. *loop1* adds looked-up discrete logarithms, $S_p = \sum_j \mathrm{dlog}_{g_p} M_{j,e_j}$, so the multiplications become additions. It adds the *difference* between this prime's table and the previous one's, so computing prime i and uncomputing prime $i-1$ is a single pass.
2. *loop2* reduces S_p modulo $p-1$ by binary long division.
3. *loop3* computes $V_p = g_p^{S_p} \bmod p$ by windowed multiplications, measuring each old result out instead of uncomputing it.
4. *loop4* adds the top f bits of $V_p u_p \bmod L \bmod N$ (the CRT basis) into the output.
5. *unloop3* and *unloop2* undo loops 3 and 2.

Only the output accumulator is large, and it keeps just $f = 33$ bits. It starts from a random mask, which is what makes the truncation harmless to period finding. Every modular addition is a subtraction with an extra borrow qubit that is cleared by measurement, and every lookup is unloaded the same way.

43.2 How it is checked

The circuit (`rns.py`) is built on this package's machine from the same adders and lookups as Part VII. A bit-exact integer model (`rns_classical.reference`) replays every register: the dlog sums, the long division, the windowed exponentiation, and the truncated, masked accumulation. At $N = 1003$ and $N = 3127$ the circuit's output equals the model on every exponent, with random masks and every ancilla checked clean. Separately, the model's approximation stays within [43]'s deviation bound for every exponent at three toy moduli.

Knowing the answer, the whole output distribution can be checked too. The circuit is a permutation on basis states, so its frequency distribution follows exactly from what it computes on each exponent: Fourier transform each set of exponents the masked output register cannot tell apart, and add the squares. For Ekerå–Håstad at $N = 241 \times 251$ with $s = 2$, the residue system has 7 primes, none dividing N, and $L \bmod N = 6$; with $f = 13$ kept bits the worst deviation is 0.11% of N

(bound 2.1%). The textbook oracle's distribution equals the closed form to 10^{-16}. The approximate, masked one is at total variation 0.105 from it and puts 0.893 of its mass on the pairs that carry 90% of the exact distribution (0.900 for the exact oracle): the same peaks. The masks trade one way: the window of values they blur together must stay well below the period (1 024 against 3 000 here), and wider masks move the distance up. At smaller moduli [43]'s bound $3|P|\ell/2^f$ exceeds one, and the approximation is visibly unfaithful. That is a statement about toy sizes, not about $n = 2048$, where the bound is $\approx 10^{-3}$.

43.3 At size

A shot is $|P|$ iterations of one prime's loops, plus a final loop1 and one merged phase repair per exponent window. Every iteration is the same circuit up to its tables, and no gate count here depends on the tables: lookups are unary or ANF, adders and comparisons have fixed width. The bench builds the iteration for three different primes of a real random residue system, checks that their counts agree, and multiplies by $|P|$:

n	$\lvert P\rvert$	one iteration	per shot	per factoring	[43]	qubits (paper)
1024	6 242	20 109	1.26×10^8	1.18×10^9	1.10×10^9	800 (742)
1536	11 956	30 446	3.64×10^8	3.39×10^9	3.10×10^9	1 138 (1 074)
2048	21 320	35 449	7.56×10^8	6.95×10^9	6.50×10^9	1 467 (1 399)
3072	47 814	44 858	2.14×10^9	1.97×10^{10}	1.90×10^{10}	2 115 (2 043)
4096	74 245	62 774	4.66×10^9	4.29×10^{10}	4.00×10^{10}	2 766 (2 692)

The Toffolis are executed ones (a measured repair counted at its probability), for comparison with the paper, whose Table 5 does the same. Per factoring means per shot times the expected 9.2 shots. The built counts are 7% above the paper's at $n = 2048$. Part of the difference is that loop3's lookup repairs are done in place here, not merged into unloop3's.

The qubit counts follow the paper's formula $m + 3f + 2\ell + \operatorname{len} m$ plus the adders' scratch. The paper's own Table 5 is lower than its formula: 1 399 against 1 432 at $n = 2048$. Fed to the surface-code model of §44, the paper's own inputs give its headline, 897 864 physical qubits and 4.96 days.

44 From logical to physical qubits

Everything so far counts *logical* qubits and Toffolis. A machine is made of noisy *physical* qubits that fail about once per thousand operations, so every logical qubit is stored in an error-correcting code and every Toffoli is fed by a magic-state factory. This section turns logical counts into physical qubits and wall-clock time, with the surface-code model of [43] (Sec. 3.2) as implemented in `physical.py`. The model assumes a physical error rate $p = 10^{-3}$, a 1 μs surface-code cycle (one round of parity checks) and a 10 μs reaction time (decoding plus control latency).

44.1 Choosing the code distance

A surface-code patch of distance d stores one logical qubit. Its parity checks are measured every round, a classical decoder pairs up the checks that change, and an error goes undetected only if a chain of about $d/2$ physical errors crosses the patch. Below threshold, the logical error per patch per round therefore falls exponentially with d:

$$P_L(d) \approx A\Big(\frac{p}{p_{\text{th}}}\Big)^{(d+1)/2}, \qquad A = 0.1, \quad p_{\text{th}} = 0.0122,$$

calibrated so that $d = 25$ is the smallest distance reaching 10^{-15} at $p = 10^{-3}$, as in [43] (Fig. 6). Each step $d \to d + 2$ multiplies P_L by p/p_{th}, a factor of about $1/12$ at $p = 10^{-3}$.

The distance follows from an error budget. A run fails if any logical qubit fails in any round, so its failure probability is about

$$P_L \times (\text{logical qubits}) \times (\text{rounds}).$$

ECDLP-256 keeps about 1 600 logical qubits busy, factories included, for 15 minutes: 1.4×10^{12} qubit-rounds. RSA-2048 keeps about 1 600 busy for 12 hours: 7×10^{13}. A target of $P_L = 10^{-15}$ per patch per round makes the first fail 0.1% of the time and the second 7%. The distance is then the smallest odd d with

$$\frac{d+1}{2} \;\geq\; \frac{\log(A/P_L)}{\log(p_{\text{th}}/p)} = \frac{14}{1.09} = 12.9 \quad\Longrightarrow\quad d = 25.$$

Because d grows with the logarithm of the run, a run ten times longer needs one more step of the distance, not ten times the qubits. The physical error rate matters far more:

physical error p	p_{th}/p	distance d	physical qubits per working logical qubit
10^{-3}	12	25	1 352
5×10^{-4}	24	21	968
10^{-4}	122	13	392

44.2 Counting physical qubits

The model divides the machine into three regions.

- **Hot storage.** A working logical qubit is a distance-d patch with its share of routing space: $2(d+1)^2$ physical qubits, 1 352 at $d = 25$.
- **Cold storage.** Logical qubits that sit idle for long stretches, like RSA's exponent register outside its control windows, are kept in *yoked* surface codes: about 430 physical qubits each at the same target, a third of a hot patch.
- **Compute.** A fixed block of 7×18 hot patches holds six CCZ factories of 3×4 patches each (magic-state cultivation, then $8T \to$ CCZ distillation) and three columns of lattice-surgery workspace: $126 \times 1\,352 = 170\,352$ physical qubits.

So

$$\text{physical qubits} = 1\,352 \cdot (\text{hot logical}) + 430 \cdot (\text{cold logical}) + 170\,352.$$

44.3 Counting time

A factory delivers one CCZ state, enough for one Toffoli, about every 150 rounds; six of them give one every 25 μs. The time of a shot is the larger of two limits: the factory throughput, Toffolis $\times\, 25\,\mu$s, and the reaction limit, the chain of Toffolis that must each wait 10 μs for the decoder. For every circuit in this tutorial the throughput binds. The expected time to a solution multiplies by the expected number of shots and divides by the probability that a run is error-free.

44.4 Two worked examples

ECDLP-256 on IonQ's cells with signed windows (Part VII): 1 457 hot logical qubits, 3.68×10^7 Toffolis, one shot.

$$1\,457 \times 1\,352 + 170\,352 = 2\,140\,216 \text{ physical qubits}, \qquad 3.68 \times 10^7 \times 25\,\mu\text{s} = 920\text{ s} \approx 15 \text{ minutes}.$$

RSA-2048 from [43]'s own inputs: 1 280 cold input qubits, 131 hot ones, 6.5×10^9 Toffolis over an expected 9.2 shots of 12.07 hours.

$$1\,280 \times 430 + 131 \times 1\,352 + 170\,352 = 550\,400 + 177\,112 + 170\,352 = 897\,864 \text{ physical qubits}.$$

The time is $12.07\text{ h} \times 9.2 = 4.63$ days of running. The paper protects fewer than 1 600 logical qubits for about 12 hours, 6.9×10^{13} qubit-rounds, so a run is error-free with probability $(1-10^{-15})^{6.9\times10^{13}} = 0.933$, and the expected time is $4.63/0.933 = 4.96$ days: the paper's headline. The two figures, 4.63 and 4.96 days, found in comparisons of this paper differ only by that division.

44.5 In code

```
@dataclass
class SurfaceCode:
    p: float = 1e-3                # physical error rate
    cycle_us: float = 1.0          # surface-code cycle
    reaction_us: float = 10.0      # control-system reaction time
    target: float = 1e-15          # per-patch per-round logical error
    a: float = 0.1
    p_th: float = 0.0122
    cold_per_logical: float = 430.0      # yoked storage at the same target
    factories: int = 6

    def logical_error(self, d):
        return self.a * (self.p / self.p_th) ** ((d + 1) / 2)

    def distance(self):
        d = 3
        while self.logical_error(d) > self.target:
            d += 2
        return d

    def hot_per_logical(self, d=None):
        d = d or self.distance()
        return 2 * (d + 1) ** 2
```

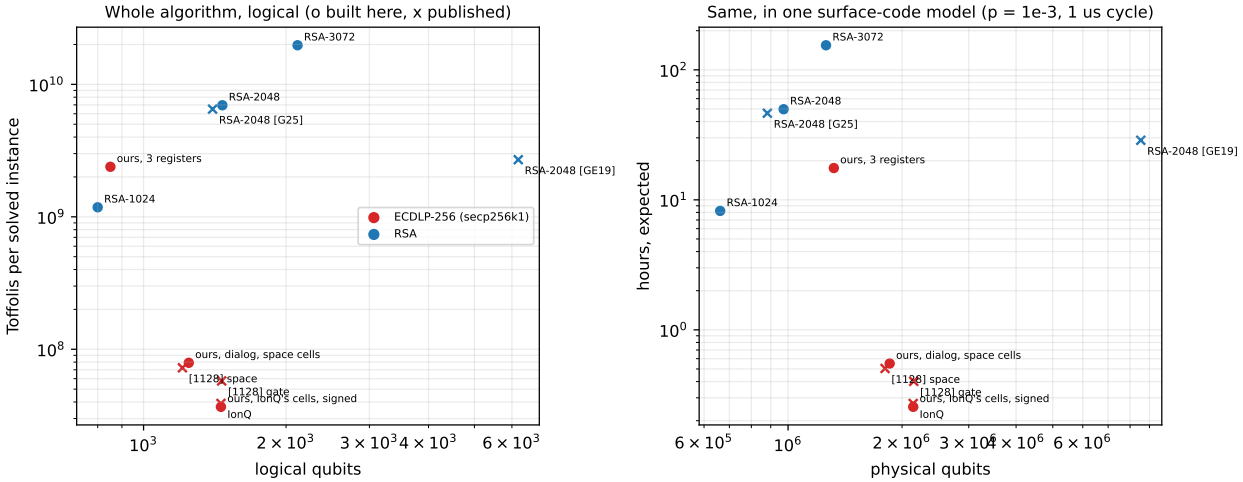
`estimate(profile)` applies the model to any logical profile, measured off this package's circuits or restated from a paper, and returns the distance, the physical qubits by region, the hours per shot and the expected days. `g25_rsa2048()` restates the paper's inputs; reproducing 897 864 qubits and 4.96 days from them is a test.

From logical to physical, in one line

At $p = 10^{-3}$, ask for 10^{-15} per qubit per round: $d = 25$. Then physical qubits $\approx 1\,352$ per working logical qubit $+430$ per idle one $+170\,352$ for factories and workspace, and runtime $\approx$ Toffolis $\times\, 25\,\mu$s.

The model is the superconducting surface-code one. QLDPC codes on atoms and ions (Pinnacle, Cain et al., IonQ) need far fewer physical qubits per logical one, but their costs depend on code tables, connectivity and schedules this module does not have; their published figures are recorded, not modelled.

45 ECDSA against RSA



whole algorithm	logical qubits	Toffolis	physical qubits	days
ECDLP-256, IonQ's cells, signed windows	1 457	3.68×10^7	2 140 216	0.01
ECDLP-256, dialog on the space cells	1 246	7.91×10^7	1 854 944	0.02
ECDLP-256, fewest qubits	851	2.39×10^9	1 320 904	0.73
RSA-1024, Gidney 2025 residue arithmetic	800	1.18×10^9	661 872	0.34
RSA-2048, Gidney 2025 residue arithmetic	1 467	6.95×10^9	973 576	2.07
RSA-3072, Gidney 2025 residue arithmetic	2 115	1.97×10^{10}	1 259 592	6.44
RSA-2048, Gidney 2025 *(published)*	1 399	6.50×10^9	881 640	1.93
ECDLP-256, IonQ *(published)*	1 457	3.90×10^7	2 140 216	0.01
ECDLP-256, [1128] space-optimised *(published)*	1 208	7.25×10^7	1 803 568	0.02
ECDLP-256, [1128] gate-optimised *(published)*	1 462	5.77×10^7	2 146 976	0.02
RSA-2048, Gidney-Ekera 2019 *(published)*	6 189	2.70×10^9	8 537 880	1.20

Every row uses the physical model of §44: $p = 10^{-3}$, a 1 μs cycle, a 10 μs reaction time, and six CCZ factories giving one CCZ every 25 μs. Runtime is the larger of that throughput and the exact Toffoli depth times the reaction time; here the throughput always binds. RSA's exponent register, idle except for its control windows, sits in cold yoked storage at a third of the hot cost. The elliptic-curve circuits recycle their control qubits and keep everything else busy. That is why RSA-2048 needs fewer *physical* qubits than secp256k1 at a comparable logical count, while taking 195 times as long.

The logical qubit counts are close: 1 457 for secp256k1 on IonQ's cells with signed windows (Part VII, §§39.3–39.4), 851 on three field registers (§40.5), against 1 467 for RSA-2048. The Toffoli counts are not: 3.68×10^7 against 6.95×10^9, a factor of 189. Four things explain that factor.

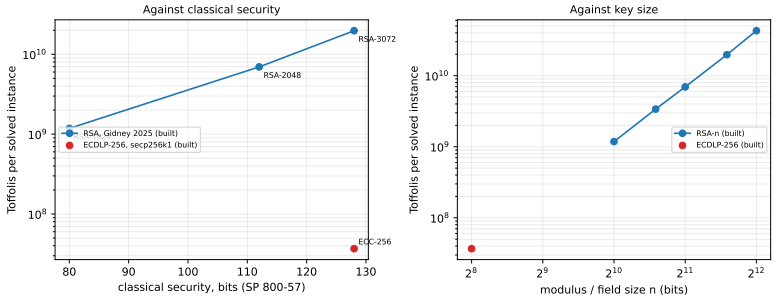
1. **The numbers are eight times shorter.** Both algorithms spend their time on modular arithmetic, whose cost grows at least as n^2 per group operation. An elliptic-curve group operation needs an inversion, dozens of multiplications' worth. That buys a far smaller n at the same classical security: generic attacks on elliptic curves run in $2^{n/2}$, while factoring has

the sub-exponential number field sieve. Shor is polynomial in both cases, so the quantum cost follows n, not the classical security level.

2. **One shot against nine.** The windowed ECDLP circuit succeeds in one run with high probability. Ekerå–Håstad with $s = 8$ needs $s + 1$ good shots, and masking and deviation cost about 1% more: 9.2 expected shots, each a full exponentiation.

3. **Approximation buys qubits, not Toffolis.** The residue arithmetic never holds an n-bit number in superposition except the output's top f bits. That is how RSA-2048 fits in 1 467 qubits, $0.7n$, while secp256k1 needs 3.3–5.7n. The price is Toffolis: per prime, loop1 reads all $W_1 = 214$ exponent windows, and there are $|P| \approx nW_1/\ell \approx 21\,000$ primes, so a shot costs $\Theta(n\,m^2/(\ell\,w_1^2))$ lookups and additions of $\approx \ell$ bits.

4. **At equal classical security the gap widens.** RSA-2048 gives 112-bit security. RSA-3072, the RSA size matched to secp256k1's 128 bits, costs 1.97×10^{10} Toffolis here, 537× the elliptic-curve count.



The gap, in one line

At comparable qubit counts, breaking secp256k1 costs about 189 times fewer Toffolis than factoring RSA-2048, and 537 times fewer than RSA-3072 at equal classical security. In the same surface-code model that is 15 minutes against 2.1 days. Elliptic-curve keys of today's sizes fall before RSA keys of today's sizes.

Part IX — The physics beneath the circuits

What this part is for

Parts I–VIII count logical qubits and Toffolis, and §44 converts them into physical qubits and days with three numbers: a physical error rate $p = 10^{-3}$, a 1 μs surface-code cycle and a 10 μs reaction time. This part explains where those numbers come from. It starts from the postulates and works down to the hardware: what a qubit is as a quantum system, why a gate is a rotation and what physically produces it, what an error is and why a code can discretise it, how transmons, ions, atoms, spins and photons each realise the two-level system, what their measured fidelities and times were in September 2026, and how all of this feeds the estimate. It ends with the record and the forecasts.

The main source is Olivier Ezratty's *Understanding Quantum Technologies 2026* [50], a 1,800-page survey updated to September 2026, with the primary papers it rests on cited where a number is quoted. The mathematics is worked out here rather than copied.

46 Quantum mechanics, as the circuits need it

Everything in Parts I–VIII used four facts about quantum mechanics: states are vectors, operations are unitary matrices, measurement is a projection with Born-rule probabilities, and several systems compose by the tensor product. This section restates those facts with the care needed to understand noise and error correction, and adds the two that the circuits never showed: how a Hamiltonian turns into a gate, and what happens when the system is not isolated.

46.1 States, observables, and the Born rule

Pure states. An isolated system is described by a unit vector $|\psi\rangle$ in a complex Hilbert space $\mathcal{H}$. A qubit is any system for which we use a two-dimensional subspace, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$. A global phase is unobservable, so a qubit has two real parameters, usually written $|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}|1\rangle$: a point on the Bloch sphere, with $|0\rangle$ at the north pole, $|1\rangle$ at the south, and the equator holding the equal superpositions $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ and $|\pm i\rangle = (|0\rangle \pm i|1\rangle)/\sqrt{2}$.

Observables. A measurable quantity is a Hermitian operator $A = A^\dagger$. By the spectral theorem $A = \sum_a a\,P_a$ with real eigenvalues a and orthogonal projectors P_a ($P_a^2 = P_a$, $P_aP_b = 0$ for $a \neq b$, $\sum_a P_a = I$). The Pauli matrices

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

are both Hermitian and unitary, square to I, anticommute pairwise and satisfy $XY = iZ$ and cyclic permutations. Together with I they span all 2×2 matrices: *any* single-qubit operator, in particular any error, is $aI + bX + cY + dZ$. That linear-algebra fact is the whole reason quantum error correction is possible (§46.6).

Born rule and projection. Measuring A on $|\psi\rangle$ yields a with probability $p(a) = \langle\psi|P_a|\psi\rangle = \|P_a|\psi\rangle\|^2$, after which the state is $P_a|\psi\rangle/\sqrt{p(a)}$. Measuring again immediately returns the same a. For a qubit measured in the computational basis, $p(0) = |\alpha|^2$ and $p(1) = |\beta|^2$; the expectation of Z is $|\alpha|^2 - |\beta|^2$. Measuring in another basis means rotating first: an "X measurement" is H followed by a Z measurement, because $HZH = X$. This is what every stabiliser circuit in §48 does, and what the ancilla measurements of §19.6 did: an X-basis measurement is a Hadamard and a readout.

Mixed states. When the state is not known exactly, or when we look at part of an entangled system, the right object is the density operator $\rho = \sum_i p_i|\psi_i\rangle\langle\psi_i|$: positive, Hermitian, trace one, with $\langle A\rangle = \mathrm{tr}(A\rho)$ and $p(a) = \mathrm{tr}(P_a\rho)$. A state is pure iff $\rho^2 = \rho$, iff $\mathrm{tr}\,\rho^2 = 1$. For one qubit every density operator is

$$\rho = \tfrac{1}{2}\big(I + r_xX + r_yY + r_zZ\big), \qquad |\vec{r}| \le 1,$$

so mixed states fill the Bloch *ball*; the surface is the pure states and the centre $I/2$ is the maximally mixed state, which gives 50/50 in every basis. Decoherence is the motion of $\vec{r}$ into the ball (§46.6). The off-diagonal entries of ρ in the computational basis, $\rho_{01} = (r_x - ir_y)/2$, are the *coherences*: they carry the relative phase φ, and they are what dephasing destroys.

46.2 Commutators, uncertainty, and why pulses must be slow

Two observables can be known simultaneously iff they commute. If $[A, B] = AB - BA \neq 0$ they cannot, and the Robertson inequality quantifies it:

$$\Delta A\,\Delta B \;\ge\; \tfrac{1}{2}\big|\langle[A, B]\rangle\big|,$$

with $\Delta A^2 = \langle A^2\rangle - \langle A\rangle^2$. For position and momentum $[x,p] = i\hbar$ gives Heisenberg's $\Delta x\,\Delta p \geq \hbar/2$. For a qubit, $[X,Z] = -2iY$, so $\Delta X\,\Delta Z \geq |\langle Y\rangle|$: a state with a sharp Z value has a random X value, which is exactly why measuring X after Z gives a coin flip.

The same mathematics governs control pulses, because a pulse of duration Δt has a frequency spread $\Delta f \gtrsim 1/(4\pi\Delta t)$ (for a Gaussian envelope, $\Delta t \cdot \Delta f \approx 0.44$ in full widths). A 20 ns microwave pulse on a 5 GHz transmon has a bandwidth of about 22 MHz. Two consequences run through §47: neighbouring qubits must be detuned from each other by more than that or the pulse drives both (*frequency crowding*), and a transmon's next level $|2\rangle$, only 200–300 MHz away, is close enough that fast pulses leak into it (*leakage*), which is what DRAG pulse shaping is for.

46.3 Unitary evolution: from Hamiltonian to gate

A closed system evolves by the Schrödinger equation $i\hbar\,\partial_t|\psi\rangle = H|\psi\rangle$ with Hermitian Hamiltonian H. For time-independent H the solution is $|\psi(t)\rangle = U(t)|\psi(0)\rangle$ with

$$U(t) = e^{-iHt/\hbar},$$

unitary because H is Hermitian. Every gate is such a U for some H applied for some time. For a qubit, any Hamiltonian is $H = \frac{\hbar}{2}(\omega_0 Z + \Omega\,\hat{n}\cdot\vec{\sigma})$ up to a constant, and

$$e^{-i\theta\,\hat{n}\cdot\vec{\sigma}/2} = \cos\tfrac{\theta}{2}\,I - i\sin\tfrac{\theta}{2}\,\hat{n}\cdot\vec{\sigma}$$

is the rotation of the Bloch vector by angle θ about the axis $\hat{n}$. So $X = -iR_x(\pi)$, $H = -iR_y(\pi/2)R_x(\pi)$ up to phase, $S = R_z(\pi/2)$, $T = R_z(\pi/4)$: every single-qubit gate is a rotation, and a rotation is a Hamiltonian switched on for a time. The small QFT rotations $R_k = R_z(2\pi/2^k)$ of Part I are physically the easiest gates of all — a shorter pulse, or no pulse at all — which is why their fault-tolerant cost of $\approx$ 60–100 T gates each (Part V) is so striking: the difficulty is not physics, it is that a code can protect only a discrete set of rotations.

Two-qubit gates come from interaction Hamiltonians. Three examples cover the hardware:

- ZZ **coupling.** $H = \frac{\hbar J}{4} Z\otimes Z$ for a time t gives $U = \mathrm{diag}(e^{-i\phi}, e^{i\phi}, e^{i\phi}, e^{-i\phi})$ with $\phi = Jt/4$. At $Jt = \pi$ this is CZ up to single-qubit Z rotations. Tunable-coupler transmons and Rydberg atoms both produce a state-dependent energy shift of this form.
- XX **coupling.** $H = \frac{\hbar\chi}{2} X\otimes X$ for $\chi t = \pi/2$ gives $e^{-i\frac{\pi}{4}X\otimes X}$, which is a maximally entangling gate equivalent to CNOT up to single-qubit rotations. This is the Mølmer–Sørensen gate of trapped ions.
- **Exchange.** $H = J\,\vec{S}_1\cdot\vec{S}_2$ between two spins swaps them: $e^{-iHt/\hbar}$ at $Jt/\hbar = \pi$ is SWAP, at $\pi/2$ it is $\sqrt{\mathrm{SWAP}}$, and two $\sqrt{\mathrm{SWAP}}$s with a Z rotation between them make a CNOT. This is the gate of silicon spin qubits.

In every case the entangling gate is slower and dirtier than single-qubit rotations because it needs an interaction between two separately isolated objects, and the interaction must be switched on and off cleanly. That is why two-qubit error rates dominate every error budget in §47.

46.4 The driven two-level system

How is $R_x(\theta)$ actually produced? Take a qubit with splitting $\hbar\omega_0$ and drive it with an oscillating field at frequency ω, amplitude Ω and phase ϕ:

$$H = \tfrac{\hbar\omega_0}{2} Z + \hbar\Omega\cos(\omega t + \phi)\,X.$$

Go to the frame rotating with the drive, $|\psi\rangle \to e^{i\omega t Z/2}|\psi\rangle$, and drop the terms oscillating at 2ω (the rotating-wave approximation, valid for $\Omega \ll \omega$):

$$H' = \tfrac{\hbar\Delta}{2} Z + \tfrac{\hbar\Omega}{2}\big(\cos\phi\, X + \sin\phi\, Y\big), \qquad \Delta = \omega_0 - \omega.$$

On resonance ($\Delta = 0$) the qubit rotates about an equatorial axis at angle ϕ, at the Rabi frequency Ω: the population of $|1\rangle$ oscillates as $\sin^2(\Omega t/2)$. A pulse of area $\Omega t = \pi$ is an X gate, $\pi/2$ is a half-turn to the equator, and the *phase* of the drive chooses the axis, so Y rotations cost nothing extra. A Z rotation needs no pulse at all: shifting the phase of all later pulses by θ is the same as $R_z(\theta)$, the *virtual Z* that every control stack uses. Off resonance the effective axis tilts out of the equator and the oscillation amplitude drops to $\Omega^2/(\Omega^2 + \Delta^2)$, which is how a drive addresses one qubit and not its neighbour at a different frequency.

Numbers make this concrete. A transmon has $\omega_0/2\pi \approx 5\,\text{GHz}$ and $\Omega/2\pi \approx 10$–$50\,\text{MHz}$, so a π pulse takes 10–50 ns and contains 50–250 carrier oscillations. A hyperfine ion qubit has $\omega_0/2\pi \approx 3$–$13\,\text{GHz}$ driven by microwaves or Raman lasers with $\Omega/2\pi \approx 10$–$100\,\text{kHz}$, so a π pulse takes 5–50 μs. The gate time is set by the drive strength the hardware tolerates, not by the qubit frequency: faster drives need more bandwidth (previous subsection), more power, and leak.

46.5 Composite systems, entanglement, and two theorems

Two systems have state space $\mathcal{H}_A \otimes \mathcal{H}_B$. A product state $|a\rangle \otimes |b\rangle$ describes two independent objects; a state that cannot be written as a product is *entangled*. The Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ is the standard example: its reduced state $\rho_A = \mathrm{tr}_B |\Phi^+\rangle\langle\Phi^+| = I/2$ is maximally mixed, so each half alone is a fair coin while the two halves are perfectly correlated in *every* basis ($|\Phi^+\rangle = (|{+}{+}\rangle + |{-}{-}\rangle)/\sqrt{2}$ too). The bilinearity of $\otimes$ used in Part I to explain phase kickback is the same structure: $(\alpha|0\rangle + \beta|1\rangle) \otimes |\phi\rangle$ is a sum of two product terms, and a controlled gate acts on each term separately.

CHSH. That these correlations are not classical is a theorem with a two-line calculation. Let Alice measure A_0 or A_1 and Bob B_0 or B_1, each with outcomes ± 1. If outcomes were fixed in advance (realism) and independent of the other side's choice (locality), then for every run $A_0B_0 + A_0B_1 + A_1B_0 - A_1B_1 = A_0(B_0 + B_1) + A_1(B_0 - B_1) = \pm 2$, so

$$S = \langle A_0B_0\rangle + \langle A_0B_1\rangle + \langle A_1B_0\rangle - \langle A_1B_1\rangle \le 2.$$

For $|\Phi^+\rangle$ and spin measurements along directions $\hat{a}, \hat{b}$ in the x–z plane, $\langle(\hat{a}\cdot\vec{\sigma}) \otimes (\hat{b}\cdot\vec{\sigma})\rangle = \cos(a - b)$. Choosing $a_0 = 0$, $a_1 = 90°$, $b_0 = 45°$, $b_1 = 135°$ gives $S = 3\cos 45° - \cos 135° = 2\sqrt{2} \approx 2.83$, Tsirelson's bound. Experiments since 1982 find $S > 2$; the loophole-free ones of 2015 (NV centres at Delft, photons at Vienna and NIST) closed the detection and locality loopholes at once [50]. The $2\sqrt{2}$ is the maximum any quantum state can give, so a measured S is also a diagnostic: it is the figure of merit for two-qubit entanglement on a new processor.

No-cloning. Suppose a unitary U copies unknown states: $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ and $U|\phi\rangle|0\rangle = |\phi\rangle|\phi\rangle$. Unitaries preserve inner products, so $\langle\phi|\psi\rangle = \langle\phi|\psi\rangle^2$, which forces $\langle\phi|\psi\rangle \in \{0, 1\}$: only orthogonal states can be copied. Equivalently, by linearity, $U(\alpha|0\rangle + \beta|1\rangle)|0\rangle = \alpha|00\rangle + \beta|11\rangle$, an entangled state, not $(\alpha|0\rangle + \beta|1\rangle)^{\otimes 2}$, which has cross terms. Three consequences shape the rest of this part: a code cannot protect a qubit by keeping copies and voting, so it must spread the state over many qubits and measure only *parities* (§48); a magic state cannot be duplicated, so every T gate consumes a freshly made one (§48.5); and the only way to move a state is to *teleport* it, which is precisely how a T gate is applied to a logical qubit.

46.6 Open systems: what an error is

A qubit is never isolated. Coupling to an environment that we do not track turns the pure-state dynamics into dynamics of ρ. When the environment has no memory on the time scale of the qubit (Markovian noise) the evolution is the Lindblad equation

$$\frac{d\rho}{dt} = -\frac{i}{\hbar}[H,\rho] + \sum_k \Big(L_k \rho L_k^\dagger - \tfrac{1}{2}\{L_k^\dagger L_k, \rho\} \Big),$$

whose jump operators L_k encode the noise channels. Two of them describe nearly everything measured on a qubit:

- **Energy relaxation**, $L = \sqrt{\gamma_1}\,\sigma_-$ with $\sigma_- = |0\rangle\langle 1|$: the excited population decays as e^{-t/T_1}, $T_1 = 1/\gamma_1$. It is a loss of energy to the environment (a photon into a lossy dielectric, a spontaneously emitted photon, a quasiparticle). It is *not* a bit flip: it is non-unital, it pushes every state towards $|0\rangle$, and it also damps the coherence at rate $\gamma_1/2$.
- **Pure dephasing**, $L = \sqrt{\gamma_\varphi/2}\,Z$: the populations are untouched and the coherences decay as e^{-t/T_φ}. Physically the qubit frequency fluctuates, so the phase φ accumulated in a superposition becomes random.

Together, the coherence decays as e^{-t/T_2} with

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\varphi}, \qquad \text{so} \qquad T_2 \le 2T_1.$$

T_1 is measured by preparing $|1\rangle$ and watching it decay; T_2^* by a Ramsey sequence ($\pi/2$ pulse, wait t, $\pi/2$ pulse, read out: the fringes decay), and T_2 by a Hahn echo that inserts a π pulse at $t/2$ to refocus slow frequency drifts, so $T_2^* \le T_2$. All three are fitted envelope constants, not the lifetime of "the superposition".

Channels and Kraus operators. Integrated over a gate time, a noise process is a completely positive trace-preserving map $\rho \mapsto \sum_k E_k \rho E_k^\dagger$ with $\sum_k E_k^\dagger E_k = I$. Amplitude damping with decay probability γ has

$$E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \qquad E_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix};$$

pure dephasing is $\rho \mapsto (1-p)\rho + pZ\rho Z$; the depolarising channel is $\rho \mapsto (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$, which contracts the Bloch vector uniformly towards the origin. A *coherent* error is different in kind: it is a small unitary over-rotation $e^{-i\epsilon \hat{n}\cdot\vec{\sigma}/2}$ from a miscalibrated pulse, and it adds in amplitude over repeated gates (error $\propto \epsilon^2 m^2$ after m gates) where stochastic errors add in probability ($\propto pm$). Error correction and randomised benchmarking both tend to convert coherent errors into stochastic ones, which is one reason the two are budgeted differently.

Why a code can correct a continuum of errors. An arbitrary single-qubit error is $E = aI + bX + cY + dZ$ with complex coefficients, a continuum. A stabiliser code measures operators that commute or anticommute with each Pauli, so after the syndrome measurement the state is *projected* onto one of the four branches: no error, X, Y or Z, with probabilities $|a|^2, |b|^2, |c|^2, |d|^2$. The measurement discretises the error, and the correction undoes a Pauli. A small over-rotation $e^{-i\epsilon X/2} = \cos\frac{\epsilon}{2} I - i \sin\frac{\epsilon}{2} X$ therefore becomes "no error" with probability $\cos^2\frac{\epsilon}{2}$ and a clean X flip with probability $\sin^2\frac{\epsilon}{2}$. This is the content of the Knill–Laflamme conditions: it is enough to correct X, $Y = iXZ$ and Z on every qubit. It is also why leakage out of the $\{|0\rangle, |1\rangle\}$ subspace is a different and nastier beast: it is not spanned by the Paulis.

46.7 Measurement, fidelity, and the error budget

Projective and dispersive readout. Textbook measurement projects onto $|0\rangle$ or $|1\rangle$ and leaves the qubit there. Real readout couples the qubit to a meter — a resonator, a fluorescence cycle, a charge sensor — and reads the meter. A readout is *quantum non-demolition* (QND) if it leaves the eigenstate it reports in place, so it can be repeated; error correction needs this for the ancillas, which are read every cycle. The readout error ("assignment error") is the probability of reporting the wrong basis state; combined with preparation errors it is quoted as SPAM.

Fidelity and randomised benchmarking. The fidelity of a gate is the overlap between what it does and what it should do, averaged over inputs; the *error* is $1 - F$. Fidelities are measured by randomised benchmarking: run random sequences of m Clifford gates followed by their inverse, measure the survival probability of $|0\rangle$, and fit $A\,p^m + B$. Random Cliffords twirl any noise into a depolarising channel, so the decay constant gives the average error per Clifford

$$r = (1-p)\,\frac{d-1}{d}, \qquad r = \tfrac{1}{2}(1-p) \text{ for one qubit}, \quad r = \tfrac{3}{4}(1-p) \text{ for two.}$$

Interleaving a specific gate between the random ones isolates that gate's error. Cross-entropy benchmarking (XEB) does the same for whole random circuits and was the metric of the 2019 supremacy experiment. All of these report *average* errors; the tails, crosstalk and leakage that matter for a code are only partly visible in them.

The budget. A circuit with N qubits and depth D has about ND locations at which an error can occur. With independent errors of probability ϵ per location, the expected number of faults is $ND\epsilon$, and the circuit has a fair chance of running cleanly only if

$$N\,D\,\epsilon \lesssim 1.$$

This is the heuristic behind every number in §44. The Shor circuits of Parts IV–VIII have $N \approx 10^3$ logical qubits and $D \approx 10^9$–10^{10} logical cycles, so they need $\epsilon \approx 10^{-13}$–$10^{-15}$ *per logical qubit per cycle*. The best physical two-qubit gates are at 10^{-3}–10^{-4}. The twelve orders of magnitude in between are what error correction has to buy, and what §48 is about.

What a circuit asks of the physics

DiVincenzo's checklist, in the form the arithmetic needs it: (1) well-defined two-level systems that can be made in quantity; (2) reset to $|0\rangle$; (3) single-qubit rotations about two axes (a drive with a controllable phase); (4) one entangling two-qubit gate; (5) QND readout, fast enough to be used mid-circuit; and (6) coherence times long compared with the gate time, since the ratio T_2/t_{gate} bounds how many gates fit before the state is lost. The estimate of §44 then consumes three derived numbers: the error per operation p, the duration of one round of parity checks (the *cycle*), and the time from a measurement to the next conditional operation (the *reaction time*).

47 How the qubits are built

Every platform below realises the same abstract object — a two-level system with a Hamiltonian we can switch — in a different physical degree of freedom. The choice fixes the gate time, the coherence time, the readout, the connectivity and the dominant noise, and through them the three numbers of the estimate.

47.1 Superconducting circuits

Superconductivity. Below a critical temperature T_c (aluminium 1.2 K, niobium 9.3 K, NbTi about 10 K) some metals lose all resistance. The BCS theory of 1957 explains it: the exchange of lattice vibrations (phonons) gives electrons near the Fermi surface a weak attraction that overcomes their Coulomb repulsion and binds them in *Cooper pairs* of opposite momentum and spin. A pair is a boson, so the pairs condense into a single macroscopic quantum state with a well-defined phase φ. Breaking a pair costs an energy 2Δ with the gap $\Delta \approx 1.76\, k_B T_c$; for aluminium $\Delta/h \approx 44$ GHz, so a 5 GHz photon cannot break a pair and the condensate is protected from the qubit's own control signals. The condensate expels magnetic fields (the Meissner effect) except within the London penetration depth of 20–100 nm; type II superconductors such as NbTi tolerate strong fields by letting flux in as quantised vortices, which is why NbTi is used for the readout cabling and MRI magnets, while the qubits themselves use aluminium and niobium, whose isotropic (s-wave) pairing gives clean, reproducible junctions [50].

The Josephson junction. Two superconductors separated by a ~ 1 nm oxide barrier exchange Cooper pairs by tunnelling. Josephson's relations (1962) are

$$I = I_c \sin\varphi, \qquad V = \frac{\hbar}{2e}\frac{d\varphi}{dt},$$

with φ the phase difference across the barrier and I_c the critical current. Combining them, the junction behaves as a nonlinear inductor, $L_J(\varphi) = \Phi_0/(2\pi I_c \cos\varphi)$ with $\Phi_0 = h/2e$ the flux quantum, storing energy $-E_J \cos\varphi$ with $E_J = \Phi_0 I_c/2\pi$. This single nonlinear element is the whole trick: a plain LC circuit is a harmonic oscillator whose levels are equally spaced, so a drive at the $0 \to 1$ frequency also drives $1 \to 2$ and nothing singles out two levels; the cosine potential makes the spacing uneven and turns the circuit into an *artificial atom*.

The transmon. A junction shunted by a large capacitor C has the Hamiltonian

$$H = 4E_C\,(\hat{n} - n_g)^2 - E_J \cos\hat{\varphi}, \qquad E_C = \frac{e^2}{2C},$$

where $\hat{n}$ counts Cooper pairs on the island and $[\hat{\varphi}, \hat{n}] = i$, the same commutator as x and p. In the transmon regime $E_J/E_C \approx 50$–100 the phase is well localised near 0, so expand the cosine: $-E_J\cos\hat{\varphi} = -E_J + \frac{1}{2}E_J\hat{\varphi}^2 - \frac{1}{24}E_J\hat{\varphi}^4 + \dots$ The quadratic term with $4E_C\hat{n}^2$ is a harmonic oscillator of plasma frequency $\hbar\omega_p = \sqrt{8E_J E_C}$ (the LC frequency with $L_J = \Phi_0^2/4\pi^2 E_J$), and the quartic term, treated in first-order perturbation theory with $\hat{\varphi} = (2E_C/E_J)^{1/4}(a + a^\dagger)$, lowers each level m by $\frac{E_C}{2}(m^2 + m)$. Hence

$$\hbar\omega_{01} \approx \sqrt{8E_J E_C} - E_C, \qquad \alpha \equiv \omega_{12} - \omega_{01} \approx -E_C/\hbar,$$

and in the qubit subspace $H \approx -\frac{\hbar\omega_{01}}{2}Z$: the abstract qubit of §46.3, with the drive term $\hbar\Omega\cos(\omega t)X$ supplied by a microwave voltage capacitively coupled to the island, which couples to $\hat{n} \propto (a - a^\dagger)$, i.e. to Y or X depending on its phase. Typical values: $E_J/h \approx 15$ GHz, $E_C/h \approx 250$ MHz, so $\omega_{01}/2\pi \approx 5$ GHz and the anharmonicity $\alpha/2\pi \approx -250$ MHz. The large E_J/E_C exponentially suppresses the sensitivity to the offset charge n_g (charge noise), which is what made the transmon (2007) usable where the earlier charge qubit (1999) was not. The price is the small anharmonicity: a 20 ns pulse has a 22 MHz bandwidth, an order of magnitude below $|\alpha|$, so leakage to $|2\rangle$ is suppressed but not absent, and pulse shaping (DRAG, which adds a derivative component to the envelope) brings it to the 10^{-4} level [50]. Fluxonium replaces the shunt capacitor by a large inductance, gains a much larger anharmonicity and $T_1 > 1$ ms, at the cost of a sub-GHz qubit frequency and more complex control; Google acquired a fluxonium company in 2025.

Why 5 GHz and why 15 mK. A qubit at 5 GHz has $hf/k_B \approx 0.24$ K. At the 15 mK of a dilution refrigerator the thermal population of $|1\rangle$ is $e^{-hf/k_BT} \approx e^{-16} \approx 10^{-7}$, so the qubit initialises itself by waiting; at 100 mK it would be 10^{-5} and at 1 K, 9%. Lower frequencies would need colder stages; higher ones run into dielectric and quasiparticle losses and the 4–8 GHz band of available microwave components. The refrigerator is a ^{3}He–^{4}He dilution unit: below 0.87 K the mixture separates into a ^{3}He-rich phase and a dilute phase, and ^{3}He crossing the boundary absorbs heat, a process that keeps working to the millikelvin range because the solubility of ^{3}He in superfluid ^{4}He stays finite (6.6%) as $T \to 0$. The cooling power at the mixing chamber scales as $\dot{Q} \approx 84\,\dot{n}_3 T^2$ and is only 10–30 μW at 20 mK, against 1–2 W at the 4 K stage; a single 15 mK cryostat draws about 16 kW from the wall. Every microwave line into it is attenuated by 60 dB in stages so that only a few hundred thermal photons' worth of noise reaches the chip, and every readout line leaves through a quantum-limited parametric amplifier (JPA or TWPA) at the base stage, a HEMT at 4 K and a room-temperature amplifier [50]. This wiring, one to two lines per qubit, is the scaling constraint that multiplexing and cryo-CMOS are trying to lift.

Gates. Single-qubit gates are the Rabi drive of §46.4: an arbitrary waveform generator shapes an envelope, a mixer puts it on a 4–8 GHz local oscillator with in-phase and quadrature components that set the rotation axis, and the pulse lasts 10–40 ns. Two-qubit gates use a coupling element between neighbours. With a *tunable coupler* (Google since Sycamore, IBM since Heron in 2023) a flux pulse briefly switches on a ZZ interaction or brings the qubits into resonance for an iSWAP; the gate lasts 20–200 ns. The older cross-resonance gate drives the control qubit at the target's frequency and lasts longer. Readout is *dispersive*: a readout resonator at ω_r coupled to the qubit with strength g at detuning $\Delta = \omega_{01} - \omega_r$ has its frequency shifted by $\pm\chi$ with $\chi \approx g^2/\Delta$ depending on the qubit state (the Jaynes–Cummings Hamiltonian $H = \hbar\omega_r a^\dagger a + \frac{\hbar\omega_{01}}{2} Z + \hbar g(a^\dagger\sigma_- + a\sigma_+)$ in the dispersive limit $g \ll \Delta$). A microwave tone reflected off the resonator picks up a state-dependent phase, and the measurement is QND because it only touches Z. It takes 50–500 ns including the resonator ring-down; a Purcell filter stops the qubit from decaying through the readout line [50].

Noise. Energy relaxation is dominated by two-level-system (TLS) defects in the amorphous oxides and interfaces, by quasiparticles, and by the Purcell channel; dephasing by $1/f$ flux and charge noise; and the hardest problem for a code is *correlated* errors: a cosmic ray or a radioactive decay in the substrate creates a burst of phonons that breaks Cooper pairs across a millimetre of chip and flips many qubits at once, a few times a minute. Gap engineering (making the junction electrodes' gaps unequal so quasiparticles are trapped away from the junction), phonon traps and shielding have brought the rate down but not to zero [50].

Where it stood in 2026. T_1 of 100–400 μs for transmons, above 1 ms for fluxonium and 1.67 ms for a tantalum-on-silicon transmon at Princeton; single-qubit errors $\approx 10^{-4}$; two-qubit errors: 99.85% median CZ on IBM's 156-qubit Heron r3 and 99.67% on Google's 105-qubit Willow, with the best pairs below 10^{-3}; readout 99.4% in 2.4 μs on Heron-class devices; chips of 105–156 qubits with four nearest neighbours (Google since 2019, IBM's 120-qubit Nighthawk since 2025) and IBM's Loon test chip adding two long-range couplers per qubit for the qLDPC codes of §48. The surface-code cycle — four layers of CZ gates, a readout and a reset — is about 1 μs on Willow, which is the cycle time of §44 [50, 51].

47.2 Trapped ions

The trap. An ion is a charged atom, so an electric field holds it. A static field cannot confine a charge in all three directions (Earnshaw's theorem), so a Paul trap uses a radio-frequency quadrupole field at $\Omega_{\rm rf}/2\pi \approx 20$–$50\,\mathrm{MHz}$ whose time average is a harmonic pseudo-potential with secular frequencies $\nu/2\pi \approx 1$–$5\,\mathrm{MHz}$. Modern traps are microfabricated surface-electrode chips with the ions 50–$100\,\mu\mathrm{m}$ above the surface, segmented so that DC voltages can split, move and merge ion chains between zones: the QCCD architecture used by Quantinuum, Oxford Ionics (IonQ since 2025) and Universal Quantum. Ions in a chain repel each other and sit 2–$5\,\mu\mathrm{m}$ apart; their motion is a set of collective normal modes, each a quantum harmonic oscillator with phonon number n, and that shared motion is the bus for two-qubit gates [50].

The qubit. Two long-lived internal states of the ion. Hyperfine qubits use the $F = 0$ and $F = 1$, $m_F = 0$ ground states of an odd isotope with nuclear spin, split by $12.64\,\mathrm{GHz}$ in $^{171}\mathrm{Yb}^+$ and $3.2\,\mathrm{GHz}$ in $^{43}\mathrm{Ca}^+$; at a magnetic field where $d\nu/dB = 0$ (a *clock* transition) they are first-order insensitive to field noise and coherence times of $T_2 \approx 70\,\mathrm{s}$ have been measured, a coherence-to-gate ratio above 10^6. Optical qubits (AQT) use a metastable D level $\approx 400\,\mathrm{THz}$ above the ground state. Because ions of one isotope are identical, there is no fabrication spread of qubit frequencies; the variability sits in the control fields.

Cooling and the Lamb–Dicke regime. Gates need the motion cold. Doppler cooling illuminates the ion with light tuned just below a cycling transition of linewidth Γ: an ion moving towards the beam sees the light Doppler-shifted into resonance, absorbs a photon and its momentum $\hbar k$ against its motion, and re-emits in a random direction, so on average it slows. The limit $T_D = \hbar\Gamma/2k_B$ is about $0.5\,\mathrm{mK}$ for Yb^+ ($145\,\mu\mathrm{K}$ for the Rb atoms of §47.3). Resolved-sideband cooling then removes phonons one at a time by driving the $|g,n\rangle \to |e,n-1\rangle$ transition. The gates work in the Lamb–Dicke regime $\eta\sqrt{2\bar{n}+1} \ll 1$, $\eta = k\sqrt{\hbar/2m\nu} \approx 0.05$–$0.1$, where the ion's excursion is small compared with the laser wavelength and carrier and sideband transitions are cleanly resolved.

Gates. Single-qubit rotations are microwave or Raman π pulses of 1–$50\,\mu\mathrm{s}$. The Mølmer–Sørensen gate is worth deriving, because it is the cleanest example of an entangling Hamiltonian that is not simply "switched on". A laser on ion j couples its spin to the shared mode through the Lamb–Dicke factor: in the interaction picture $H_j = \hbar\Omega\,\sigma_x^{(j)} \cos\big(\omega_L t - \eta(a e^{-i\nu t} + a^\dagger e^{i\nu t})\big)$, and to first order in η a bichromatic drive at $\omega_0 \pm (\nu + \delta)$ leaves, after the rotating-wave approximation,

$$H_{\rm MS} = \hbar\eta\Omega\,\big(\sigma_x^{(1)} + \sigma_x^{(2)}\big)\big(a\,e^{i\delta t} + a^\dagger e^{-i\delta t}\big).$$

This is a force on the oscillator whose sign depends on the collective spin $S_x = \sigma_x^{(1)} + \sigma_x^{(2)}$. Its exact propagator is a displacement $D(\alpha(t)S_x)$ with $\alpha(t) = (\eta\Omega/\delta)(1 - e^{-i\delta t})$, a circle in phase space, times a phase $e^{i\Phi(t)S_x^2}$ with $\Phi(t) = (\eta\Omega/\delta)^2(\delta t - \sin\delta t)$. At $t = 2\pi/\delta$ the circle closes, the displacement vanishes whatever the initial phonon number, and what remains is $e^{i\Phi S_x^2}$ with $\Phi = 2\pi(\eta\Omega/\delta)^2$; since $S_x^2 = 2I + 2\sigma_x^{(1)}\sigma_x^{(2)}$, choosing $\eta\Omega/\delta = 1/\sqrt{8}$ gives $\Phi = \pi/4$ and the gate $e^{i\frac{\pi}{4}X\otimes X}$ of §46.3, up to a global phase. The entanglement is a *geometric* phase, the area enclosed by the loop; the motion is only borrowed. Residual spin–motion entanglement from an unclosed loop, mode-frequency drift and laser phase noise are the gate's error sources. Any pair in a chain can be gated this way — effective all-to-all connectivity — but the gate takes 10–$100\,\mu\mathrm{s}$ (longer for larger chains, up to a millisecond on IonQ's Forte), and shuttling between zones adds milliseconds [50]. Readout is state-dependent fluorescence: one qubit state scatters photons on a cycling transition, the other is dark;

collecting enough photons takes 100 μs to a few ms on cameras, 11 μs at 99.93% with a trap-integrated superconducting nanowire detector.

Where it stood in 2026. The best fidelities of any platform: single-qubit error 1.5×10^{-7} on a $^{43}Ca^{+}$ clock qubit (Oxford–Osaka, 2025), two-qubit 99.92% averaged over all pairs of Quantinuum's 98-qubit Helios (November 2025), $> 99.99\%$ on a two-ion Oxford Ionics chip, SPAM error 9×10^{-5} in $^{137}Ba^{+}$. Helios is the largest gate-based ion processor; 512-ion crystals are controlled for analog simulation [50, 52]. The limitation is the clock: with gates of tens of microseconds and readout of hundreds, one round of parity checks takes of the order of a millisecond rather than a microsecond, so the same logical algorithm runs a thousand times longer unless connectivity and transversal gates recover part of it (§47.7).

47.3 Neutral atoms

Traps and arrays. A neutral atom has no charge to hold, so it is held by light. A far-detuned laser induces a dipole, and the AC Stark shift $U = -\frac{1}{2}\alpha|E|^2$ with polarisability $\alpha > 0$ for red detuning pulls the atom to the focus: an *optical tweezer*. A spatial light modulator imprints a hologram that makes hundreds to thousands of tweezers at once, a magneto-optical trap loads them from a vapour at random (about 50% occupancy), and acousto-optic deflectors move atoms to fill a defect-free array, in 2D or 3D, 3–5 μm apart. The atoms are laser-cooled to a few μK; the chamber can be at room temperature, and the record arrays — 6,100 caesium atoms in 12,000 tweezers at Caltech in 2024, with 12.6 s hyperfine coherence and 99.98% single-qubit gates — were [50, 53].

The qubit and the Rydberg blockade. The qubit is a pair of hyperfine ground states (^{87}Rb, ^{133}Cs) or nuclear-spin states (^{171}Yb, ^{87}Sr), coherent for seconds. The interaction is borrowed temporarily: a laser excites an atom to a *Rydberg state* of principal quantum number $n \approx 50$–100, whose orbital radius grows as $n^2 a_0$ (about 0.5 μm at $n = 100$), whose lifetime grows as n^3 (tens to hundreds of μs), and whose van der Waals interaction with another Rydberg atom at distance r is $V = C_6/r^6$ with $C_6 \propto n^{11}$. When V exceeds the laser's Rabi frequency $\hbar\Omega$, two atoms within the blockade radius

$$R_b = \left(C_6/\hbar\Omega\right)^{1/6} \approx 5\text{–}10\,\mu\text{m}$$

cannot both be excited: the doubly excited state is shifted out of resonance. In the basis $\{|0\rangle, |1\rangle, |r\rangle\}$ per atom, with the laser coupling only $|1\rangle \leftrightarrow |r\rangle$,

$$H = \frac{\hbar\Omega}{2} \sum_{j=1,2} \left(|1\rangle\langle r|_j + |r\rangle\langle 1|_j\right) + V|rr\rangle\langle rr|, \qquad V \gg \hbar\Omega.$$

Start in $|00\rangle$: nothing couples, no phase. In $|01\rangle$ or $|10\rangle$ one atom Rabi-cycles $|1\rangle \to |r\rangle \to |1\rangle$ in a 2π pulse and picks up the phase -1 of a full spin rotation. In $|11\rangle$ the laser couples $|11\rangle$ only to the symmetric state $(|1r\rangle + |r1\rangle)/\sqrt{2}$ with enhanced Rabi frequency $\sqrt{2}\Omega$, because $|rr\rangle$ is blocked by V; the Levine–Pichler protocol (2019) uses two pulses with a phase jump between them so that both the singly excited and the doubly blocked branches return home with phases that differ by exactly π: the result is $\mathrm{diag}(1, e^{i\phi}, e^{i\phi}, e^{i(2\phi+\pi)})$, a CZ after single-qubit Z rotations. The blockade is the ZZ energy shift of §46.3 in disguise, and it needs no individual addressing: a global beam gates every pair placed within R_b in parallel, which is how 60 pairs were gated at once in 2023. The pulse lasts 0.2–2 μs; the fidelity reached 99.5% on 60 atoms in parallel in 2023 and 99.85% (raw) in a 2026 zoned architecture [50, 54].

Architecture. The decisive advantage is that the atoms move. The Harvard–MIT–QuEra experiment of December 2023 kept data atoms in a storage zone, carried them with tweezers into an entangling zone, and read ancillas in a separate zone: 280 atoms, up to 48 logical qubits, transversal logical CNOTs by moving whole code blocks next to each other [55]. Movement gives any connectivity at all, so codes that need long-range checks (qLDPC, §48) are natural here, and the cost is time: coherent transport, cooling and camera imaging take milliseconds even though the gate pulse is sub-microsecond, so a logical cycle is again of order a millisecond. Alkaline-earth atoms add a second gift: decay out of the qubit manifold can be detected (*erasure conversion*), and an error whose location is known is far cheaper to correct than one whose location must be inferred. Atom loss is the other specific error, replenished from a reservoir in the newer architectures.

47.4 Spins in silicon

An electron (or hole) confined in a gate-defined quantum dot, $\sim 50\,\mathrm{nm}$ across, has a spin $\frac{1}{2}$ whose degeneracy a static field B lifts by the Zeeman energy $E_Z = g\mu_B B$: at $B = 1\,\mathrm{T}$ with $g \approx 2$, $E_Z = 116\,\mu\mathrm{eV}$, a $28\,\mathrm{GHz}$ qubit. This is larger than $k_B T$ at $1\,\mathrm{K}$ ($86\,\mu\mathrm{eV}$), so spin qubits can run at 0.5–1.5 K where the cryostat has milliwatts of cooling power instead of microwatts, enough for control electronics next to the qubits. Single-qubit gates are electron spin resonance (an oscillating magnetic field) or electric-dipole spin resonance (an oscillating gate voltage converted to a magnetic drive by spin–orbit coupling or a micromagnet gradient); two-qubit gates lower the tunnel barrier between two dots so that the exchange interaction $J\vec{S}_1 \cdot \vec{S}_2$ of §46.3 acts for $\sim 75\,\mathrm{ns}$ — J is not a magnetic force but a consequence of the Pauli principle: with tunnelling t between the dots and charging energy U for two electrons on one dot, second-order perturbation theory of the Hubbard model gives $J \approx 4t^2/U$, non-zero only for the spin singlet, whose antisymmetric spin part allows the symmetric orbital part that tunnels, so a voltage on the barrier gate tunes t and switches J on and off by orders of magnitude; readout converts spin to charge (Pauli spin blockade: a second electron can tunnel onto the dot only if the two spins form a singlet) and senses the charge with a single-electron transistor or gate reflectometry. Isotopically purified ^{28}Si removes the nuclear spins that dephase the electron, and the dominant noise is then charge noise in the oxide. Fidelities of 99.5% for two-qubit and 99.8–99.9% for single-qubit gates have been reached on two-qubit devices, and $> 99\%$ on unit cells from a 300 mm foundry; the largest arrays are 18 qubits (HRL exchange-only, TU Delft, 2026). The appeal is density — gate pitches below 100 nm and CMOS manufacturing — and the open problems are variability, wiring and demonstrated entanglement beyond a handful of qubits [50].

47.5 Photons

A photon carries a qubit in its polarisation, its path, its arrival time bin, or its frequency, and flies at c through waveguides, so it decoheres only by being lost. Single-qubit gates are free: a half-wave plate or a Mach–Zehnder interferometer with a phase shifter is an arbitrary rotation. The difficulty is the opposite of the matter platforms': photons do not interact, so a two-qubit gate cannot be a Hamiltonian switched on. Linear optics with measurement (KLM, 2001) makes probabilistic entangling gates, and the practical architectures sidestep gates altogether: *measurement-based* computing prepares a large entangled cluster state and computes by measuring it in chosen bases, and PsiQuantum's *fusion-based* variant builds the cluster from small resource states fused by Bell measurements, each of which succeeds with probability 1/2 unless boosted. Everything then depends on components: single-photon sources (quantum dots at 4 K with $> 99\%$ indistinguishability and 57–70% end-to-end efficiency; parametric down-conversion pairs at room temperature but probabilistic) and detectors (superconducting nanowires at 1–3 K, $> 95\%$ efficiency, sub-nanosecond timing). A photon lost is an *erasure*, located by the detector that did not fire, and the fault-tolerance thresholds for loss

are a few percent per component. The continuous-variable branch (Xanadu) encodes qubits in field quadratures as GKP states and needs > 10 dB of squeezing, against a record of 15 dB measured. The largest demonstrations in 2026 were 12-mode processors and 99.22% fusions on PsiQuantum's test chips; the largest fault-tolerant plans are helium-cooled data-centre buildings, not chandeliers [50].

47.6 Three others

Bosonic cat qubits. A microwave cavity holds a superposition of two coherent states $|\pm\alpha\rangle$ stabilised by two-photon dissipation. Bit flips between $|\alpha\rangle$ and $|-\alpha\rangle$ are exponentially suppressed in the photon number (T_1 for bit flips of 22 s reported), while phase flips grow only linearly, so the noise is *biased* by $\eta = p_Z/p_X > 10^5$. A one-dimensional repetition code then suffices for the phase flips, and the Alice & Bob group's estimates are 349, 133 cat qubits for RSA-2048 in four days (2019) and 126, 133 for a 256-bit elliptic curve in nine hours (2023) [56, 57].

NV centres. A nitrogen-vacancy defect in diamond is a spin-1 electron system readable optically at room temperature, with nearby ^{13}C and ^{14}N nuclear spins as long-lived register qubits. It is the platform of the 2015 loophole-free Bell test and of quantum sensing; as a processor it has stayed below ten qubits for years.

Majorana (topological) qubits. Zero-energy modes at the ends of a one-dimensional topological superconductor would store a qubit non-locally in the fermion parity of a pair, and braiding them would implement Clifford gates with topological protection. Microsoft's 2025–2026 devices measure parities in gate-defined InAs–Al nanowires, but as of September 2026 no operational qubit metrics (coherence, gate fidelities) had been published, earlier Majorana claims had been retracted, and the T gate is not protected even in principle, so a code is still needed [50].

47.7 The platforms side by side, and what they feed into the estimate

	superconducting	trapped ions	neutral atoms	silicon spins	photons
qubit	two levels of a nonlinear LC circuit	hyperfine or optical states of an ion	hyperfine or nuclear states of an atom	electron/hole spin in a quantum dot	polarisation, path or time bin
entangling gate	tunable coupler ZZ/iSWAP	Mølmer–Sørensen via phonons	Rydberg blockade CZ	exchange $\sqrt{\text{SWAP}}$/CZ	fusion (probabilistic)
2-qubit time	20–200 ns	10–1000 μs	0.2–2 μs (+ms transport)	40–75 ns	ns (circuit traversal)
best 2-qubit fidelity, commercial	99.85% (IBM Heron r3), 99.67% (Willow)	99.92% (Helios, 98 q.)	99.5–99.85%	99.5% (2 q.)	99.22% fusion
T_1 / coherence	0.1–1.7 ms	seconds to hours	10 s–1 h trap; 12.6 s T_2	> 1 s; ms T_2	loss-limited
readout	0.1–2.4 μs dispersive	11 μs–ms fluorescence	ms imaging (2.4 μs record)	0.2–few μs reflectometry	< 1 ns detector
operating T	15 mK	ions < 1 mK, trap RT–4 K	atoms μK, chamber RT	0.1–1.5 K	RT circuits, 2–4 K sources/detectors
connectivity	4 neighbours (+2 long range on Loon)	all-to-all in a zone	any, by moving atoms	neighbours (+shuttling)	any, by routing
largest system, 2026	156 (IBM), 105 (Google)	98 (Quantinuum)	256 gated; 6,100 trapped	18	12 modes

Figures as reported by vendors and papers to September 2026, compiled in [50] (Tables 17–18, 26); laboratory records on two qubits are better in every column.

The estimate of §44 reads this table through three numbers:

- p is the error per *location* of the syndrome circuit — gates, idles, measurements and resets alike — at scale, with crosstalk, not the best pair in a lab. The model's $p = 10^{-3}$ sits at the median two-qubit error of the 2026 superconducting chips and somewhat above the ion and atom figures. Halving p to 5×10^{-4} drops the distance from 25 to 21 and the qubits per logical qubit from 1,352 to 968 (§44); at 10^{-4} it is 392. The sensitivity is exponential in the exponent and only polynomial in the prefactor.
- **The cycle** is one round of parity checks: four entangling layers, a readout and a reset. Superconducting chips do it in $\approx 1\,\mu$s; ions and atoms, with microsecond gates but 10^2–$10^3\,\mu$s readouts and transport, in about a millisecond. Runtime is linear in the cycle, so the 15 minutes for ECDLP-256 and 5 days for RSA-2048 become ten days and thirteen years at a 1 ms cycle, unless the architecture compensates. That is why the slow platforms lean on all-to-all connectivity, transversal gates and higher-rate codes: IonQ's 2026 estimate of 26 days for a 256-bit curve on 20,000 ions [41] buys its time back with those, and still reports a 63% success probability.
- **The reaction time** is decoder latency plus control: the 10 μs of the model is the time between an ancilla measurement and the next operation that depends on it, which matters because every T gate ends in a measurement whose outcome decides a Clifford correction (§48.5). It is the only one of the three numbers set by classical electronics rather than physics.

Connectivity enters through the code family and the routing overhead rather than through a number,

and leakage, loss and correlated errors enter through whatever the chosen code does not model: the estimates assume independent Pauli errors, and the hardware does not quite deliver them.

47.8 The challenges, platform by platform

What stands between each platform and the thousand $d = 25$ patches of the estimate, in the order the field itself lists them [50] (Table 17):

- **Superconducting.** Fidelity uniformity across a chip (the median, not the best pair, sets p); crosstalk and frequency crowding growing with qubit count; correlated errors from radiation; leakage; the wiring, cooling budget and energy of 10^6 control lines — hence cryo-CMOS, superconducting (SFQ) logic, multiplexed readout and chip-to-chip microwave links; and two-qubit errors improving more slowly than single-qubit ones. The breakthroughs were the transmon (2007), tunable couplers (2019), the Willow below-threshold memory (2024) and gap engineering against quasiparticles (2024).
- **Trapped ions.** Gate and readout speed; control of many motional modes as chains grow; anomalous heating of the motion from the trap surface; laser power, phase noise and beam-pointing stability; shuttling overhead. The breakthroughs were the QCCD architecture, microwave-driven gates (no lasers), 10^{-7} single-qubit errors (2025) and the 98-ion Helios with all-pairs 99.92% (2025).
- **Neutral atoms.** Loading, rearrangement and imaging times that dominate the duty cycle; atom loss and leakage; optical power and uniformity across thousands of tweezers; QND mid-circuit readout without disturbing neighbours. The breakthroughs were parallel high-fidelity Rydberg gates (2023), zoned logical processing (2023), erasure conversion in ytterbium (2023–2025) and 6,100-atom arrays (2024).
- **Silicon spins.** Charge noise and device-to-device variability, which forces per-qubit calibration; valley splitting in silicon; demonstrated entanglement only on a handful of dots; fabrication cycles of months. The breakthroughs were $> 99\%$ two-qubit gates (2022), operation above 1 K (2020) and foundry-made 300 mm unit cells (2025).
- **Photons.** Source efficiency and indistinguishability; loss in every component; probabilistic entanglement that multiplies the resource states needed; large cluster states; cryogenic detector integration. The breakthroughs were bright quantum-dot sources, fusion-based architectures and $> 99\%$ fusion on test chips (2025–2026).

Common to all: no platform had, by September 2026, run a logical circuit with real-time decoding and without post-selection at a distance above 7, and every resource estimate assumes a noise model (independent, Pauli, uniform at 10^{-3}) that the hardware does not yet deliver.

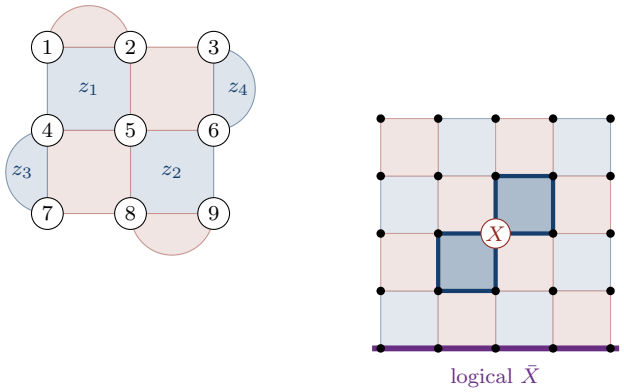
48 Errors, codes, and the price of a logical qubit

§44 used the surface code as a black box with one formula. This section opens it: how parity checks locate errors without touching the data, why the logical error falls exponentially with the distance below a threshold, what the decoder must do in real time, and how logical gates — above all the T gate — are performed. The last subsection returns the favour by deriving the "one CCZ every 25 μs" that drives every runtime in Part VIII.

48.1 Stabilisers: measuring parity, not data

A stabiliser code on n qubits is defined by $n-k$ commuting Pauli operators S_j; the code space is their joint $+1$ eigenspace and holds k logical qubits. Each S_j is measured by an ancilla: for $S = Z_1Z_2$, two CNOTs from the data qubits onto an ancilla in $|0\rangle$ and a readout give the parity of the two bits without revealing either bit, since $|00\rangle$ and $|11\rangle$ both leave the ancilla at 0. A Pauli error E that anticommutes with S_j flips its outcome; the pattern of flipped outcomes is the *syndrome*. Because the measurement projects (§46.6), a continuum of errors becomes a Pauli with a syndrome.

The rotated surface code of distance d places d^2 data qubits on a grid with d^2-1 ancillas in the faces: each face measures the parity of its four corners, in the Z basis on half the faces (catching X errors) and the X basis on the other half (catching Z errors), with weight-two checks along the edges. It is $[[d^2, 1, d]]$ with $2d^2-1$ physical qubits.



Left: $d = 3$, nine data qubits; blue Z-checks catch X errors, red X-checks catch Z errors. Right: $d = 5$ (data qubits at the vertices, checks on the faces): one X error lights the two Z-faces beside it; the logical $\bar{X}$ is a string of d Paulis across the patch.

Written as a parity-check matrix for the $d = 3$ patch, the Z-checks are

$$H_Z = \begin{array}{c|ccccccccc} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \hline z_1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ z_2 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ z_3 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ z_4 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{array} \qquad s = H_Z\, e \pmod 2.$$

An X error on qubit 5 lights z_1 and z_2, and qubit 5 is the only one in both; an error on qubit 7 lights z_3 alone. Errors on qubits 1 and 2 give the same syndrome, but X_1X_2 commutes with every check and acts trivially on the code space — the two corrections are equivalent modulo a stabiliser (*degeneracy*), so either works. An undetectable error must commute with all checks and act non-trivially: the shortest is a string of d Paulis across the patch, which is the logical operator itself. Hence the distance d, and the code corrects any $\lfloor (d-1)/2 \rfloor$ errors.

48.2 The stabiliser formalism, with the mathematics

The group. The Pauli group on n qubits, $\mathcal{P}_n$, is the set of tensor products of I, X, Y, Z with phases $\pm 1, \pm i$. Two of its elements either commute or anticommute, and the commutation is computed by a symplectic inner product: write $P = X^{\mathbf{a}}Z^{\mathbf{b}}$ with bit vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_2^n$; then P and $P' = X^{\mathbf{a}'}Z^{\mathbf{b}'}$ commute iff $\mathbf{a}\cdot\mathbf{b}' + \mathbf{b}\cdot\mathbf{a}' = 0 \pmod 2$. This turns every question about a code into linear algebra over $\mathbb{F}_2$, which is why the parity-check matrix H_Z above told the whole story.

The code. A stabiliser group $\mathcal{S} = \langle S_1, \dots, S_{n-k} \rangle$ is an abelian subgroup of $\mathcal{P}_n$ not containing $-I$. The code space $\mathcal{C} = \{|\psi\rangle : S|\psi\rangle = |\psi\rangle \ \forall S \in \mathcal{S}\}$ has dimension 2^k: each independent generator halves the space. The logical operators are the Paulis that commute with all of $\mathcal{S}$ but are not in it, the *normaliser* $N(\mathcal{S})\backslash\mathcal{S}$; they come in k anticommuting pairs $(\bar{X}_i, \bar{Z}_i)$, and the distance d is the minimum weight among them. For the $d = 3$ patch of the figure, $\bar{Z} = Z_1Z_2Z_3$ (a row) and $\bar{X} = X_1X_4X_7$ (a column) commute with every check, anticommute with each other, and have weight 3; multiplying $\bar{Z}$ by the check $z_1 = Z_1Z_2Z_4Z_5$ gives $Z_3Z_4Z_5$, another representative of the same logical operator, which is the degeneracy seen earlier from the other side.

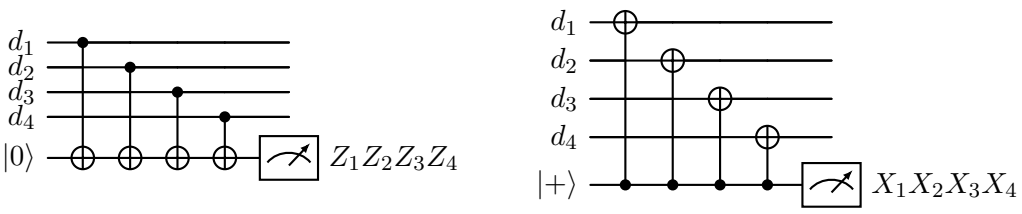
Why errors are detected without disturbing the data. Let $E \in \mathcal{P}_n$. For $|\psi\rangle \in \mathcal{C}$, $S_j(E|\psi\rangle) = \pm ES_j|\psi\rangle = \pm E|\psi\rangle$, with the sign -1 iff E anticommutes with S_j. So $E|\psi\rangle$ is still a joint eigenvector of all the checks, with a sign pattern (the syndrome) that depends on E and not on $|\psi\rangle$. Measuring the checks therefore learns E's syndrome and nothing about the encoded state: the data is untouched because the code space is a common eigenspace and the measurement commutes with every logical operator. This is the precise content of "measure parities, never the data".

Knill–Laflamme. A set of errors $\{E_a\}$ is correctable iff, with P the projector onto $\mathcal{C}$,

$$P\,E_a^\dagger E_b\,P = c_{ab}\,P \qquad \text{for all } a, b,$$

for some matrix of scalars c_{ab}: no error product may map one codeword onto a different codeword, nor distinguish codewords. For a stabiliser code this holds iff every product $E_a^\dagger E_b$ is either in $\mathcal{S}$ (then $c_{ab} = \pm 1$, the errors are degenerate) or outside the normaliser (then it anticommutes with some check and $c_{ab} = 0$). All Paulis of weight $\le t$ satisfy it iff $2t < d$, and by linearity so does every operator in their span, which is how the continuum of §46.6 is covered.

The extraction circuit. One weight-four Z-check and one X-check of the surface code, as actually run every cycle:



The left ancilla ends in $|s\rangle$ with s the parity of the four data bits; the right one, read in the X basis, gives the parity of their X values. The order of the four CNOTs is not arbitrary: a CNOT propagates X from control to target and Z from target to control, so an ancilla error in the middle of the sequence lands on two data qubits (a *hook* error), and the schedule is chosen so that such pairs lie along the direction in which the code is tolerant. One cycle is these four layers of CNOTs on every face at once, one measurement and one reset: on Willow, $\approx 1\,\mu$s. Because the ancilla measurement itself errs with probability $\sim p$, the syndrome is accepted only after it has been seen consistently over d rounds, which is where the "d rounds per logical operation" of §48.5 comes from.

48.3 Threshold and the exponential

A chain of errors is corrected unless the decoder pairs its ends the wrong way, which it does only when the chain is at least $d/2$ long. With independent errors at rate p, a specific chain of length ℓ

has probability $\sim p^\ell$, and the number of chains grows like A^ℓ for a constant A set by the lattice, so the logical error per round is

$$P_L \;\sim\; \sum_{\ell \geq (d+1)/2} (Ap)^\ell \;\approx\; C\Big(\frac{p}{p_{\text{th}}}\Big)^{(d+1)/2}, \qquad p_{\text{th}} \sim 1/A.$$

Below threshold every two steps of d divide P_L by p_{th}/p; at threshold the distance buys nothing and the physical overhead diverges; above it, larger codes are worse. Under circuit-level depolarising noise with a matching decoder the surface-code threshold is about 1%; the model of §44 uses $C = 0.1$ and $p_{\text{th}} = 1.22\%$, so that $d = 25$ is the first distance below 10^{-15} at $p = 10^{-3}$ [43]; the older Fowler fit is $C = 0.03$ with the same exponent [58]. This is also the origin of the engineering rule that p must be about ten times below threshold: at $p/p_{\text{th}} = 1/10$ a factor 10 per step gives $d \approx 27$ for 10^{-15}; at 1/3 it would need $d \approx 61$ and 7,700 qubits per logical qubit; at 1/2, $d \approx 97$. The qubit count is quadratic in d and d is logarithmic in the target, so the prize is in p.

Threshold theorem. That an arbitrarily long computation can be made reliable at all is the threshold theorem (Aharonov–Ben-Or, Kitaev, Knill–Laflamme–Zurek, 1996–1999): if the error per location is below a constant p_{th}, with noise that is local and not too strongly correlated, the logical error can be driven to any target with overhead polylogarithmic in the circuit size. Early proofs with concatenated codes gave $p_{\text{th}} \sim 10^{-6}$; the surface code raised it to $\sim 10^{-2}$, which is what made superconducting qubits viable. The assumptions matter: a cosmic-ray burst or a leaked qubit is neither local nor Pauli, which is why §47.1 spent a paragraph on them.

Measured. Google's Willow experiment (December 2024) ran surface-code memories of $d = 3, 5, 7$ on 101 qubits and found the logical error per cycle halving at each step ($\Lambda = P_L(d)/P_L(d+2) \approx 2.1$), reaching 0.14% per cycle at $d = 7$ with a logical lifetime longer than the best physical qubit's: the first demonstration *below threshold* [51]. A ratio of 2.1 is far from the 12 the model assumes at $p = 10^{-3}$; it corresponds to $p/p_{\text{th}} \approx 1/2$ in the fit above, which is a measure of how far 2024 hardware was from the hypothesis of the estimates.

48.4 Decoding, in real time

The syndrome is measured every cycle because the measurements themselves err, so the decoder sees a three-dimensional history (two space dimensions, one time) of flipped checks and must pair them up: minimum-weight perfect matching finds the most probable pairing in time polynomial in d (empirically $O(d^3)$–$O(d^5)$ per round), union-find does it faster with a small loss in accuracy, and belief propagation with post-processing serves the qLDPC codes. The result is not applied as physical gates: it updates a classical *Pauli frame*, and the frame is folded into the interpretation of later measurements. Two constraints follow. Throughput: the decoder must consume syndromes at least as fast as the 1 μs cycle produces them for $\sim 10^3$ patches, or a backlog grows without bound. Latency: the frame must be resolved before any operation whose basis depends on it, which happens at every non-Clifford gate. Google's Willow decoding ran at 63 μs latency; FPGA decoders reached sub-microsecond per round on $d \leq 12$ codes in 2025–2026; DeepMind's AlphaQubit 2 decodes faster than 1 μs per cycle up to $d = 11$ and is projected to fail beyond $d \approx 23$ [50]. The 10 μs reaction time of the model is in this range; the open problem is real-time decoding at $d = 25$ for a thousand patches at once, not decoding as such.

48.5 Logical gates, and the one that costs

A code protects a memory. Computing on it requires logical gates that do not spread errors: a single fault anywhere in the gadget must cause at most one error per code block, so that the next round of correction removes it. Three constructions do this.

Transversal gates. Apply the same physical gate to every qubit of the block (or qubit-by-qubit between two blocks). A fault then touches one qubit per block by construction. For any CSS code — a code whose checks are pure X or pure Z, like the surface code — the transversal CNOT is a logical CNOT: n physical CNOTs between blocks A and B map $X_A^{\mathbf{a}} \mapsto X_A^{\mathbf{a}} X_B^{\mathbf{a}}$ and $Z_B^{\mathbf{b}} \mapsto Z_A^{\mathbf{b}} Z_B^{\mathbf{b}}$, so each X-check of A becomes itself times the *same* X-check of B, which is in the stabiliser; the code space is preserved, and $\bar{X}_A \mapsto \bar{X}_A \bar{X}_B$, $\bar{Z}_B \mapsto \bar{Z}_A \bar{Z}_B$ is exactly the action of CNOT on logical Paulis. The same calculation shows why the surface code's transversal H is *not* a logical H (it swaps X- and Z-checks, whose layouts differ by a rotation, hence the patch-rotation trick) and why $T^{\otimes n}$ is never logical for it. The **Eastin–Knill theorem** (2009) makes this general: no code that corrects at least one error has a universal set of transversal gates, because a continuous group of transversal unitaries would commute with the error-detection projector and let an undetectable error rotate the logical state continuously. Transversal gates need every qubit of one block to meet its partner in the other, which ions and movable atoms provide and a fixed chip does not.

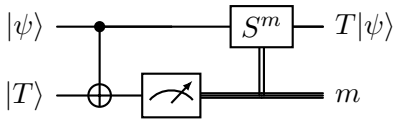
Lattice surgery. On a planar chip the logical CNOT is built from joint measurements. Two patches side by side have Z-type boundaries facing each other. Measuring the weight-four Z-checks that straddle the seam *merges* them into one longer patch; the product of those new checks is $\bar{Z}_A \bar{Z}_B$, so the merge measures the logical operator $Z_A Z_B$ with outcome ± 1 while learning nothing else, since every seam check commutes with every logical operator of the merged code except that product. Stopping the seam checks *splits* the patches again. Each merge and split is repeated for d rounds so that measurement errors on the new checks are corrected, hence the $O(d)$ time per operation and the $O(d^3)$ space-time volume. A CNOT between control C and target T then uses an ancilla patch A prepared in $|+\rangle_L$:

$$\text{measure } Z_C Z_A \ (m_1), \qquad \text{measure } X_A X_T \ (m_2), \qquad \text{measure } Z_A \ (m_3),$$

and apply $X_T^{m_1 \oplus m_3}$ and $Z_C^{m_2}$ as Pauli-frame updates — a direct calculation on $|\psi\rangle_C |+\rangle_A |\phi\rangle_T$ (checked numerically for all eight outcome patterns) shows the result is $\mathrm{CNOT}_{C\to T} |\psi\rangle|\phi\rangle$. Three joint measurements of d rounds each, so a logical CNOT costs about $3d$ cycles, $75\,\mu$s at $d = 25$, and needs the ancilla patch plus routing space between C and T: that routing is the "bus" and the reason the compute region of §44 reserves patches it never stores data in. The **Pauli frame** is the other half of the trick: X_L and Z_L corrections are never applied physically, only recorded and folded into the interpretation of later measurements, so Clifford gates (H, S, CNOT) can even be commuted to the end of the circuit and absorbed into the measurement bases, leaving the non-Clifford gates as the only operations that cost time.

What is left. The surface code thus has cheap Paulis (free), CNOT ($3d$ rounds), H and S (patch deformation and a cheap injected $|+i\rangle$ state). What it lacks is T.

Magic states. The way round is gate teleportation. Prepare the *magic state* $|T\rangle = T|+\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$, then



applies T to $|\psi\rangle$ using only a CNOT, a measurement and an S gate conditioned on the outcome m — all Clifford, all cheap — consuming one $|T\rangle$. (If $m = 1$ the output is $S\,T^\dagger|\psi\rangle$ up to phase, and S fixes it; the correction depends on a measurement, so the decoder must have resolved the frame first — this is the reaction-time dependence of §47.7.) The difficulty is moved into making $|T\rangle$ clean. A noisy $|T\rangle$, prepared by a physical T gate on one qubit and injected into a patch, has error $\epsilon \sim p$. *Distillation* (Bravyi–Kitaev 2005) uses a code with a transversal T: the $[[15, 1, 3]]$ quantum Reed–Muller code has $T^{\otimes 15} = \bar{T}^\dagger$ up to a Clifford. Encode $|+\rangle_L$ in it, apply T on all fifteen qubits using fifteen noisy $|T\rangle$ states by the teleportation circuit above, measure the code's checks and keep the output only if no error was detected. A single error among the fifteen is always caught; the output is wrong only if three or more errors conspire into a logical operator, and counting the $\binom{15}{3}$ patterns that do gives

$$\epsilon_{\text{out}} \approx 35\,\epsilon^3, \qquad \text{so} \qquad 10^{-3} \;\to\; 3.5 \times 10^{-8} \;\to\; 1.5 \times 10^{-21},$$

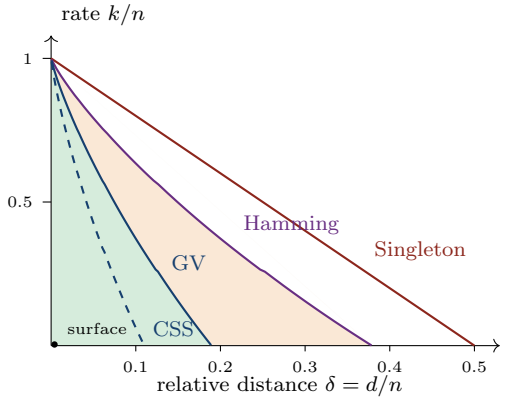
at the cost of fifteen inputs per output and a success probability $\approx 1 - 15\epsilon$ per round. Two rounds therefore reach any target the estimates need, and a factory is a few $[[15, 1, 3]]$ blocks laid out as surface-code patches, which is where the $O(d^2 \log 1/\epsilon)$ footprint of magic-state factories comes from. Gidney, Shutty and Jones's *cultivation* (2024) instead grows a $|T\rangle$ in a $d = 3$ colour code, checks it, grows to $d = 5$, checks again, and converts it to a surface-code patch, post-selecting on every check: about $30,000$ qubit-rounds per $|T\rangle$ at 10^{-7} error, an order of magnitude cheaper than distillation, which is the figure `physical.py` uses [59]. Cultivation's acceptance rate falls with the distance of the check, so beyond about 10^9 T states per run it is used to seed distillation rather than replace it [50].

The 25 microseconds. A Toffoli is a CCZ between Hadamards, and a CCZ is made from four $|T\rangle$ states with the temporary-AND construction of Part V (§20), or eight in the direct form the factory uses. The model's factory is a block of 3×4 patches of distance 25, $12 \times 1,352 = 16,224$ qubits. Cultivating eight $|T\rangle$ states costs $8 \times 30,000 = 240,000$ qubit-rounds, that is $240,000/16,224 \approx 15$ rounds of the whole factory; assembling them into a CCZ state takes six layers of lattice surgery of $2d/3$ rounds each, 100 rounds; the total 115 is padded to 150 for slack. Six factories in parallel therefore deliver one CCZ every $150/6 = 25$ rounds, and a round is the $1\,\mu$s cycle: one Toffoli every $25\,\mu$s, which is §44's "counting time" and the origin of every runtime in Part VIII. The compute region of 7×18 patches, $170,352$ qubits, holds the six factories and the lattice-surgery workspace that consumes their output. Since 25 rounds is also one logical cycle at $d = 25$, the machine is factory-bound and logic-bound in equal measure; a seventh factory would not help without more workspace.

48.6 Beyond the surface code

The surface code pays $2d^2$ qubits per logical qubit because every check is local on a plane: the Bravyi–Poulin–Terhal bound $kd^2 = O(n)$ makes that the best any two-dimensional local code can do. Quantum LDPC codes keep the checks sparse but let them reach across the chip, and then k and d can both grow linearly with n (Panteleev–Kalachev, 2021). IBM's *gross* code $[[144, 12, 12]]$ stores twelve logical qubits in 144 data qubits with weight-6 checks, needing degree-6 connectivity, which is what the Loon couplers provide; Iceberg Quantum's Pinnacle architecture (2026) stacks such

codes into an RSA-2048 machine of under 10^5 physical qubits at the price of a month's runtime and degree-8 to 10 connectivity, which Google argued is not feasible on superconducting chips [60, 50]. Google's answer stays planar: *yoked* surface codes (2023) halve the qubits per logical qubit by adding a few parity checks across patches, used for RSA's idle exponent register in §44 at 430 qubits per logical qubit, and a "denser planar surface code" (May 2026) claims a further factor 4.5 [50]. On atoms and ions, where qubits move, the connectivity is free and the qLDPC codes are the natural choice, which is why the 2026 atom-based estimates (§49) use far fewer qubits and far more time.



Which large codes can exist, in rate k/n against relative distance $\delta = d/n$. Gilbert–Varshamov: codes exist with $k/n \geq 1 - \delta \log_2 3 - H_2(\delta)$ (green), CSS codes with $1 - 2H_2(\delta)$ (dashed). Singleton: $k/n \leq 1 - 2\delta$ for every code. Hamming: $k/n \leq 1 - \frac{\delta}{2} \log_2 3 - H_2(\frac{\delta}{2})$ is proven for non-degenerate codes and for $\delta \leq 0.235$ or $d = 3$ in general; whether degenerate codes can beat it elsewhere is open, so the peach band is "unknown" above GV and "impossible" only above Singleton. The surface code sits at the origin: vanishing rate and vanishing relative distance, the price of local checks.

48.7 Logical qubits, as demonstrated

when	who	code	physical / logical	what was shown
2021	Maryland	Bacon–Shor-13, ions	15 / 1	first fault-tolerant logical qubit; logical fidelity 99.7%
2022	Quantinuum	Steane colour, ions	10 / 1	logical SPAM better than physical (break-even)
2022	Google	surface $d = 5$, SC	49 / 1	logical error *not* better than $d = 3$: above threshold
Dec 2023	Harvard–QuEra	colour + qLDPC, atoms	280 / 48	logical processor, transversal gates, CCZ; post-selected
Dec 2024	Google	surface $d = 7$, SC	101 / 1	**below threshold**: $\Lambda \approx 2.1$, 0.14% per cycle
2024	Quantinuum–Microsoft	tesseract / Steane, ions	56 / 12	12-qubit logical GHZ states, post-selected
2025	QuEra	colour + qLDPC, atoms	200 of 448	logical CZ by surgery and teleportation, CCZ, rotations; atom-loss post-selection
2026	Quantinuum	Steane, Helios ions	98 / 48	H, CZ, T on 48 logical qubits; teleported T states; small QAOA and HHL
2026	China (Hefei et al.)	surface $d = 7$, SC	125 / 2	two $d = 7$ patches, transversal teleportation

From [50] (Tables 9–10). None is a full fault-tolerant stack: as of September 2026 no experiment had run a distilled or cultivated T state through a below-threshold memory with real-time decoding and no post-selection, and the largest code distance on hardware was 7 against the 25 the estimates need.

49 Where things stand

49.1 Milestones

1994	Shor's factoring and discrete-logarithm algorithms.
1995	Cirac–Zoller trapped-ion gate proposal; first BEC.
1996–97	Shor and Steane codes; Gottesman's stabiliser formalism; Kitaev's toric code; threshold theorem.
1999	Nakamura: first coherent superconducting (charge) qubit, ~ 1 ns coherence.
2001	Shor's algorithm factors 15 on a 7-qubit NMR machine (compiled). KLM linear optics.
2007	The transmon.
2010	First Rydberg-blockade CNOT between neutral atoms.
2012	Fowler et al.: surface-code architecture with $\sim 1\%$ threshold; 21 factored photonically (compiled).
2016	Monz et al.: scalable Shor circuit factors 15 with trapped ions.
2015	Loophole-free Bell tests (NV centres at Delft; photons at Vienna and NIST).
2019	Sycamore random-circuit sampling ("supremacy"); Gidney–Ekerå: RSA-2048 in 8 hours with 20 million qubits.
2021	Maryland: first fault-tolerant logical qubit (ions); Gouzien–Sangouard memory-based estimate; good qLDPC codes (Panteleev–Kalachev).
2023	Harvard–QuEra 48 logical qubits; Evered: 99.5% Rydberg CZ on 60 atoms; Litinski: ECDLP-256 in 50 million Toffolis.
2024	Google Willow below threshold; Caltech 6,100-atom array; Gidney–Shutty–Jones cultivation; IBM gross code.
2025	Gidney: RSA-2048 under a million qubits in under a week; Quantinuum Helios 98 ions at 99.92%; Oxford 1.5×10^{-7} single-qubit error; QuEra fault-tolerant architecture paper.
2026	Pinnacle: RSA-2048 under 10^5 qubits (qLDPC); IonQ: ECDLP-256 in 26 days on 20,000 ions; Quantinuum 48 logical qubits with T; Nighthawk and Loon; the ECDLP circuits of Part VII.

49.2 What has actually been factored

Every Shor demonstration to date has been compiled for its answer. The 2001 NMR and 2016 ion experiments factored 15; the 2012 photonic experiment factored 21 with qubit recycling; and 21 is still the largest integer factored by a Shor-type circuit on hardware in 2026. The reason is in Part V: a circuit for 15 collapses to 21 entangling gates because $a^x \bmod 15$ has a trivial structure, while a general circuit for 21 needs 2,405 entangling gates (Gidney, 2025), beyond what any uncorrected processor runs coherently [50]. Larger "factorings" — 35 on IBM hardware, $56,153$, $1,099,551,473,989$ with variational methods, 2048-bit integers with specially close factors on an annealer, the 2026 "Q-Day prize" for a 15-bit elliptic-curve key whose 98,000-gate circuit on 65 qubits succeeded with probability 10^{-215} and was filtered by a classical checker — are not Shor's algorithm and do not scale; the 2022 claim of RSA-2048 with 372 qubits (Schnorr's lattice method plus QAOA) was shown not to scale past 70 bits even with perfect qubits. Classically, the record is RSA-250, 829 bits, with the number

field sieve in 2020 at about 2,700 core-years; a 862-bit RSA-260 was claimed in September 2026 without an audited computation [50].

49.3 The resource-estimate landscape

estimate	target	physical qubits	time	assumptions and caveats
Gidney–Ekerå 2019 [19]	RSA-2048	20 M	8 h	surface code, $p = 10^{-3}$, 1 μs cycle
Gouzien–Sangouard 2021	RSA-2048	13,436 + memory	177 d	2 logical qubits plus a 28-million-mode quantum memory
Alice & Bob 2019 / 2023 [56, 57]	RSA-2048 / ECC-256	349 k / 126 k cat qubits	4 d / 9 h	repetition code on biased-noise cats
Gidney 2025 [43]	RSA-2048	$<$ 1 M	$<$ 1 week	residue arithmetic, yoked codes, cultivation; the model of Part VIII
Pinnacle 2026 [60]	RSA-2048	94 k / 135 k / 381 k	1 mo / 1 wk / 1 d	qLDPC, degree-8 to 10 connectivity, SC
IonQ 2026 [41]	ECC-256	20,000	26 d	walking-cat codes, 99.99% gates, 63% success
Oratomic 2026	RSA-2048 / ECC-256	10^4–10^5	months–years	qLDPC on atoms; slow cycle
Chicago 2026 (distributed)	RSA-2048	∼M	∼ 1 yr	modular SC with interconnects
this tutorial, Part VIII	ECC-256 / RSA-2048	2.14 M / 898 k	15 min / 5 d	Gidney 2025's model; circuits of Parts VII–VIII

Compiled from [50] (Figures 285–286) and the papers cited; Google's 2026 elliptic-curve estimate [36], the surface-code reference for ECC-256, is compared with Part VII's circuits there rather than here. All assume 99.9% fidelities (99.99% for IonQ) uniformly at scale, independent Pauli noise, and a monolithic processor unless stated. The trend from 20 million qubits to under 10^5 in seven years came from arithmetic (Parts V–VIII), from codes (yoked, qLDPC), from magic states (cultivation) and from trading time for space; it did not come from hardware, which in the same period went from no logical qubit to one below-threshold memory at $d = 7$.

49.4 Forecasts

Mosca's inequality frames the question for a defender: data must stay secret for D years, migration to post-quantum cryptography takes T years, and a cryptographically relevant quantum computer arrives in G_c years; if $D + T > G_c$ the migration is already late. D and T are knowable; G_c is not. A 2020 statistical study of expert forecasts gave a 90% interval of 2039–2058 with a median of 2050 for a Shor attack on RSA-2048, and 2026–2033 for a proof-of-concept fault-tolerant machine; the yearly evolutionQ expert surveys give similar spreads; Shor's own guess was "between 20 and 40 years" with the possibility of never. Vendor roadmaps are more aggressive — IBM's Starling (200 logical qubits, 10^8 gates) for 2029 and Blue Jay (2,000 logical qubits, 10^9 gates) for 2033, Quantinuum's universal fault-tolerant Apollo for 2029, Microsoft's fault-tolerant machine for 2029 — and past NISQ roadmaps slipped by one to three years [50]. Against the numbers of this tutorial: ECDLP-256 needs about 1,600 logical qubits running 3.7×10^7 Toffolis with a 10^{-15} error per cycle, which is a Blue Jay-class machine with a hundredfold lower logical error than Starling's target and a 25 μs Toffoli; none of the 2026 demonstrations is within four orders of magnitude of it in any single dimension.

The physics, in one line each

A qubit is a two-level system with a Hamiltonian we can switch; a gate is a rotation, produced by a resonant drive whose phase picks the axis and whose area picks the angle; an entangling gate is an interaction switched on for a time, and it is the slowest and dirtiest operation on every platform. An error is a channel on the density operator, and a stabiliser measurement projects its continuum onto Paulis with a syndrome, which is why a code can correct it. Below a threshold near 1% the surface code's logical error falls as $(p/p_{\mathrm{th}})^{(d+1)/2}$, so the physical error rate is the one number that matters exponentially. A T gate is teleported from a magic state that a factory makes in ~ 150 rounds, and six factories give one Toffoli per $25\,\mu$s on a $1\,\mu$s-cycle machine. In 2026 the hardware had one below-threshold memory at $d = 7$ and 98 ions at 99.92%; the estimates need $d = 25$ on a thousand patches.

References

[1] P. W. Shor, *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*, SIAM Review **41**(2):303–332, 1999 (SIAM J. Comput. **26**(5):1484–1509, 1997). arXiv:quant-ph/9508027.

[2] A. Yu. Kitaev, *Quantum measurements and the Abelian stabilizer problem*, 1995. arXiv:quant-ph/9511026.

[3] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca, *Quantum algorithms revisited*, Proc. R. Soc. Lond. A **454**:339–354, 1998. arXiv:quant-ph/9708016.

[4] V. Vedral, A. Barenco, and A. Ekert, *Quantum networks for elementary arithmetic operations*, Phys. Rev. A **54**:147–153, 1996. arXiv:quant-ph/9511018.

[5] D. Beckman, A. N. Chari, S. Devabhaktuni, and J. Preskill, *Efficient networks for quantum factoring*, Phys. Rev. A **54**:1034–1063, 1996. arXiv:quant-ph/9602016.

[6] C. Zalka, *Fast versions of Shor's quantum factoring algorithm*, 1998. arXiv:quant-ph/9806084.

[7] T. G. Draper, *Addition on a quantum computer*, 2000. arXiv:quant-ph/0008033.

[8] S. Beauregard, *Circuit for Shor's algorithm using* $2n + 3$ *qubits*, Quantum Inf. Comput. **3**(2):175–185, 2003. arXiv:quant-ph/0205095.

[9] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, *A new quantum ripple-carry addition circuit*, 2004. arXiv:quant-ph/0410184.

[10] A. Pavlidis and D. Gizopoulos, *Fast quantum modular exponentiation architecture for Shor's factoring algorithm*, Quantum Inf. Comput. **14**(7–8):649–682, 2014. arXiv:1207.0511.

[11] A. Barenco, A. Ekert, K.-A. Suominen, and P. Törmä, *Approximate quantum Fourier transform and decoherence*, Phys. Rev. A **54**:139–146, 1996. arXiv:quant-ph/9601018.

[12] D. Cheung, *Improved bounds for the approximate QFT*, 2004. arXiv:quant-ph/0403071.

[13] R. B. Griffiths and C.-S. Niu, *Semiclassical Fourier transform for quantum computation*, Phys. Rev. Lett. **76**:3228–3231, 1996. arXiv:quant-ph/9511007.

[14] M. Mosca and A. Ekert, *The hidden subgroup problem and eigenvalue estimation on a quantum computer*, Proc. QCQC'98, LNCS 1509, 1999. arXiv:quant-ph/9903071.

[15] S. Parker and M. B. Plenio, *Efficient factorization with a single pure qubit and* $\log N$ *mixed qubits*, Phys. Rev. Lett. **85**:3049–3052, 2000. arXiv:quant-ph/0001066.

[16] C. Gidney, *Halving the cost of quantum addition*, Quantum **2**:74, 2018. arXiv:1709.06648.

[17] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, and H. Neven, *Encoding electronic spectra in quantum circuits with linear T complexity*, Phys. Rev. X **8**:041015, 2018. arXiv:1805.03662.

[18] C. Zalka, *Shor's algorithm with fewer (pure) qubits*, 2006. arXiv:quant-ph/0601097. — the coset representation of modular integers.

[19] C. Gidney and M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum **5**:433, 2021. arXiv:1905.09749.

[20] C. Gidney, *Approximate encoded permutations and piecewise quantum adders*, 2019. arXiv:1905.08488.

[21] C. Gidney, *Windowed quantum arithmetic*, 2019. arXiv:1905.07682.

[22] C. Chevignard, P.-A. Fouque and A. Schrottenloher, *Reducing the number of qubits in quantum factoring*, CRYPTO 2025, LNCS pp. 384–415; IACR ePrint 2024/222. eprint.iacr.org/2024/222.

[23] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Ed., Cambridge University Press, 2010.

[24] L. M. K. Vandersypen, M. Steffen, G. Breyta, C. S. Yannoni, M. H. Sherwood, and I. L. Chuang, *Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance*, Nature **414**:883–887, 2001.

[25] *Building Shor's algorithm from scratch: factoring 15*, Jupyter notebook (Qiskit), after [7, 8], Nielsen–Chuang [23], and the Qiskit community tutorials.

[26] B. Assel, *qiskit-shor: full implementation of Shor's factoring algorithm with the Qiskit SDK*, GitHub repository, 2025–2026. https://github.com/benjamin-assel/qiskit-shor.

[27] B. Assel, *Where are we with Shor's algorithm?*, Towards Data Science, 2025. https://towardsdatascience.com/where-are-we-with-shors-algorithm/.

[28] J. Proos and C. Zalka, *Shor's discrete logarithm quantum algorithm for elliptic curves*, Quantum Inf. Comput. **3**(4):317–344, 2003.

[29] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, ASIACRYPT 2017, LNCS 10625, pp. 241–270.

[30] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, and M. Soeken, *Improved quantum circuits for elliptic curve discrete logarithms*, PQCrypto 2020, LNCS 12100, pp. 425–444.

[31] H. Kim, K. Jang, S. Wang, V. Srivastava, A. Baksi, G. Song, H. Seo, and A. Chattopadhyay, *New quantum circuits for ECDLP: breaking prime elliptic curve cryptography*, IACR ePrint 2026/106. https://eprint.iacr.org/2026/106.

[32] A. Schrottenloher, *Optimized point addition circuits for elliptic curve discrete logarithms*, IACR ePrint 2026/1128. https://eprint.iacr.org/2026/1128.

[33] J. Long et al., *ECDSA.Fail: Open Autoresearch for Optimizing Elliptic-Curve Point Addition in Shor's Algorithm*, arXiv:2609.09582, 2026. https://arxiv.org/abs/2609.09582.

[34] A. Gu, A. H. Kemeny, S. F. Yelin, and M. D. Lukin, *Discovering state-of-the-art quantum circuits with large language models*, arXiv:2610.22367, 2026. https://arxiv.org/abs/2610.22367.

[35] H. Luo, Z. Yang, J. Luo, Z. Wang, Y. Su, X. Sun, L. Li, and T. Li, *Quantum algorithm for elliptic curve discrete logarithms with space-efficient point addition*, arXiv:2607.13816, 2026. https://arxiv.org/abs/2607.13816.

[36] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, and D. Boneh, *Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations*, arXiv:2603.28846, 2026.

[37] T. Khattar, N. Shutty, C. Gidney, A. Zalcman, N. Yosri, D. Maslov, R. Babbush, and S. P. Jordan, *Verifiable quantum advantage via optimized DQI circuits*, arXiv:2510.10967, 2025.

[38] M. Ekerå, *Revisiting Shor's quantum algorithm for computing general discrete logarithms*, arXiv:1905.09084, 2019.

[39] C. Gidney, *A classical-quantum adder with constant workspace and linear gates*, arXiv:2507.23079, 2025.

[40] T. Häner, M. Roetteler, and K. M. Svore, *Factoring using $2n + 2$ qubits with Toffoli based modular multiplication*, Quantum Inf. Comput. **17**(7–8):673–684, 2017. arXiv:1611.07995.

[41] T. Häner, F. Tripier, J. Young, M. Naehrig, et al. (IonQ), *Computing 256-bit elliptic curve discrete logarithms in 26 days on a fault-tolerant trapped-ion quantum computer with 20,000 qubits*, arXiv:2609.05625, 2026.

[42] T. G. Draper, S. A. Kutin, E. M. Rains, and K. M. Svore, *A logarithmic-depth quantum carry-lookahead adder*, Quantum Inf. Comput. **6**(4–5):351–369, 2006. arXiv:quant-ph/0406142.

[43] C. Gidney, *How to factor 2048 bit RSA integers with less than a million noisy qubits*, arXiv:2505.15917, 2025.

[44] D. Litinski, *How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates*, arXiv:2306.08585, 2023.

[45] G. H. Low, V. Kliuchnikov, and L. Schaeffer, *Trading T-gates for dirty qubits in state preparation and unitary synthesis*, 2018. arXiv:1812.00954.

[46] C. Gidney, *Constructing large increment gates*, 2015. https://algassert.com/circuits/2015/06/12/Constructing-Large-Increment-Gates.html.

[47] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, *Applying Grover's algorithm to AES: quantum resource estimates*, PQCrypto 2016, LNCS 9606, pp. 29–43.

[48] S. Josefsson and I. Liusvaara, *Edwards-Curve Digital Signature Algorithm (EdDSA)*, RFC 8032, January 2017, §5.1.4. https://www.rfc-editor.org/rfc/rfc8032#section-5.1.4.

[49] Classiq, *Elliptic curve discrete logarithm* (tutorial notebook), 2025–2026. https://docs.classiq.io/explore/algorithms/number_theory_and_cryptography/elliptic_curves/elliptic_curve_discrete_log.

[50] O. Ezratty, *Understanding Quantum Technologies 2026*, 8th edition, September 2026, about 1,800 pages. Chapters 3 (quantum physics 101), 4 (gate-based computing), 5.2 (error handling), 6 (hardware), 7 (enabling technologies), 9.3 (Shor), 10.5 (benchmarking) and 12.1 (threats).

[51] R. Acharya et al. (Google Quantum AI), *Quantum error correction below the surface code threshold*, Nature **638**:920–926, 2025. arXiv:2408.13687.

[52] A. Ransford, M. S. Allman, et al. (Quantinuum), *Helios: a 98-qubit trapped-ion quantum computer with all-to-all connectivity*, Nature, 2026.

[53] H. J. Manetsch, G. Nomura, E. Bataille, K. H. Leung, X. Lv, and M. Endres, *A tweezer array with 6100 highly coherent atomic qubits*, Nature, 2025. arXiv:2403.12021.

[54] S. J. Evered, D. Bluvstein, M. Kalinowski, et al., *High-fidelity parallel entangling gates on a neutral-atom quantum computer*, Nature **622**:268–272, 2023. arXiv:2304.05420.

[55] D. Bluvstein, S. J. Evered, A. A. Geim, et al., *Logical quantum processor based on reconfigurable atom arrays*, Nature **626**:58–65, 2024. arXiv:2312.03982.

[56] J. Guillaud and M. Mirrahimi, *Repetition cat qubits for fault-tolerant quantum computation*, Phys. Rev. X **9**:041053, 2019. arXiv:1904.09474.

[57] É. Gouzien, D. Ruiz, F.-M. Le Régent, J. Guillaud, and N. Sangouard, *Performance analysis of a repetition cat code architecture: computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits*, Phys. Rev. Lett. **131**:040602, 2023. arXiv:2302.06639.

[58] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, *Surface codes: towards practical large-scale quantum computation*, Phys. Rev. A **86**:032324, 2012. arXiv:1208.0928.

[59] C. Gidney, N. Shutty, and C. Jones, *Magic state cultivation: growing T states as cheap as CNOT gates*, arXiv:2409.17595, 2024.

[60] Iceberg Quantum, *Pinnacle* architecture preprint (RSA-2048 with quantum LDPC codes), arXiv:2602.11457, 2026.